

These are the slides for the C Programming For Beginners - Master the C Language on Udemy. They are provided free of charge to all students.

More information about the course: <https://lpa.dev/u1cpfb>

If you have any questions or queries, please add your feedback in the Q&A section of the course on Udemy.

Best regards,

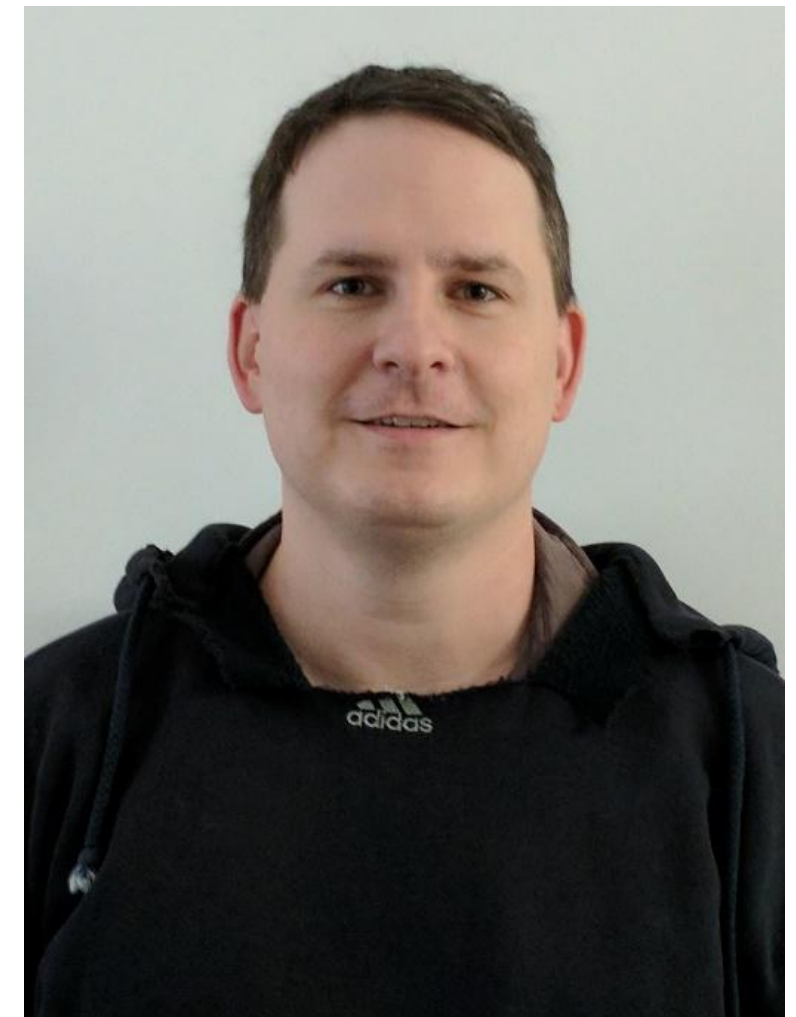
Tim Buchalka
Learn Programming Academy

Welcome to Class!

Jason Fedin

A little bit about myself

- Masters of Science (Computer Science) Binghamton University
- Software Developer (16 Years)
- Online Instructor (11 years)
 - Instructed over 20 different classes, everything from Object-Oriented Programming to Compiler Theory
- Working with the C programming language for over 16 years
- Created beginner C and advanced classes at multiple universities
- Software Developer at Xerox Corporation
- Mainly concentrate on Object Oriented Languages and Mobile Programming
- Focus on writing High Quality Code



Topics

- Overview of C – efficient, portable, power and flexibility, programmer oriented
- Language Features – imperative language, top-down planning, structured programming and modular design
- Advantages of using C – small, fast programs, reliable, easy to learn and understand
- How to use a modern, cross-platform Integrated Development Environment to write, edit, and debug your C code
- Basic C concepts – structure of a program, comments, output using printf, “Hello World”
- Makefiles – how to build
- Variables – declaring and using
- Data Types – int, float, double, char, etc (as well as enums and type definitions)

Topics (cont'd)

- Basic Operators – logical, arithmetic and assignment
- Conditional Statements – making decisions (if, switch)
- Repeating code – looping (for, while, and do-while)
- Arrays – defining and initializing, multi-dimensional
- Functions – declaration and use, arguments and parameters, call by value vs. call by reference
- Debugging – call stack, common mistakes, understanding compiler messages
- Structs – initializing, nested structures, variants
- Character Strings – basics, arrays of chars, character operations

Topics (cont'd)

- Pointers – definition and use, using with functions and arrays, malloc, pointer arithmetic
- The Preprocessor - #define, #include
- Input and Output – getchar, scanf
- File Input/Output – reading and writing to a file, file operations
- Standard C Library – string functions, math functions, utility functions, standard header files

Course Outcomes

- You will be able to write beginner C programs
- You will be able to write efficient, high quality C code
 - modular
 - low coupling
- You will be able to find and fix your errors
 - Understand compiler messages
- You will understand fundamental aspects of the C Programming language
- You will have fun!

Class Organization

Jason Fedin

Class

Organization

- Lectures are designed around explaining the why and providing the how

Class

Organization

- Lectures are designed around explaining the why and providing the how
 - A complete learning experience
 - Understand the programming language as a whole
 - Lacking in most Udemy courses

Class

Organization

- Lectures are designed around explaining the why and providing the how
 - A complete learning experience
 - Understand the programming language as a whole
 - Lacking in most Udemy courses
- Powerpoint slides

Class

Organization

- Lectures are designed around explaining the why and providing the how

- A complete learning experience
- Understand the programming language as a whole
- Lacking in most Udemy courses

- Powerpoint slides

- a way to present abstract concepts and definitions that I am not able to demonstrate in code (as easily)
- Not just reading from slides!
 - Providing insight via experience
 - Providing thorough explanations

Demonstrations of code

Demonstrations of code

- Demonstrations in the IDE (Integrated Development Environment)
 - Code examples (concrete, real-world)
 - More practical/hand-on learning

Demonstrations of code

- Demonstrations in the IDE (Integrated Development Environment)
 - Code examples (concrete, real-world)
 - More practical/hand-on learning
- Challenges (coding assignments/projects)
 - A way for you to assess your own learning
 - Code alongs – walking through the code of a solution for a particular challenge
 - All source code provided in class

Fundamentals of a program

Jason Fedin

Basics

- Computers are very dumb machines
 - they only do what they are told to do

Basics

- Computers are very dumb machines
 - they only do what they are told to do
- The basic operations of a computer will form what is known as the computer's instruction set

Basics

- Computers are very dumb machines
 - they only do what they are told to do
- The basic operations of a computer will form what is known as the computer's instruction set
- To solve a problem using a computer, you must provide a solution to the problem by sending instructions to the instruction set
 - a computer program sends the instructions necessary to solve a specific problem

Basics

- The approach or method that is used to solve the problem is known as an algorithm
 - So, if we were to create a program that tests if a number is odd or even
 - The statements that solve the problem becomes the program
 - The method that is used to test if the number is even or odd is the algorithm

Basics

- The approach or method that is used to solve the problem is known as an algorithm
 - So, if we were to create a program that tests if a number is odd or even
 - The statements that solve the problem becomes the program
 - The method that is used to test if the number is even or odd is the algorithm
- To write a program, you need to write the instructions necessary to implement the algorithm
 - these instructions would be expressed in the statements of a particular computer language, such as Java, C++, Objective-C, or C

Terminology

- CPU (central processing unit)
 - does most of the computing work
 - Instructions are executed here

Terminology

- CPU (central processing unit)
 - does most of the computing work
 - Instructions are executed here
- RAM (random access memory)
 - Stores the data of a program while it is running

Terminology

- CPU (central processing unit)
 - does most of the computing work
 - Instructions are executed here
- RAM (random access memory)
 - Stores the data of a program while it is running
- hard drive (permanent storage)
 - Stores files that contain program source code, even while the computer is turned off

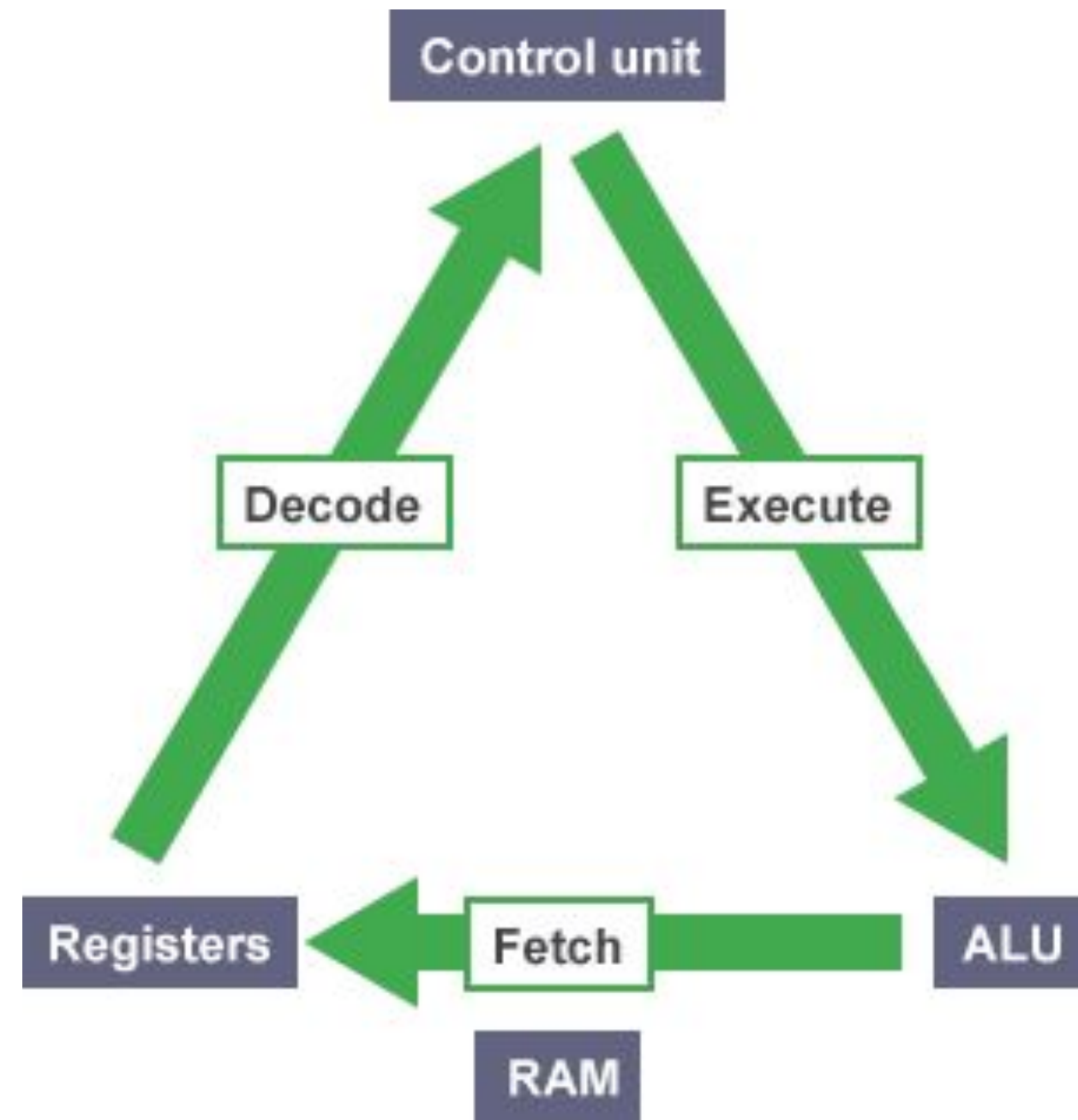
Terminology

- Operating System
 - developed to help make it more convenient to use computers
 - a program that controls the entire operation of a computer
 - All input and output
 - manages the computer's resources and handles the execution of programs
 - Windows, Unix, Android, etc.

Terminology

- Operating System
 - developed to help make it more convenient to use computers
 - a program that controls the entire operation of a computer
 - All input and output
 - manages the computer's resources and handles the execution of programs
 - Windows, Unix, Android, etc.
- fetch / Execute Cycle (life of a CPU)
 - fetches an instruction from memory (using registers) and executes it (loop)
 - A gigahertz CPU can do this about a billion times a second

Fetch / Execute Cycle



<https://www.bbc.co.uk/education/guides/z2342hv/revision/5>

Higher Level Programming Languages

Higher Level Programming

Languages

High-level programming languages make it easier to write programs

Higher Level Programming

Languages

High-level programming languages make it easier to write programs

- Opposite of assembly language

Higher Level Programming

Languages

High-level programming languages make it easier to write programs

- Opposite of assembly language
- C is a higher level programming language that describes actions in a more abstract form

Higher Level Programming

Languages

High-level programming languages make it easier to write programs

- Opposite of assembly language
- C is a higher level programming language that describes actions in a more abstract form
- the instructions (statements) of a program look more like problem solving steps

Higher Level Programming

Languages

High-level programming languages make it easier to write programs

- Opposite of assembly language
- C is a higher level programming language that describes actions in a more abstract form
- the instructions (statements) of a program look more like problem solving steps
- do not have to worry about the precise steps a particular CPU would have to take to accomplish a particular task
 - `total = x +` vs. `mv ax, 5, mv cx 4, etc.....`

Higher Level Programming Languages

Higher Level Programming

Compilers Languages

- a program that translates the high-level language source code into the detailed set of machine language instructions the computer requires
- the program does the high-level thinking and the compiler generates the tedious instructions to the cpu

Higher Level Programming

Compilers Languages

- a program that translates the high-level language source code into the detailed set of machine language instructions the computer requires
- the program does the high-level thinking and the compiler generates the tedious instructions to the cpu
- Compilers will also check that your program has valid syntax for the programming language that you are compiling
 - finds errors and it reports them to you and doesn't produce an executable until you fix them

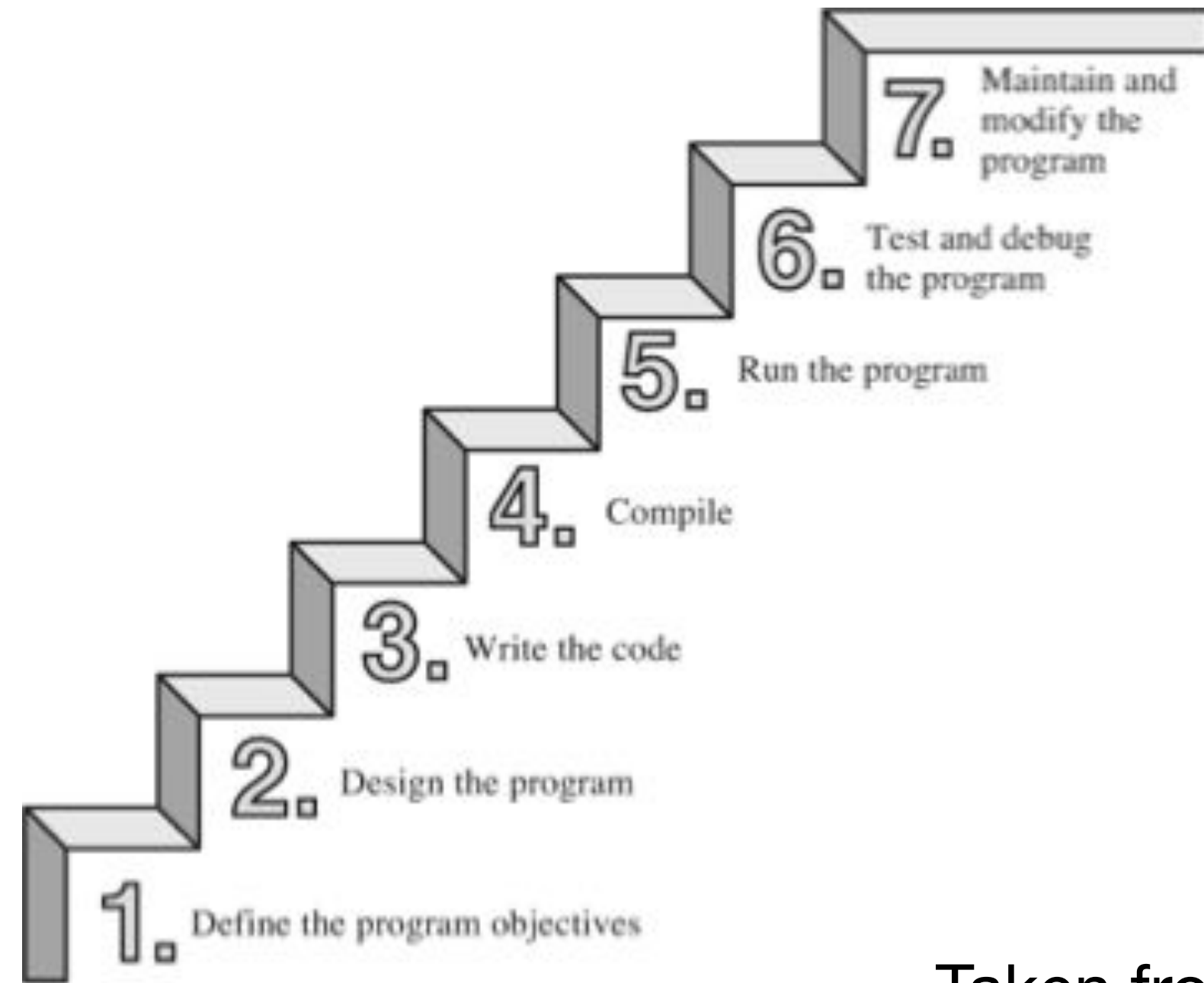
Higher Level Programming

Compilers Languages

- a program that translates the high-level language source code into the detailed set of machine language instructions the computer requires
- the program does the high-level thinking and the compiler generates the tedious instructions to the cpu
- Compilers will also check that your program has valid syntax for the programming language that you are compiling
 - finds errors and it reports them to you and doesn't produce an executable until you fix them
- high-level languages are easier to learn and much easier to program in than are machine languages

Writing a program

the act of writing a C program can be broken down into multiple steps



Taken from “C Primer Plus”, Prata

Steps in writing a

program

1. Define the Program Objectives

- Understand the requirements of the program
- Get a clear idea of what you want the program to accomplish

Steps in writing a

program

1. Define the Program Objectives

- Understand the requirements of the program
- Get a clear idea of what you want the program to accomplish

2. Design

- Decide how the program will meet the above requirements
- What should the user interface be like?
- How should the program be organized?

Steps in writing a

program

1. Define the Program Objectives

- Understand the requirements of the program
- Get a clear idea of what you want the program to accomplish

2. Design

- Decide how the program will meet the above requirements
- What should the user interface be like?
- How should the program be organized?

3. Write the Code

- Start implementation, translate the design in the syntax of C
- you need to use a text editor to create what is called a *source code* file

Steps in writing a

program

4. Compile

- translate the source code into machine code (executable code)
- consists of detailed instructions to the CPU expressed in a numeric code

Steps in writing a

4. Compile program

- translate the source code into machine code (executable code)
- consists of detailed instructions to the CPU expressed in a numeric code

5. Run the Program

- the executable file is a program you can run

Steps in writing a

4. Compile program

- translate the source code into machine code (executable code)
- consists of detailed instructions to the CPU expressed in a numeric code

5. Run the Program

- the executable file is a program you can run

6. Test and Debug

- Just because a program is running, does not mean it works as intended
- Need to test, to see that your program does what it is supposed to do (may find bugs)
 - Debugging is the process of finding and fixing program errors
 - Making mistakes is a natural part of learning

Steps in writing a

7 Maintain and Modify the Program

- Programs are released and used by many people
- have to continue to fix new bugs or add new features

Steps in writing a

program

7 Maintain and Modify the Program

- Programs are released and used by many people
 - have to continue to fix new bugs or add new features
-
- For the above steps, you may have to jump around steps and repeat steps
 - E.g. when you are writing code, you might find that your plan was impractical

Steps in writing a program

Many new programmers ignore steps 1 and 2 and go directly to writing code

- A big mistake for larger programs, may be ok for very simple programs
- the larger and more complex the program is, the more planning it requires
- should develop the habit of planning before coding

Steps in writing a program

Many new programmers ignore steps 1 and 2 and go directly to writing code

- A big mistake for larger programs, may be ok for very simple programs
 - the larger and more complex the program is, the more planning it requires
 - should develop the habit of planning before coding
-
- Also, while you are coding, you always want to work in small steps and constantly test (divide and conquer)

Overview

Jason Fedin

Overview

- W**C is a general-purpose, imperative computer programming language that supports structured programming
- Uses statements that change a program's state, focuses on how
 - Currently, it is one of the most widely used programming languages of all time
 - C is a modern language
 - has most basic control structures and features of modern languages
 - designed for top-down planning
 - organized around the use of functions (modular design) structured programming
 - a very reliable and readable program

Overview

- **W**C is used on everything from minicomputers, Unix/Linux systems to pc's and mainframes
- C is the preferred language for producing word processing programs, spreadsheets and compilers
- C has become popular for programming embedded systems
- used to program microprocessors found in automobiles, cameras, DVD players, etc

Overview

(cont'd)

C has and continues to play a strong role in the development of Linux

- C programs are easy to modify and easy to adapt to new models or languages
- In the 1990s, many software houses began turning to the C++ language for large programming projects
- C is a subset of C++ with object-oriented programming tools added
- any C program is a valid C++ program
- By learning C, you also learn much of C++
- C remains a core skill needed by corporations and ranks in the top 10 of desired skills

Overview

(cont'd)

- C provides constructs that map efficiently to typical machine instructions and thus is used by programs that were previously implemented in assembly language
- provides low-level access to memory (has many low-level capabilities)
- requires minimal run-time support
- was invented in 1972 by Dennis Ritchie of Bell Laboratories
- Ritchie was working on the design of the UNIX operating System
- was created as a tool for working programmers
- main goal is to be a useful language
- Easy readability and writability

Histor

- Y**C initially became widely known as the development language of the UNIX operating system
- virtually all new major operating systems are written in C and/or C++
 - C evolved from a previous programming language named B
 - uses many of the important concepts of B while adding data typing and other powerful features
 - B was a “typeless” language—every data item occupied one “word” in memory, and the burden of typing variables fell on the shoulders of the programmer
 - C is available for most computers
 - C is also hardware independent

History

(cont'd)

By the late 1970s, C had evolved into what is now referred to as “traditional C”

- The rapid expansion of C working on many different hardware platforms led to many variations that were similar but often incompatible
- a standard version of C was created (C89/C90, C99, C11)
- A program written only in Standard C and without any hardware-dependent assumptions will run correctly on any platform with a standard C compiler
- Non-standard C programs may run only on a certain platform or with a particular compiler

History

(cont'd)

• C99 is supported by current C compilers

- most C code being written today is based on it

- C99 is a revised standard for the C programming language that refines and expands the capabilities of C

- has not been widely adopted and not all popular C compilers support it

Standard

Sthe current standard is commonly referred to as C11

- some elements of the language as defined by C11 are optional
- also possible that a C11 compiler may not implement all of the language features mandated by the C11 standard
- this class will base its examples and concepts off C11
- C is one of the most important and popular programming languages
- It has grown because people try it and like it
- In the past decade or two, many have moved from C to languages such as C++, Objective C, and Java
- C is still an important language in its own right, as well a migration path to these others

Language Features

Jason Fedin

Overview

WC produces compact and efficient programs

- C is one of the most important programming languages and will continue to be so
- The main features of C are the following
 - Efficient
 - Portable
 - Powerful and Flexible
 - Programmer Oriented

Efficiency and

Portability

• C is an efficient language

- takes advantage of the capabilities of current computers
- C programs are compact and fast (similar to assembly language programs)
- programmers can fine-tune their programs for maximum speed or most efficient use of memory
- C is a portable language
- C programs written on one system can be run on other systems with little or no modification
- a leader in portability
- compilers are available for many computer architectures
- Linux/Unix systems typically come with a C compiler as part of the package
- compilers are available for personal computers
- A good chance that you can get a C compiler for whatever device you are using

Power and

Flexibility

the Linux kernel is written in C

- many compilers and interpreters for other languages (FORTRAN, Perl, Python, Pascal, LISP, Logo, and BASIC) have been written in C
- C programs have been used for solving physics and engineering problems and even for animating special effects for movies
- C is flexible
- used for developing just about everything you can imagine by way of a computer program
- accounting applications to word processing and from games to operating systems
- it is the basis for more advanced languages, such as C++
- It is also used for developing mobile phone apps in the form of Objective C
- C is easy to learn because of its compactness
- is an ideal first language to learn if you want to be a programmer
- you will acquire sufficient knowledge for practical application development quickly and easily

Programmer

Oriented

C fulfills the needs of programmers

- gives you access to hardware
- enables you to manipulate individual bits in memory
- C contains a large selection of operators which allows you to express yourself succinctly
- C is less strict than most languages in limiting what you can do
- Can be both an advantage and a danger
- advantage is that many tasks (such as converting forms of data) are easier in C
- danger is that you can make mistakes (pointers) that are impossible in some languages
- C gives you more freedom, but it also puts more responsibility on you
- C implementations have a large library of useful C functions
- deal with common needs of most programmers

Other

Features

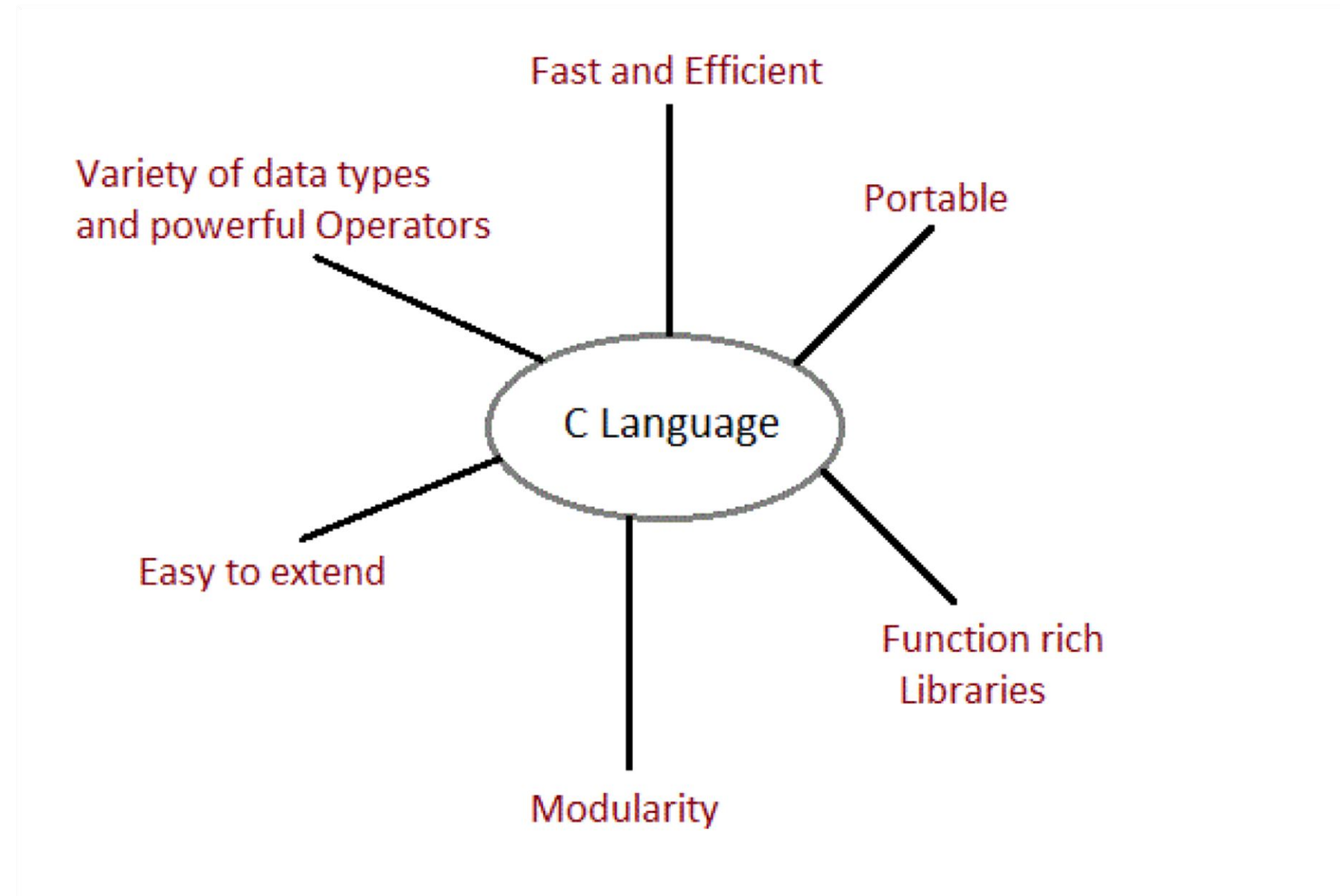
- provides low level features that are generally provided by the Lower level languages
- programs can be manipulated using bits
- Ability to manage memory representation at bit level
- provides wide variety of bit manipulation operators
- pointers play a big role in C
- direct access to memory
- Supports efficient use of pointers

Disadvantage

SFlexibility and freedom also requires added responsibility

- use of pointers is problematic and abused
- you can make programming errors that are difficult to trace
- Sometimes because of its wealth of operators and its conciseness, it makes the language difficult to read and follow
- there is an opportunity to write obscure code

Summary



<http://www.studytonight.com/c/features-of-c.php>

Creating a C program

Jason Fedin

Overview

Where are four fundamental tasks in the creation of any C program

- Editing
 - Compiling
 - Linking
 - Executing
-
- These tasks will become second nature to you because you will be doing it so often
 - The processes of editing, compiling, linking, and executing are essentially the same for developing programs in any environment and with any compiled language
 - Editing is the process of creating and modifying your C source code
 - source code is inside a file and contains the program instructions you write
 - choose a wise name for your base file name (all source files end in the .c extension)
 - we will use an IDE (code blocks) for this class, but, you can use any editor (notepad, etc) to create your source code

Compilin

g A compiler converts your source code into machine language and detects and reports errors in your code

- The input to the compiler is the file you produce during your editing (source file)
- Compilation is a two-stage process
- the first stage is called the preprocessing phase, during which your code may be modified or added to
- the second stage is the actual compilation that generates the object code
- the compiler examines each program statement and checks it to ensure that it conforms to the syntax and semantics of the language
- can also recognize structural errors (dead code)
- does not find logic errors
- typical errors reported might be due to an expression that has unbalanced parentheses (syntactic error), or due to the use of a variable that is not “defined” (semantic error)

Compiling

(cont'd)

- After all errors are fixed, the compiler will then take each statement of the program and translate it into assembly language
- the compiler will then translate the assembly language statements into actual machine instructions
 - the output from the compiler is known as object code and it is stored in files called object files (same name as source file with a .obj or .o extension)
 - The standard command to compile your C programs will be cc (or the GNU compiler, which is .gcc)
 - `cc -c myprog.c` or `gcc -c myprog.c`
 - if you omit the -c flag, your program will automatically be linked as well
 - In Codeblocks we will be using a menu option from within an IDE to compile

Linkin

9 After the program has been translated into object code, it is ready to be linked

- The purpose of the linking phase is to get the program into a final form for execution on the computer
- linking usually occurs automatically when compiling depending on what system you are on, but, can sometimes be a separate command
- The linker combines the object modules generated by the compiler with additional libraries needed by the program to create the whole executable
- also detects and reports errors
- if part of your program is missing or a nonexistent library component is referenced
- Program libraries support and extend the C language by providing routines to carry out operations that are not part of the language
- input and output libraries, mathematical libraries, string manipulation libraries

Linking

(cont'd)

• A failure during the linking phase means that once again you have to go back and edit your source code

- Success will produce an executable file
 - In a Windows environment, the executable file will have an .exe extension
 - in UNIX / Linux, there will be no such extension (a.out by default)
 - Many IDEs have a build option, which will compile and link your program in a single operation to produce the executable
-
- A program of any significant size will consist of several source code files
 - each source code file needs the compiler to generate the object file that need to be linked
-
- the program is much easier to manage by breaking it up into a number of smaller source files
 - It is cohesive and makes the development and maintenance of the program a lot easier
 - the set of source files that make up the program will usually be integrated under a project name, which is used to refer to the whole program

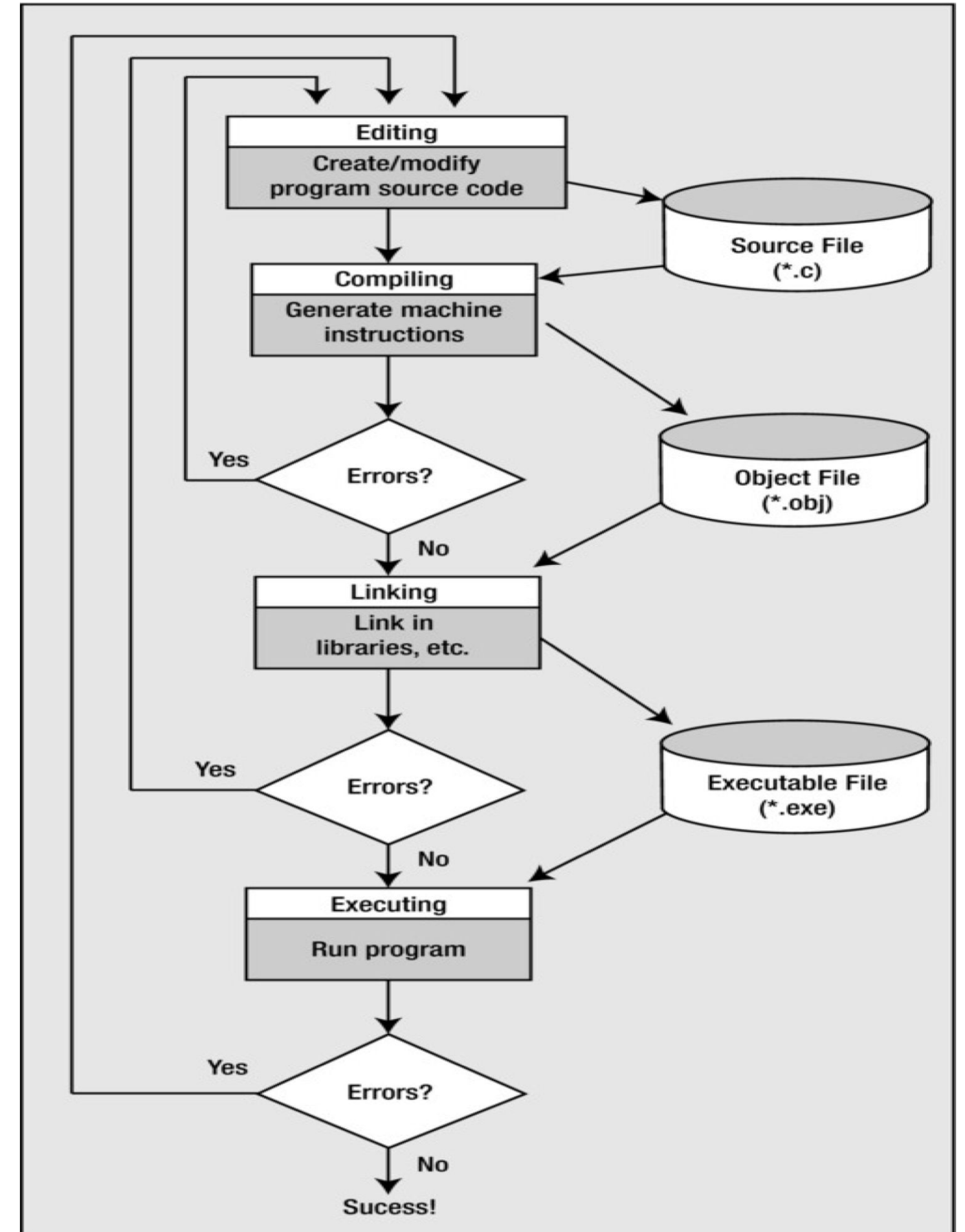
Executin

- g**In most IDEs, you'll find an appropriate menu command that allows you to run or execute your compiled program
- Otherwise double click the exe file or type a.out on the console in linux manually
 - The execution stage is where you run your program
 - each of the statements of the program is sequentially executed in turn
 - If the program requests any data from the user the program temporarily suspends its execution so that the input can be entered
 - Results that are displayed by the program (output) appear in a window called the console
 - this stage can also generate a wide variety of error conditions
 - producing the wrong output
 - just sitting there and doing nothing
 - crashing your computer
 - If the program does not perform the intended functionality then it will be necessary to go back and reanalyze the program's logic
 - known as the debugging phase, correct all the known problems or bugs from the program
 - Will need to make changes to the original source program
 - the entire process of compiling, linking, and executing the program must be repeated until the desired results are obtained

C

Stages

Taken from Beginning C, Horton



Installing Visual Studio Code

Jason Fedin

Visual Studio Code Download

- <https://code.visualstudio.com/>
- then, Install the c/c++ extension.
 - <https://marketplace.visualstudio.com/items?itemName=formulahendry.code-runner>
- Open your C code file in Text Editor, then use shortcut Ctrl+Alt+N or right click the Text Editor and then click Run Code
 - in context menu, the code will be compiled and run, and the output will be shown in the Output Window.

Overview

Jason Fedin

Overview

- Windows
 - C compiler (Cygwin)
 - IDE's (Integrated Development Environment)
 - CodeLite

Overview

- Mac
 - C compiler (developer tools)
 - IDE's (Integrated Development Environment)
 - CodeLite
 - xcode

Overview

- Linux
 - Compiler comes installed
 - IDE's (Integrated Development Environment)
 - CodeLite
 - vi, gedit, compile from command line

Installing the C Compiler

Jason Fedin

Overview

- you need to install the GNU gcc compiler, make, and gdb debugger from cygwin.com
- simulates a unix environment
- download either the 32 or 64 bit version of the setup file
- <https://cygwin.com/install.html>
- setup-x86_64.exe or setup-x86_64.exe

Overview

- run the downloaded Cygwin installer
 - accept the defaults until you reach the Select Your Internet Connection page
 - select the option on this page that is best for you. Click Next.
- on the Choose Download Site page, choose a download site you think might be relatively close to you. Click Next.
- on the Select Packages page you select the packages to download
 - Click the + next to Devel to expand the development tools category
 - You may want to resize the window so you can see more of it at one time
 - Select each package you want to download by clicking the Skip label next to it, which reveals the version number of the package to download
 - at a minimum, select
 - gcc-core: C compiler
 - gdb: The GNU Debugger
 - make: the GNU version of the 'make' utility
 - Packages that are required by the packages you select are automatically selected as well.
- Click Next to connect to the download site and download the packages you selected, and click Finish when the installation is complete

Setting your path up

- Open the Control Panel:
 - On Windows XP select Start > Settings > Control Panel and double-click System.
 - On Windows 7, type var in the Start menu's search box to quickly find a link to Edit the system environment variables
- Select the Advanced tab and click Environment Variables
- In the System Variables panel of the Environment Variables dialog, select the Path variable and click Edit
- Add the path to the cygwin-directory\bin directory to the Path variable, and click OK
 - By default, cygwin-directory is C:\cygwin (for 32 bit Cygwin distribution) or
 - C:\cygwin64 (for 64 bit Cygwin distribution)
 - Directory names must be separated with a semicolon
- Click OK in the Environment Variables dialog and the System Properties dialog.

Verifying the Installation



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\USX20620>gcc
gcc: fatal error: no input files
compilation terminated.

C:\Users\USX20620>gcc.exe
gcc.exe: fatal error: no input files
compilation terminated.

C:\Users\USX20620>gcc.exe help
gcc.exe: error: help: No such file or directory
gcc.exe: fatal error: no input files
compilation terminated.

C:\Users\USX20620>gcc.exe -help
gcc.exe: error: unrecognized command line option '-h'; did you mean '-h'?
gcc.exe: fatal error: no input files
compilation terminated.

C:\Users\USX20620>clear
'clear' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\USX20620>sysclear
'sysclear' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\USX20620>vi test.c
'vi' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\USX20620>gcc.exe
gcc.exe: fatal error: no input files
compilation terminated.

C:\Users\USX20620>
```


Installing Code::Blocks

Jason Fedin

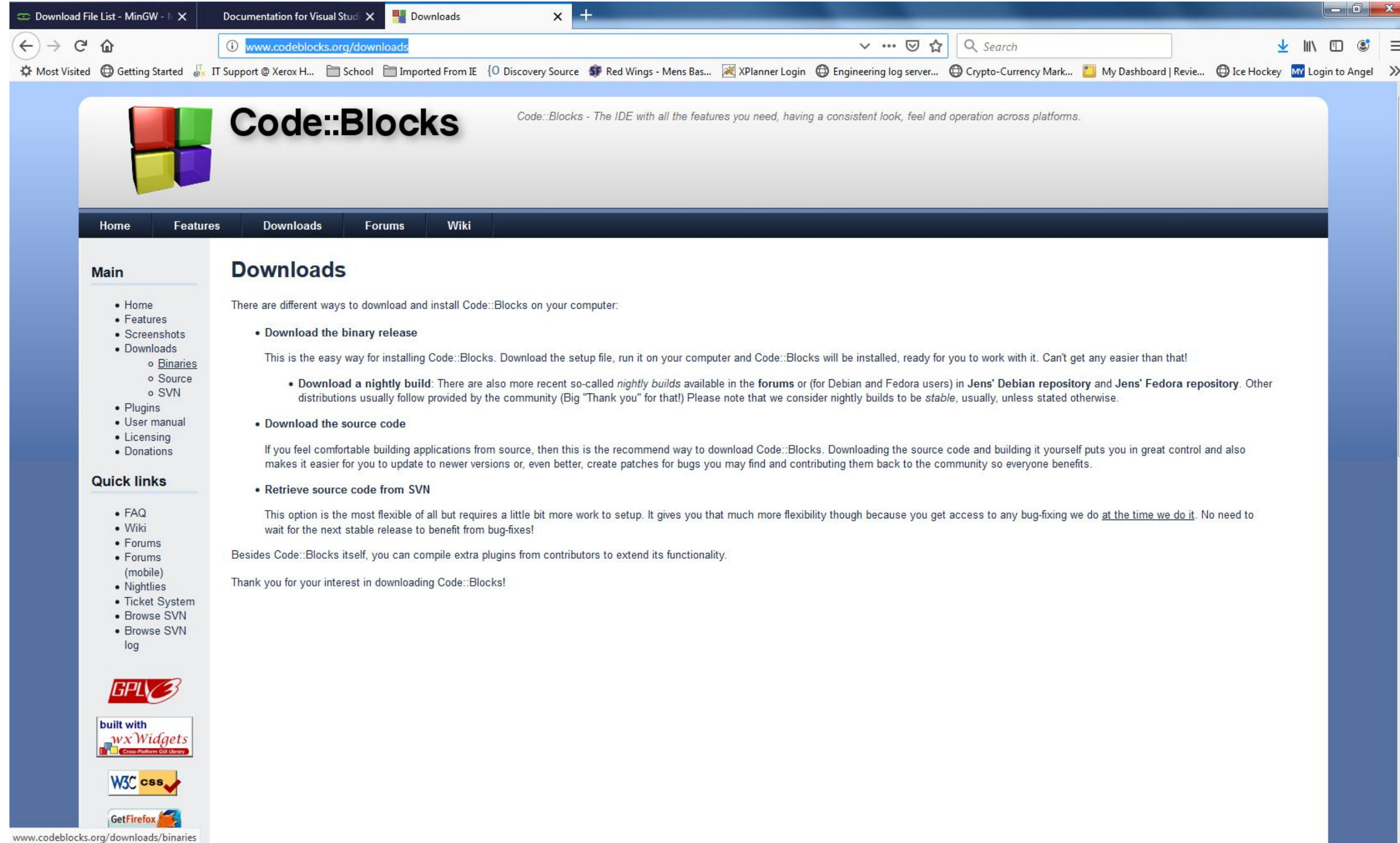
Overview

- <http://www.codeblocks.org/>

Watch:

https://www.youtube.com/watch?v=24_xtaAePDY

Topics



Topics (cont'd)

Download File List - MinGW - X

Documentation for Visual Studi

Download binary

www.codeblocks.org/downloads/binaries

Search

Most Visited

Getting Started

IT Support @ Xerox H...

School

Imported From IE

Discovery Source

Red Wings - Mens Bas...

XPlanner Login

Engineering log server...

Crypto-Currency Mark...

My Dashboard | Revie...

Ice Hockey

MY Login to Angel

Main

Home

Features

Screenshots

Downloads

- Binaries
 - Changelog
- Source
- SVN

Plugins

User manual

Licensing

Donations

Quick links

FAQ

Wiki

Forums

Forums (mobile)

Nightlies

Ticket System

Browse SVN

Browse SVN log

GPLv3

built with wxWidgets Cross-Platform GUI Library

W3C CSS

Get Firefox

SOURCEFORGE

MiSUMi

STANDARD & CONFIGURABLE FACTORY AUTOMATION COMPONENTS

Please select a setup package depending on your platform:

Windows XP / Vista / 7 / 8.x / 10

Linux 32 and 64-bit

Mac OS X

NOTE: For older OS'es use older releases. There are releases for many OS version and platforms on the Sourceforge.net page.

NOTE: There are also more recent nightly builds available in the forums or (for Debian and Fedora users) in Jens' Debian repository and Jens' Fedora repository. Please note that we consider nightly builds to be stable, usually.

NOTE: We have a Changelog for 17.12, that gives you an overview over the enhancements and fixes we have put in the new release.

Windows XP / Vista / 7 / 8.x / 10:

File	Date	Download from
codeblocks-17.12-setup.exe	30 Dec 2017	FossHUB or Sourceforge.net
codeblocks-17.12-setup-nonadmin.exe	30 Dec 2017	FossHUB or Sourceforge.net
codeblocks-17.12-nosetup.zip	30 Dec 2017	FossHUB or Sourceforge.net
codeblocks-17.12mingw-setup.exe	30 Dec 2017	FossHUB or Sourceforge.net
codeblocks-17.12mingw-nosetup.zip	30 Dec 2017	FossHUB or Sourceforge.net
codeblocks-17.12mingw_fortran-setup.exe	30 Dec 2017	FossHUB or Sourceforge.net

NOTE: The codeblocks-17.12-setup.exe file includes Code::Blocks with all plugins. The codeblocks-17.12-setup-nonadmin.exe file is provided for convenience to users that do not have administrator rights on their machine(s).

NOTE: The codeblocks-17.12mingw-setup.exe file includes additionally the GCC/G++ compiler and GDB debugger from TDM-GCC (version 5.1.0, 32 bit, SJLJ). The codeblocks-17.12mingw_fortran-setup.exe file includes additionally to that the GFortran compiler (TDM-GCC).

NOTE: The codeblocks-17.12(mingw)-nosetup.zip files are provided for convenience to users that are allergic against installers. However, it will not allow to select plugins / features to install (it includes everything) and not create any menu shortcuts. For the "installation" you are on your own.

If unsure, please use codeblocks-17.12mingw-setup.exe!

Linux 32 and 64-bit:

Distro	File	Date	Download from
	codeblocks_17.12-1_amd64_stable.tar.xz	06 Jan 2018	FossHUB or Sourceforge.net
	codeblocks_17.12-1_i386_stable.tar.xz	06 Jan 2018	FossHUB or Sourceforge.net

sourceforge.net/projects/codeblocks/files/Binaries/17.12/Windows/codeblocks-17.12-setup.exe

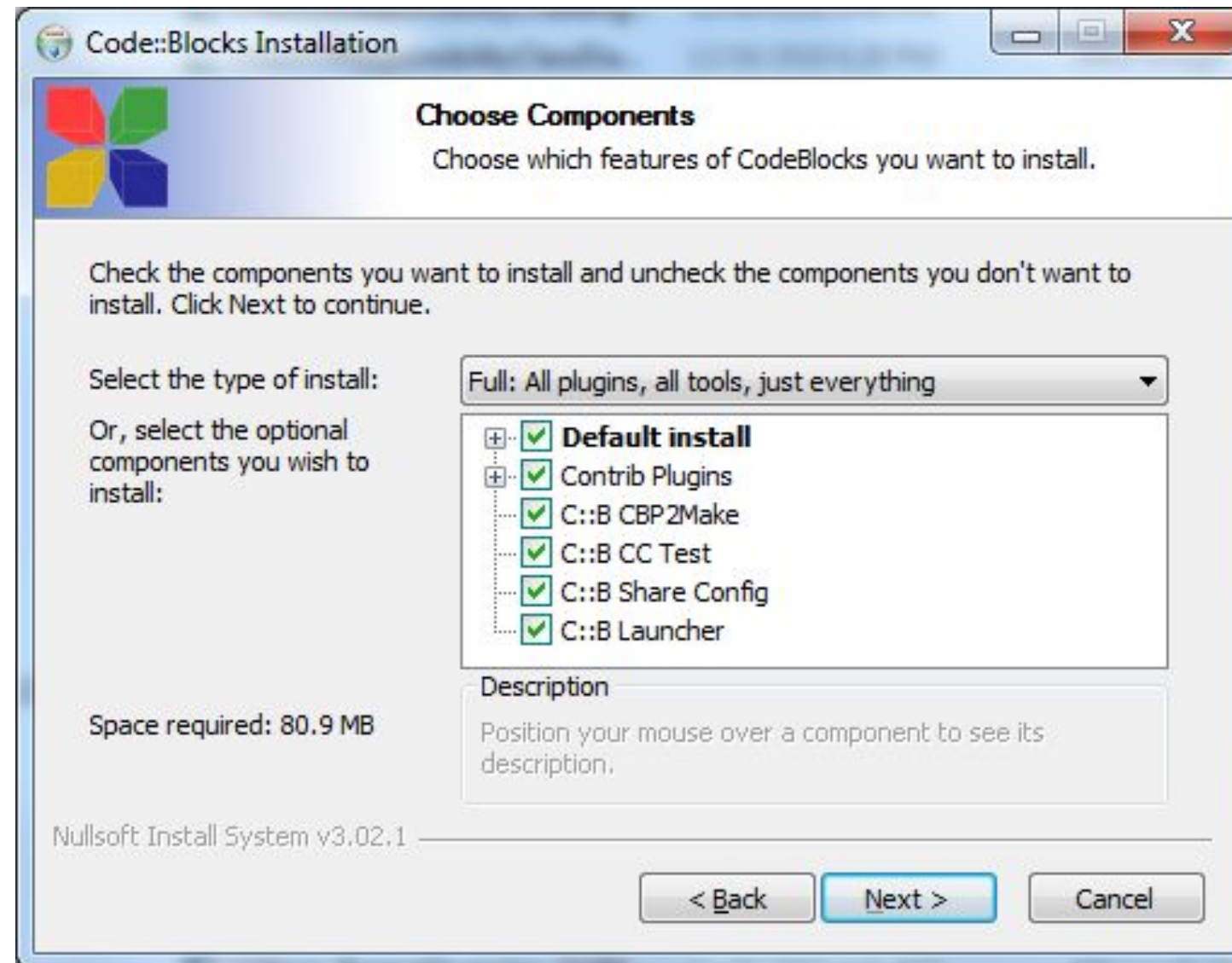
C FOR BEGINNERS

Installing Code::Blocks (Windows)

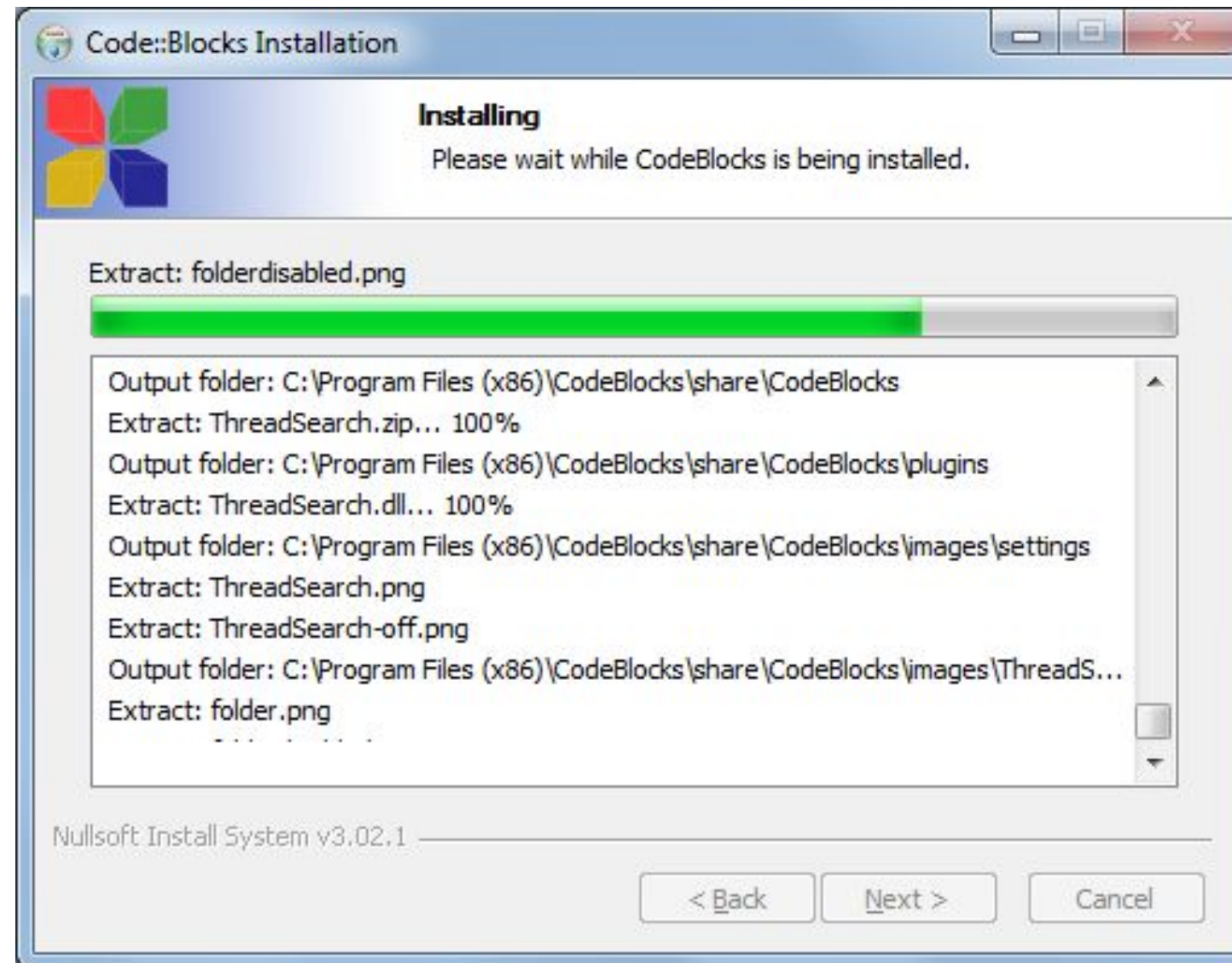
{LP} LearnProgramming

academy

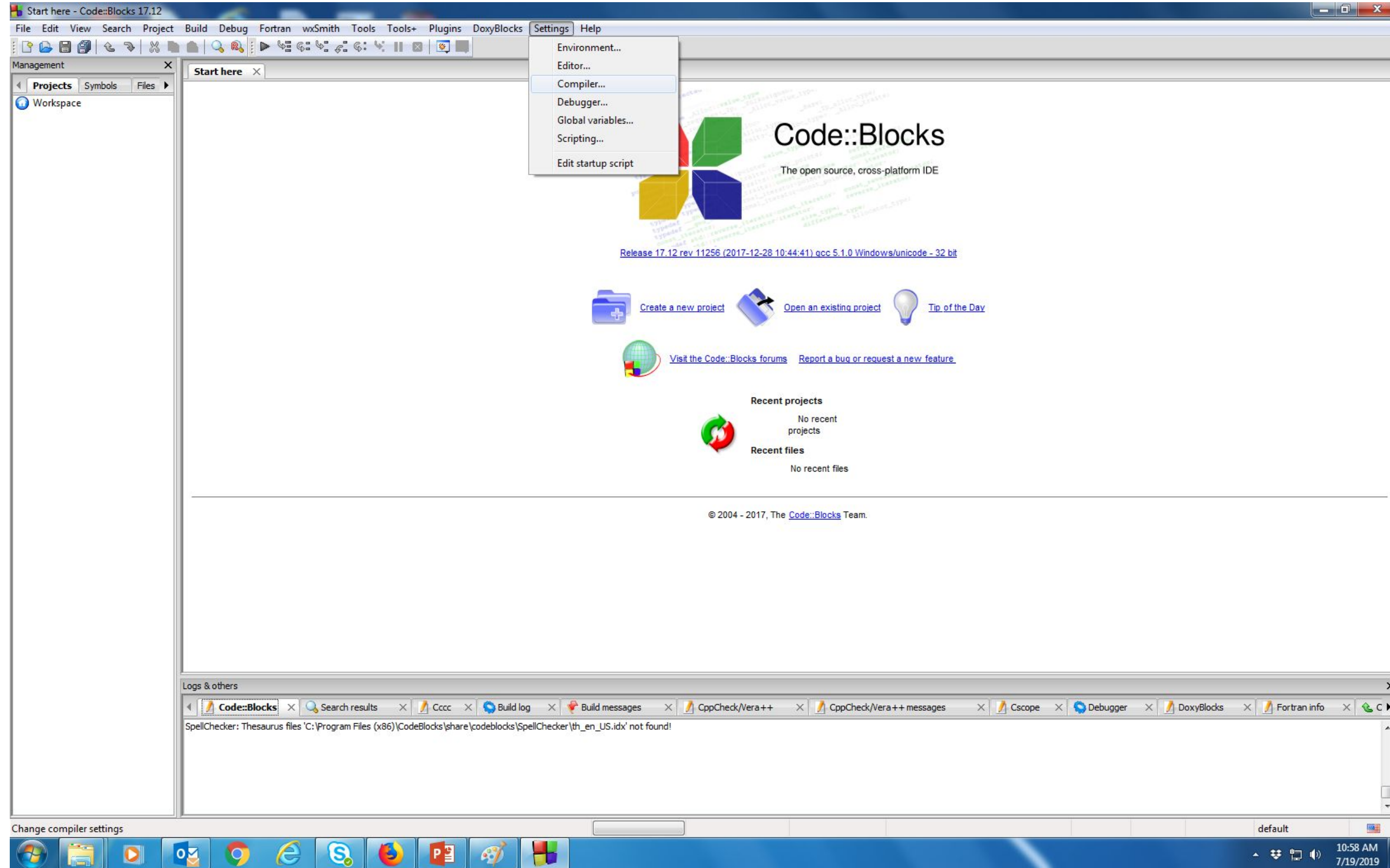
Topics (cont'd)



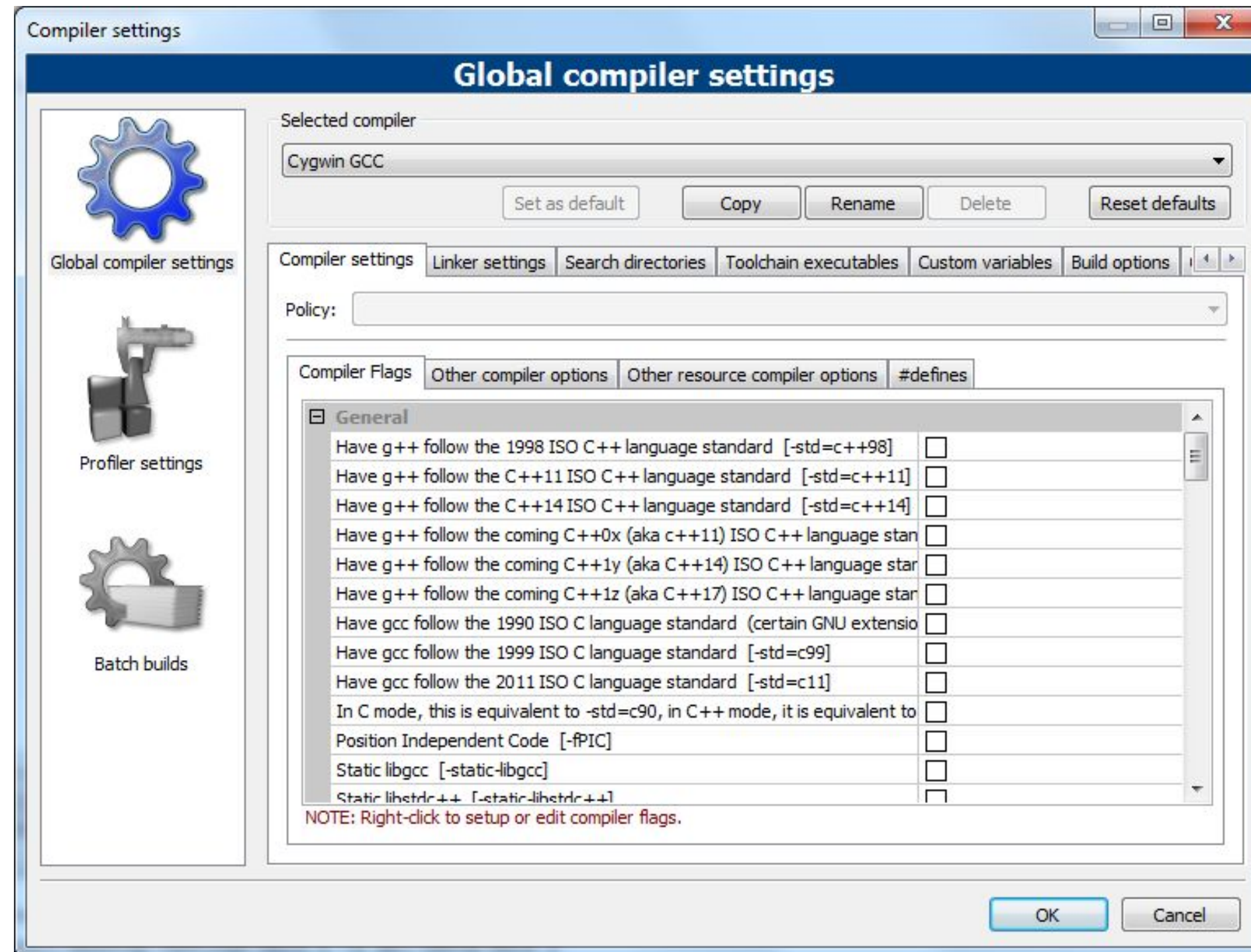
Topics (cont'd)



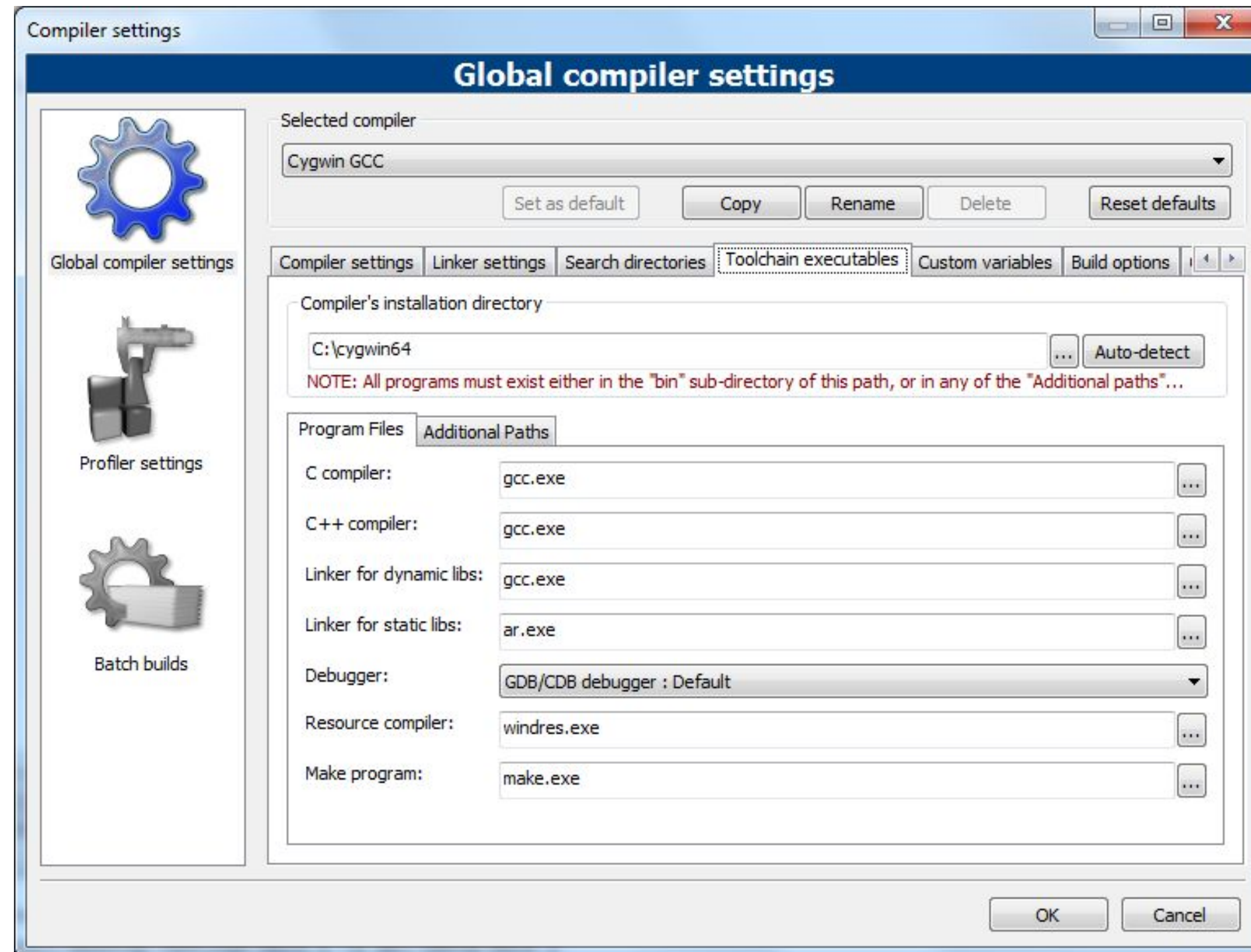
Topics (cont'd)



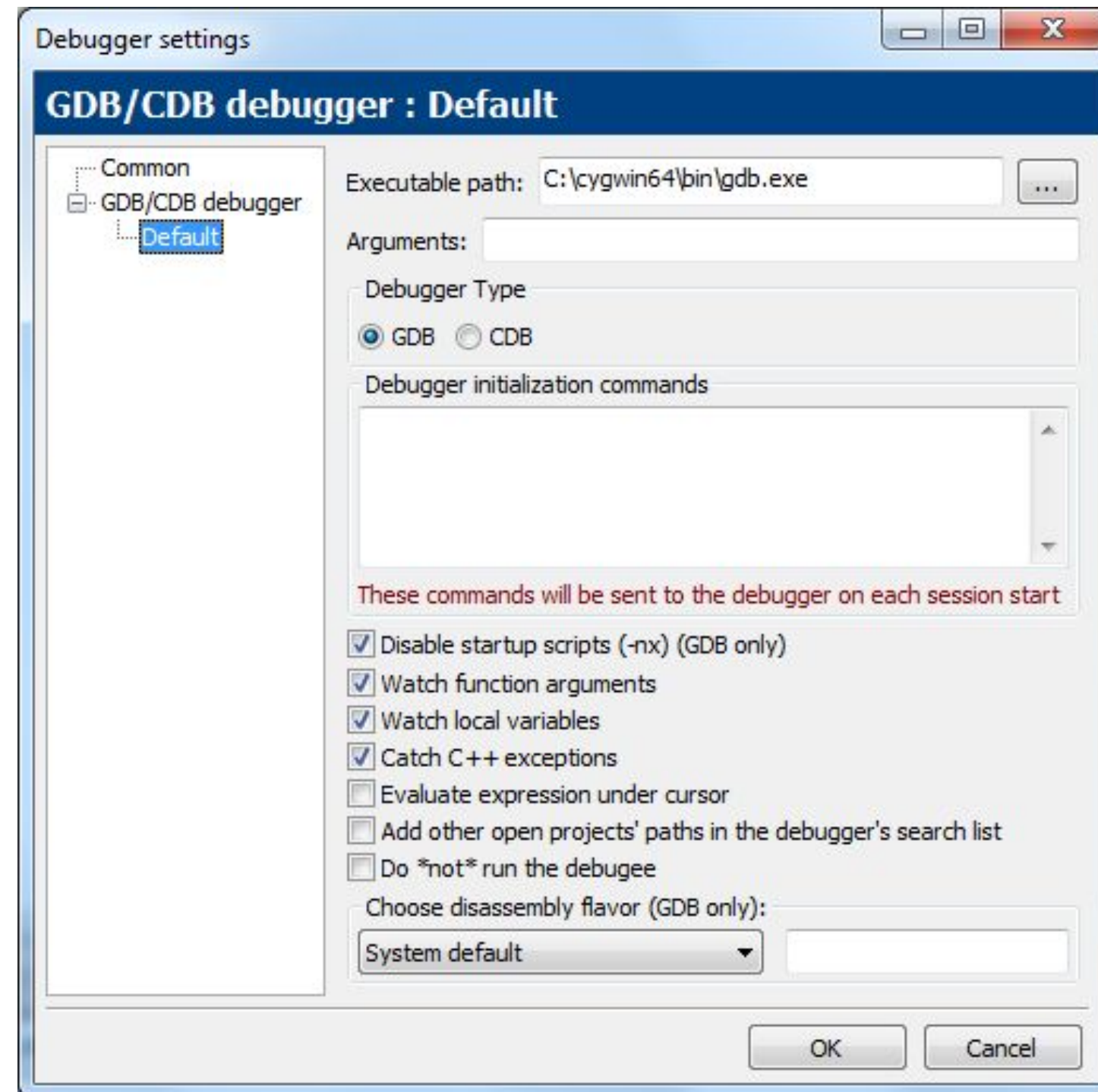
Topics (cont'd)



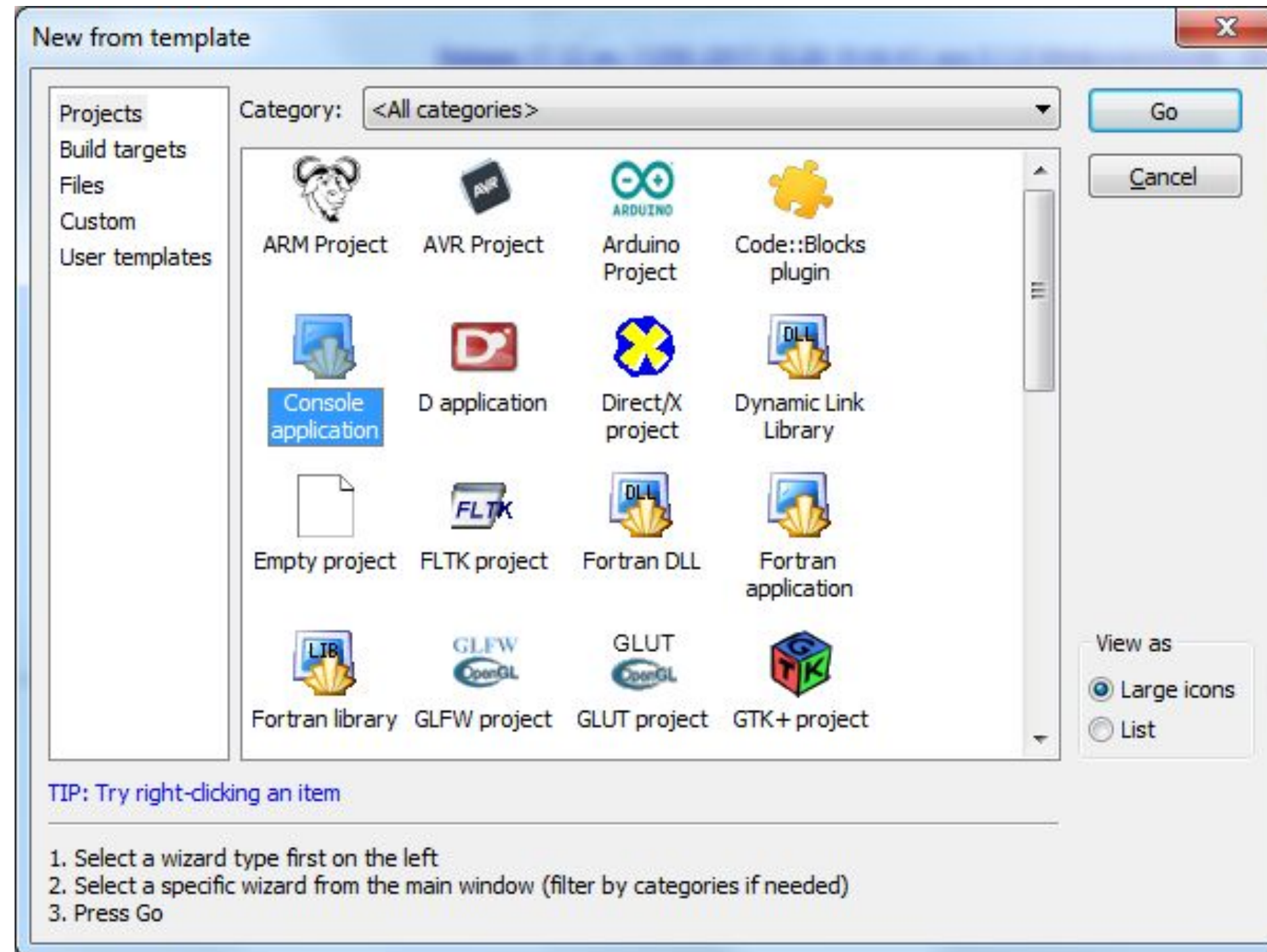
Topics (cont'd)



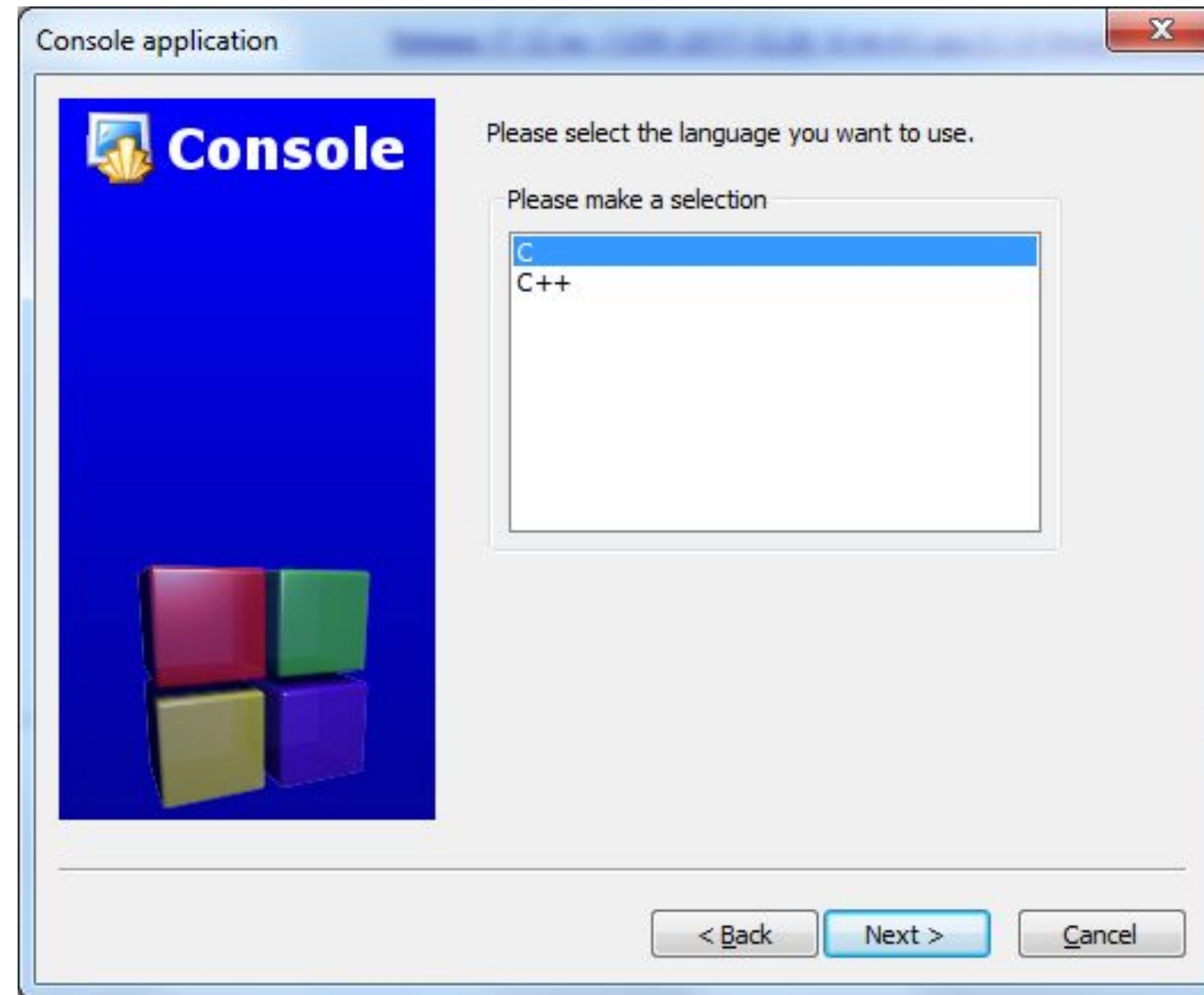
Topics (cont'd)



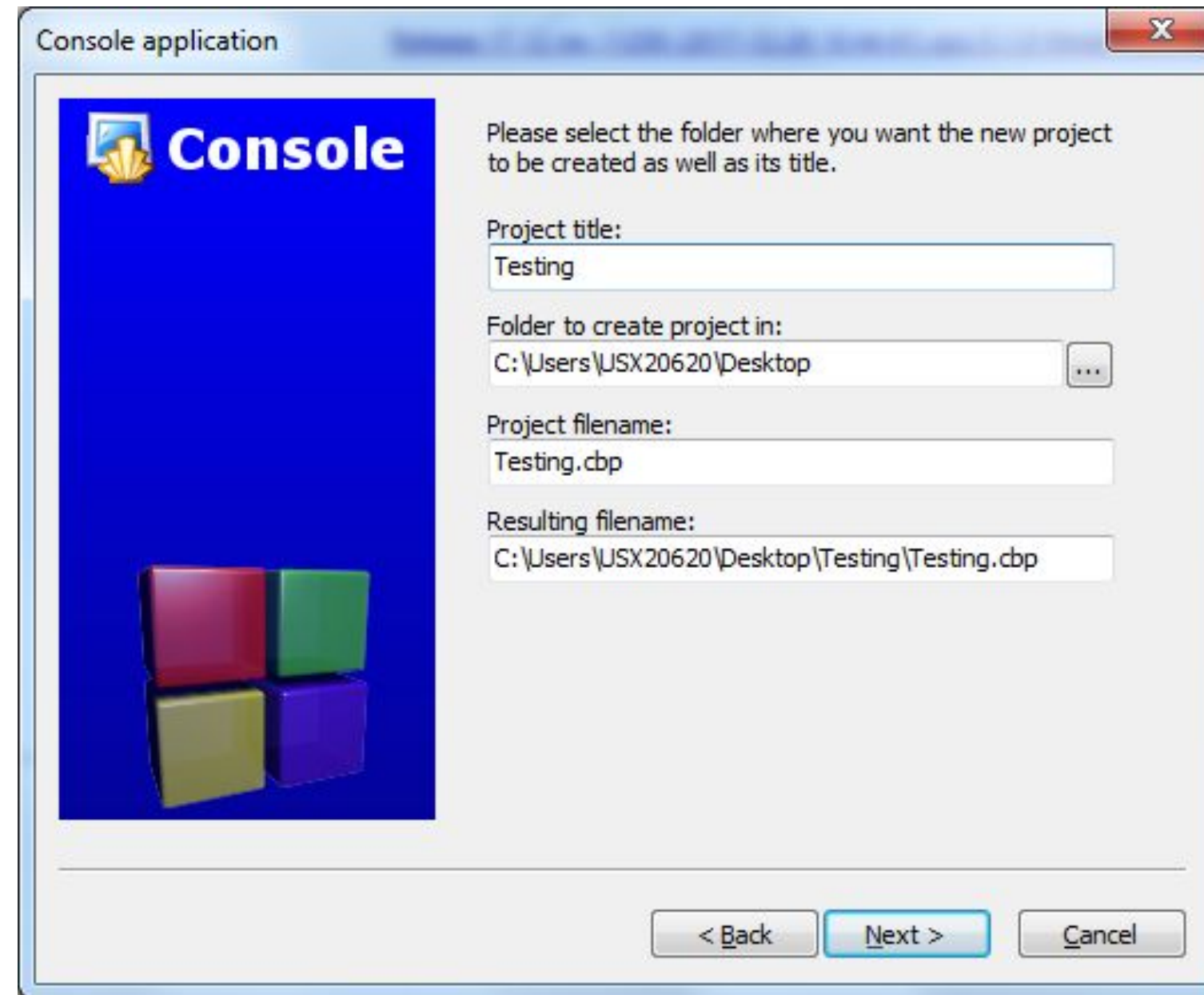
Topics (cont'd)



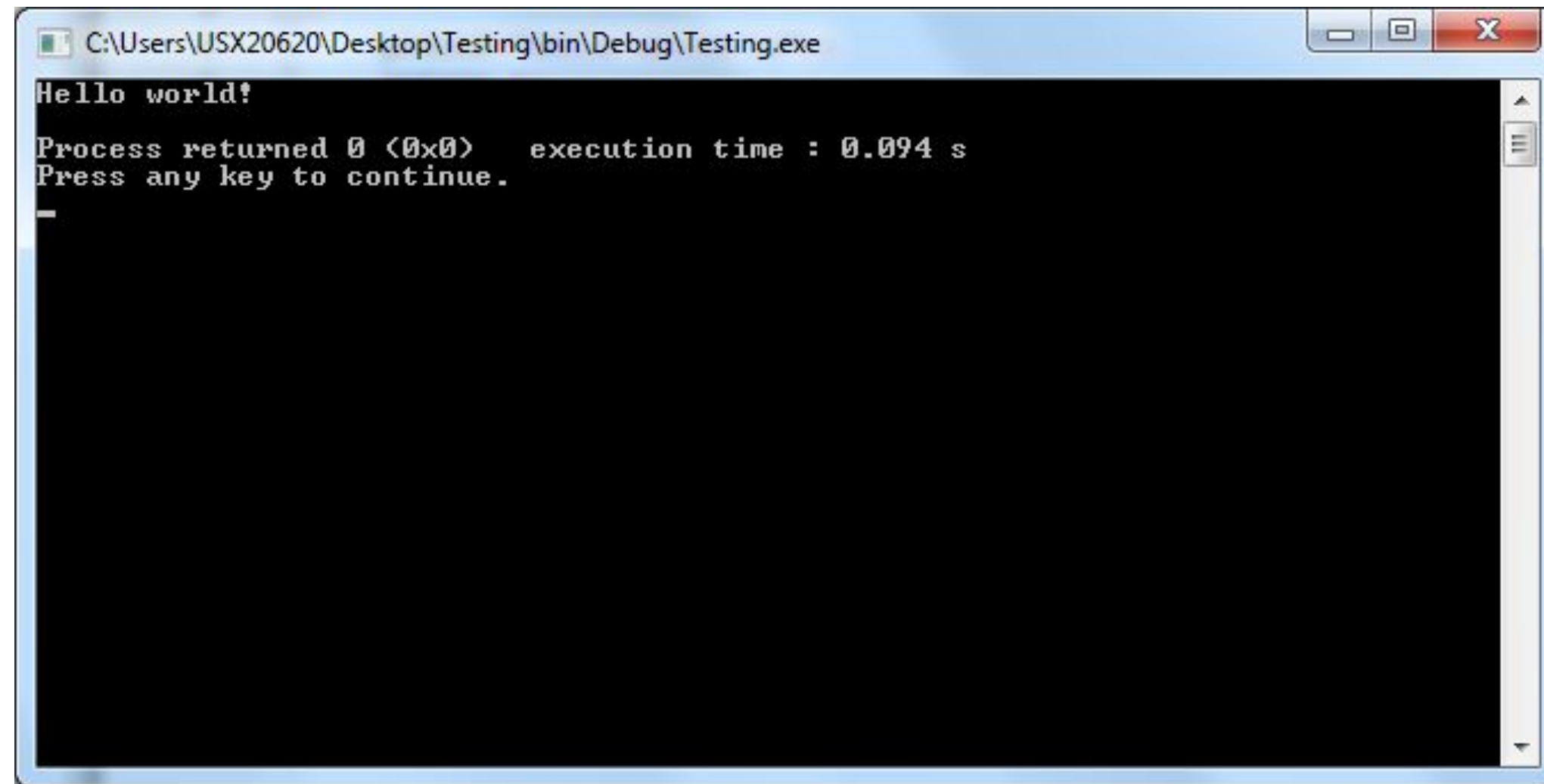
Topics (cont'd)



Topics (cont'd)



Topics (cont'd)



A screenshot of a Windows command prompt window. The title bar shows the file path: C:\Users\USX20620\Desktop\Testing\bin\Debug\Testing.exe. The window has a black background with white text. The text displayed is: "Hello world!", "Process returned 0 (0x0) execution time : 0.094 s", and "Press any key to continue.". A single underscore character is visible on the line following the prompt.

```
C:\Users\USX20620\Desktop\Testing\bin\Debug\Testing.exe
Hello world!
Process returned 0 (0x0) execution time : 0.094 s
Press any key to continue.
_
```

Installing Visual Studio Code and C Extension Windows

Installing Visual Studio Code

Jason Fedin

Overview

- <https://code.visualstudio.com/docs/editor/whyvscode>
- Required for mac and linux users
- <https://code.visualstudio.com/>
- Optional for windows (Follow the wizard, very straight forward, for installation)

Topics (cont'd)

- Install the C++ extension for VS Code (for all distributions)
 - <https://marketplace.visualstudio.com/items?itemName=ms-vscode.cpptools>

Installing Visual Studio Code

Jason Fedin

Overview

- Required for mac and linux users
- <https://code.visualstudio.com/>
- For linux users
 - Select the rpm or deb depending on your platform
 - Follow the instructions here based on your platform:
 - [https://code.visualstudio.com/docs/setup/linux# rhel-fedora-and-centos-based-distributions](https://code.visualstudio.com/docs/setup/linux#_rhel-fedora-and-centos-based-distributions)
 - For example, on fedora, do a yum install (sudo yum install code)

Topics (cont'd)

- Install the C++ extension for VS Code (for all distributions)
 - <https://marketplace.visualstudio.com/items?itemName=ms-vscode.cpptools>

Using The Command Line Interface

Jason Fedin

Using the Command Line Interface

- A text editor (not a Word Processor)
 - A command-prompt or terminal window
 - An installed C compiler
-
- No need for an IDE
 - Simple, efficient workflow
 - Better as you gain experience
 - Can be used if you are overwhelmed by IDEs
 - Useful if hardware resources are limited

Creating a Workspace and configuring the compiler in Visual Studio Code

Jason Fedin

Follow these instructions

- Already installed Cygwin, skip this step, if on linux and windows, find gcc binary directory (usually in /usr/bin/gcc)
- Follow these directions to do the below:
 - <https://code.visualstudio.com/docs/cpp/config-mingw>
- Configure the bash console to use (settings.json)
- Create a workspace
- Configure the compiler path
- Create a build task
- Configure the debug settings
- Add a source code file
- Build the program
- Run the program

C_cpp properties

- {
- "configurations": [- {
- "name": "Win32",
- "defines": [- "_DEBUG",
- "UNICODE"
-],
- "compilerPath": "C:/cygwin64/bin/gcc.exe",
- "intelliSenseMode": "gcc-x64",
- "browse": {
- "path": [- "\${workspaceFolder}"
-],
- "limitSymbolsToIncludedHeaders": true,
- "databaseFilename": ""
- }
- }
-],
- "version": 4
- }

Launch.json

```
• {  
• "version": "0.2.0",  
• "configurations": [  
• {  
• "name": "(gdb) Launch",  
• "type": "cppdbg",  
• "request": "launch",  
• "program": "${workspaceFolder}/helloworld.exe",  
• "args": [],  
• "stopAtEntry": true,  
• "cwd": "${workspaceFolder}",  
• "environment": [],  
• "externalConsole": false,  
• "MIMode": "gdb",  
• "miDebuggerPath": "C:/cygwin64/bin/gdb.exe",  
• "setupCommands": [  
• {  
• "description": "Enable pretty-printing for gdb",  
• "text": "-enable-pretty-printing",  
• "ignoreFailures": true  
• }  
• ]  
• }  
• ]  
• }  
• ]  
• }
```

Tasks.json

- {
- "version": "2.0.0",
- "tasks": [- {
- "label": "build hello world",
- "type": "shell",
- "command": "gcc",
- "args": [- "-g",
- "-o",
- "helloworld",
- "helloworld.c"
-],
- "group": {
- "kind": "build",
- "isDefault": true
- }
- }
-]
- }

Settings.json

```
{  
  "terminal.integrated.shell.windows": "C:\\cygwin64\\bin\\bash.exe"  
}
```

- To set settings, go to view->command pallet and type in settings
- To compile press ctrl->shift->b
- Also, to debug (and only debug) press F5 (will not see output in terminal???)
- To run the program and see output, go to the terminal and type in the executable:
 - .\\helloworld.exe
- If you see “not able to connect to gdb.exe”, you have too many gdb connections open, go to the terminal and keep hitting ctrl c or ctrl z
- Also, tell students they can install “Code Runner” extension and run programs by pressing “ctrl->alt->n

Follow these instructions

- SHOW DEBUGGING
- https://code.visualstudio.com/docs/cpp/config-mingw#_start-a-debugging-session

Challenge

Jason Fedin

Requirements

- Woohoo!! Our First challenge
- Write a C program that displays your first name as output
 - Create a project in code::blocks
 - Delete the “main.c” file that was auto-generated when creating the project
 - Create a new c source file in the above project (name the file test.c)
 - Copy the source code on the next slide into your test.c program

Source code

```
#include <stdio.h>
```

```
int main()
```

```
{  
    printf("Hi, my name is .....");  
    return 0;  
}
```


Next Steps

- Modify the source code to display your name (edit line starting with printf)
- Compile and link the source code
- Run the program!!
- Analyze the output to confirm it is correct and displays your name!!

Comments

Jason Fedin

Comments

- comments are used in a program to document a program and to enhance its readability
- there to remind you, or someone else reading your code, what the program does or what a particular line of code is doing
- comments are ignored by the compiler
- comments are very useful
- a programmer may return to a program that he coded six months ago and may not remember the purpose of a particular function or line of code
- A simple comment can save a significant amount of time otherwise wasted on having to re-understand the code

Syntax

- There are two ways to add comments into a C program
 - by using the two characters / and *
 - marks the beginning of the comment
 - these types of comments have to be terminated
 - to end the comment, the characters * and / are used without any embedded spaces
 - all characters included between the opening /* and the closing */ are treated as part of the comment
 - Referred to as a multi-line comment
-
- by using two consecutive slash characters //
 - Any characters that follow these slashes up to the end of the line are ignored by the compiler
 - Single line comment

Style

```
/*  
 * Written by Jason Fedin  
 * Copyright 2017  
 */
```

You can also embellish comments to make them stand out:

```
/******  
 * This is a very important comment      *  
 * so please read this.                  *  
*****/
```

You can add a comment at the end of a line of code

```
printf("Hello, nope!");    // This line displays a quotation
```


Example

```
/* This program adds two integer values  
and displays the results      */
```

```
#include <stdio.h>
```

```
int main (void)
```

```
{
```

```
    // Declare variables
```

```
    int value1, value2, sum;
```

```
    // Assign values and calculate their sum
```

```
    value1 = 50;
```

```
    value2 = 25;
```

```
    sum = value1 + value2;
```

```
    // Display the result
```

```
    printf ("The sum of %i and %i is %i\n", value1, value2, sum);
```

```
    return 0;
```

```
}
```

Use of Comments

- it is possible to insert so many comments into a program that the readability of the program is actually degraded instead of improved!
- You need to intelligently use comments
- It is a good idea to get into the habit of inserting comment statements into the program as the program is being written or typed in
 - easier to document the program while the particular program logic is still fresh in your mind
 - reap the benefits of the comments during the debug phase, when program logic errors are being isolated and debugged
- A comment can help you read through the program, but it can also help point the way to the source of the logic mistake
- Self documenting comments by using meaningful names

The Preprocessor

Jason Fedin

Overview

- another unique feature of the C language that is not found in many other higher-level programming languages
- allows for programs to be easier to develop, easier to read, easier to modify, and easier to port to different computer systems
- part of the C compilation process that recognizes special statements
- analyzes these statements before analysis of the C program itself takes place
- an instruction to your compiler to do something before compiling the source code
- could be anywhere in your code

Overview (cont'd)

- Preprocessor statements are identified by the presence of a pound sign, #, which must be the first non-space character on the line
- Our first “challenge” used a preprocessor directive, specifically the #include directive
- We will utilize the preprocessor to:
 - create our own constants and macros with the #define statement
 - build your own library files with the #include statement
 - make more powerful programs with the conditional #ifdef, #endif, #else, and #ifndef Statements
- We will cover the above in future lectures

#include

Jason Fedin

Overview

- the `#include` statement is a preprocessor directive
- `#include <stdio.h>`
- It is not strictly part of the executable program, however, the program won't work without it
- The symbol `#` indicates this is a preprocessing directive
- an instruction to your compiler to do something before compiling the source code
- many preprocessing directives
- are usually some at the beginning of the program source file, but they can be anywhere
- similar to the `import` statement in java
- In the above example, the compiler is instructed to “include” in your program the contents of the file with the name `stdio.h`
- called a header file because it is usually included at the head of a program source file
- `.h` extension

Header Files

- header files define information about some of the functions that are provided by that file
 - `stdio.h` is the standard C library header and provides functionality for displaying output, among many other things
 - we need to include this file in a program when using the `printf()` function from the standard library
 - `stdio.h` contains the information that the compiler needs to understand what `printf()` means, as well as other functions that deal with input and output
 - `stdio`, is short for standard input/output.
-
- header files specify information that the compiler uses to integrate any predefined functions within a program
 - You will be creating your own header files for use with your programs

Syntax

- header file names are case sensitive on some systems, so you should always write them in lowercase
- Two ways to #include header files in a program
- Using angle brackets (#include <Jason.h>)
- tells the preprocessor to look for the file in one or more standard system directories
- Using double quotes (#include "Jason.h")
- tells the preprocessor to first look in the current directory
- Every C compiler that conforms to the C11 standard will have a set of standard header files supplied with it
- you should use #ifndef and #define to protect against multiple inclusions of a header file

Syntax

```
// some header
```

```
// typedefs
```

```
typedef struct names_st names;
```

```
// function prototypes
```

```
void get_names(names *);
```

```
void show_names(const names *);
```

```
char * s_gets(char * st, int n);
```

header files includes many different things

#define directives

structure declarations

typedef statements

function prototypes

executable code normally goes into a source code file, not a header file

Displaying Output

Jason Fedin

Overview

- In our first challenge, you should have noticed that there was a line of code that included the word “printf”
- `printf(“Hi, my name is Jason”);`
- `printf()` is a standard library function
- it outputs information to the command line (the *standard output stream*, which is the command line by default)
- the information displayed is based on what appears between the parentheses that immediately follow the function name (`printf`)
- also notice that this line *does* end with a semicolon

printf Function

- probably the most common function used in C
- provides an easy and convenient means to display program results
- Not only can simple phrases be displayed, but the values of *variables* and the results of computations can also be displayed
- Used for debugging

Reading Input from the Terminal

Jason Fedin

Overview

- We have discussed output, now lets learn a bit about input
- Very useful to ask the user to enter data into a program
 - via the terminal/console
- The C library contains several input functions, and `scanf()` is the most general of them
 - can read a variety of formats
- reads the input from the standard input stream **stdin** and scans that input according to the **format** provided
 - format can be a simple constant string, but you can specify `%s`, `%d`, `%c`, `%f`, etc., to read strings, integer, character or floats
- If the stdin is input from the keyboard then text is read in because the keys generate text characters: letters, digits, and punctuation
 - when you enter the integer 2014, you type the characters 2 0 1 and 4
 - If you want to store that as a numerical value rather than as a string, your program has to convert the string character-by-character to a numerical value and this is the job of the `scanf` function

scanf()

- like printf(), scanf() uses a control string followed by a list of arguments
 - control string indicates the destination data types for the input stream of characters
- the printf() function uses variable names, constants, and expressions as its argument list
- the scanf() function uses pointers to variables
 - you do not have to know anything about pointers to use the function
- Remember these 3 rules about scanf
 - returns the number of items that it successfully reads
 - If you use scanf() to read a value for one of the basic variable types we've discussed, precede the variable name with an &
 - If you use scanf() to read a string into a character array, don't use an &.
- The scanf() function uses whitespace (newlines, tabs, and spaces) to decide how to divide the input into separate fields
- scanf is the inverse of printf(), which converts integers, floating-point numbers, characters, and C strings to text that is to be displayed onscreen

scanf() example

```
#include <stdio.h>
```

```
int main( ) {
```

```
    char str[100];
```

```
    int i;
```

```
    printf( "Enter a value :");
```

```
    scanf("%s %d", str, &i);
```

```
    printf( "\nYou entered: %s %d ", str, i);
```

```
    return 0;
```

```
}
```

scanf (cont'd)

- when a program uses scanf to gather input from the keyboard, it waits for you to input some text
 - when you enter some text and press enter, then program proceeds and reads the input
- Remember, scanf() expects input in the same format as you provided %s and %d
 - you have to provide valid inputs like "string integer"

Variables and Data Types

Jason Fedin

Overview

- Remember that a program needs to store the instructions of its program and the data that it acts upon while your computer is executing that program
 - This information is stored in memory (RAM)
 - RAM's contents are lost when the computer is turned off
 - Hard drives store persistent data
- You can think of RAM as an ordered sequence of boxes
 - the box is full when it represents 1 or the box is empty when it represents 0
 - each box represents one binary digit, either 0 or 1 (*true* and *false*)
 - each box is called a *bit*
- bits in memory are grouped into sets of eight (byte)
 - each byte has been labeled with a number (*address*)
 - the address of a byte uniquely references that byte in your computer's memory.
- Again, memory consists of a large number of bits that are in groups of eight (called bytes) and each byte has a unique address

Variables

- The true power of programs you create is their manipulation of data
 - So, we need to understand the different data types you can use, as well as how to create and name variables
- Constants are types of data that do not change and retain their values throughout the life of the program
- Variables are types of data may change or be assigned values as the program runs
- Variables are the names you give to computer memory locations which are used to store values in a computer program
- For example, assume you want to store two values 10 and 20 in your program and at a later stage, you want to use these two values
 - Create variables with appropriate names
 - Store your values in those two variables.
 - Retrieve and use the stored values from the variables

Naming Variables

- The rules for naming variables in C is that all names must begin with a letter or underscore (_) and can be followed by any combination of letters (upper- or lowercase), underscores, or the digits 0–9

Jason

myFlag

i

J5x7

my_data

_anotherVariable

Naming Variables (cont'd)

- The below lists some examples of invalid variable names

temp\$value - \$ is not a valid character

my flag - embedded spaces are not permitted

3Jason – variable names cannot start with a number

int - int is a reserved word

- use meaningful names when selecting variable names
 - can dramatically increase the readability of a program and pay off in the debug and documentation phases

Data Types

- Some types of data in programs are numbers, letters or words
 - computer needs a way to identify and use these different kinds
- a data type represents a type of the data which you can process using your program
 - examples include ints, floats, doubles, etc.
 - also correspond to byte sizes on the platform of your program
- primitive data types are types that are not objects and store all sorts of data

Declaring Variables

- declaring a variable is when you specify the type of the variable followed by the variable name
 - specifies to the compiler how a particular variable will be used by the program
- for example, the keyword `int` is used to declare the basic integer variable
 - first comes `int`, and then the chosen name of the variable, and then a semicolon
 - *type-specifier variable-name;*
 - to declare more than one variable, you can declare each variable separately, or you can follow the `int` with a list of names in which each name is separated from the next by a comma
 - C requires that all program variables be declared before they are used in a program

```
int x;
```

```
int x, y, z;
```

- The above creates variables but does not provide values for them
 - we can assign a variable a value by using the `=` operator

```
x = 112;
```

Initializing Variables

- To initialize a variable means to assign it a starting, or initial, value
- this can be done as part of the declaration
 - follow the variable name with the assignment operator (=) and the value you want the variable to have

```
int x = 21;
```

```
int y = 32, z = 14;
```

```
int x, z = 94;    /* valid, but poor, form */
```

- In the last line, only z is initialized
 - it is best to avoid putting initialized and noninitialized variables in the same declaration statement

Basic Data Types

Jason Fedin

Overview

- We understand that C supports many different types of variables and each type of variable is used for storing kind of data
 - types that store integers
 - types that store nonintegral numerical values
 - types that store characters
- Some examples of basic data types in C are:

int

float

double

char

_Bool

- the difference between the types is in the amount of memory they occupy and the range of values they can hold
 - the amount of storage that is allocated to store a particular type of data
 - depends on the computer you are running (machine-dependent)
 - an integer might take up 32 bits on your computer, or perhaps it might be stored in 64

int

- a variable of type int can be used to contain integral values only (values that do not contain decimal places)
- a minus sign preceding the data type and variable indicates that the value is negative
- the int type is a signed integer
 - it must be an integer and it can be positive, negative, or zero
- if an integer is preceded by a zero and the letter x (either lowercase or uppercase), the value is taken as being expressed in hexadecimal (base 16) notation
 - `int rgbColor = 0xFFEF0D;`
- the values 158, -10, and 0 are all valid examples of integer constants
 - no embedded spaces are permitted between the digits
 - values larger than 999 cannot be expressed using commas (12,000 must be written as 12000)

float

- A variable to be of type float can be used for storing floating-point numbers (values containing decimal places)
- the values 3., 125.8, and $-.0001$ are all valid examples of floating-point constants that can be assigned to a variable
- floating-point constants can also be expressed in scientific notation
 - $1.7e4$ is a floating-point value expressed in this notation and represents the value 1.7×10 to the power of 4

double

- the double type is the same as type float, only with roughly twice the precision
 - used whenever the range provided by a float variable is not sufficient
 - can store twice as many significant digits
 - most computers represent double values using 64 bits
- all floating-point constants are taken as double values by the C compiler
- To explicitly express a float constant, append either an f or F to the end of the number
 - 12.5f

_Bool

- the _Bool data type can be used to store just the values 0 or 1
 - used for indicating an on/off, yes/no, or true/false situation (binary choices)
- _Bool variables are used in programs that need to indicate a Boolean condition
 - a variable of this type might be used to indicate whether all data has been read from a file
- 0 is used to indicate a false value
- 1 indicates a true value

Example

```
#include <stdio.h>
```

```
int main (void)
{
    int    integerVar = 100;
    float  floatingVar = 331.79;
    double doubleVar = 8.44e+11;

    _Bool  boolVar = 0;

    return 0;
}
```

Other Data Types

- the int type will probably meet most of your integer needs when beginning in C
- However, C offers many other integer types
 - gives the programmer the option of matching a type to a particular use case
 - integer types vary in the range of values offered and in whether negative numbers can be used
- C offers three adjective keywords to modify the basic integer type (can also be used by itself)
 - short, long, and unsigned
- The type short int, or short may use less storage than int, thus saving space when only small numbers are needed
 - can be used when the program needs a lot of memory and the amount of available memory is limited
- The type long int, or long, may use more storage than int, thus enabling you to express larger integer values
- The type long long int, or long long may use more storage than long
 - A constant value of type long int is formed by optionally appending the letter L (upper- or lowercase) onto the end of an integer constant
 - `long int numberOfPoints = 131071100L;`

Other Data Types (cont'd)

- Type specifiers can also be applied to doubles
 - `long double US_deficit_2017;`
- A long double constant is written as a floating constant with the letter `l` or `L` immediately following
 - `1.234e+7L`
- The type `unsigned int`, or `unsigned`, is used for variables that have only nonnegative values (positive)
 - `unsigned int counter;`
 - the accuracy of the integer variable is extended
- The keyword `signed` can be used with any of the signed types to make your intent explicit
 - `short`, `short int`, `signed short`, and `signed short int` are all names for the same type

Enums and Char

By Jason Fedin

Enums

- a data type that allows a programmer to define a variable and specify the valid values that could be stored into that variable
 - can create a variable named “myColor” and it can only contain one of the primary colors, red, yellow, or blue, and no other values
- Your first have to define the enum type and give it a name
 - initiated by the keyword enum
 - then the name of the enumerated data type
 - then list of identifiers (enclosed in a set of curly braces) that define the permissible values that can be assigned to the type

```
enum primaryColor { red, yellow, blue };
```

- variables declared to be of this data type can be assigned the values red, yellow, and blue inside the program, and no other values

Enums (cont'd)

- To declare a variable to be of type enum primaryColor
 - use the keyword enum
 - followed by the enumerated type name
 - followed by the variable list. So the statement
- `enum primaryColor myColor, gregsColor;`
- defines the two variables myColor and gregsColor to be of type primaryColor
 - the only permissible values that can be assigned to these variables are the names red, yellow, and blue
 - `myColor = red;`
- Another example
`enum month { January, February, March, April, May, June, July, August, September, October, November, December };`

Enums as ints

- the compiler actually treats enumeration identifiers as integer constants
 - first name in list is 0

```
enum month thisMonth;
```

```
...
```

```
thisMonth = February;
```

- the value 1 is assigned to thisMonth (and not the name February) because it is the second identifier listed inside the enumeration list
- if you want to have a specific integer value associated with an enumeration identifier, the integer can be assigned to the identifier when the data type is defined

```
enum direction { up, down, left = 10, right };
```

- an enumerated data type direction is defined with the values up, down, left, and right
- up = 0 because it appears first in the list
- 1 to down because it appears next
- 10 to left because it is explicitly assigned this value
- 11 to right because it appears immediately after left in the list

Char

- Chars represent a single character such as the letter 'a', the digit character '6', or a semicolon (';')
- Character literals use single quotes such as 'A' or 'Z'
- You can also declare char variables to be unsigned
 - can be used to explicitly tell the compiler that a particular variable is a signed quantity
- We will talk about a character string in another lecture, much different than a single character

Declaring a char

```
char broiled;    /* declare a char variable    */
broiled = 'T';   /* OK                        */
broiled = T;     /* NO! Thinks T is a variable    */
broiled = "T";   /* NO! Thinks "T" is a string   */
```

- If you omit the quotes, the compiler thinks that T is the name of a variable
- If you use double quotes, it thinks you are using a string
- you can also use the numerical code to assign values

```
char grade = 65; /* ok for ASCII, but poor style */
```

Escape Characters

- C contains special characters that represent actions
 - backspacing
 - going to the next line
 - making the terminal bell ring (or speaker beep)
- We can represent these actions by using special symbol sequences
 - called escape sequences
- Escape sequences must be enclosed in single quotes when assigned to a character variable

`char x = '\n';`

- and then print the variable x to advance the printer or screen one line

Escape Characters (cont'd)

Sequence	Meaning
<code>\a</code>	Alert (ANSI C).
<code>\b</code>	Backspace.
<code>\f</code>	Form feed.
<code>\n</code>	Newline.
<code>\r</code>	Carriage return.
<code>\t</code>	Horizontal tab.
<code>\v</code>	Vertical tab.
<code>\\</code>	Backslash (\).
<code>\'</code>	Single quote (').
<code>\"</code>	Double quote (").
<code>\?</code>	Question mark (?).
<code>\0oo</code>	Octal value. (o represents an octal digit.)
<code>\xhh</code>	Hexadecimal value. (h represents a hexadecimal digit.)

Taken from "C Primer Plus",
Prata

Format Specifiers

Jason Fedin

Overview

- format specifiers are used when displaying variables as output
 - they specify the type of data of the variable to be displayed

```
int sum = 89;  
printf ("The sum is %d\n", sum);
```

- The printf() function can display as output the values of variables
 - has two items or arguments enclosed within the parentheses
 - arguments are separated by a comma
 - first argument to the printf() routine is always the character string to be displayed
 - along with the display of the character string, you might also frequently want to have the value of certain program variables displayed
- The percent character inside the first argument is a special character recognized by the printf() function
 - the character that immediately follows the percent sign specifies what type of value is to be displayed

Code Walkthrough

- `#include <stdio.h>`

```
int main (void)
{
    int    integerVar = 100;
    float  floatingVar = 331.79;
    double doubleVar = 8.44e+11;
    char   charVar = 'W';

    _Bool  boolVar = 0;

    printf ("integerVar = %i\n", integerVar);
    printf ("floatingVar = %f\n", floatingVar);
    printf ("doubleVar = %e\n", doubleVar);
    printf ("doubleVar = %g\n", doubleVar);
    printf ("charVar = %c\n", charVar);

    printf ("boolVar = %i\n", boolVar);

    return 0;
}
```

Example Output

```
integerVar = 100
```

```
floatingVar = 331.790009
```

```
doubleVar = 8.440000e+11
```

```
doubleVar = 8.44e+11
```

```
charVar = W
```

```
boolVar = 0;
```

- In the second line of the program's output, notice that the value of 331.79, which is assigned to `floatingVar`, is actually displayed as 331.790009. In fact, the actual value displayed is dependent on the particular computer system you are using. The reason for this inaccuracy is the particular way in which numbers are internally represented inside the computer. You have probably come across the same type of inaccuracy when dealing with numbers on your pocket calculator. If you divide 1 by 3 on your calculator, you get the result .33333333, with perhaps some additional 3s tacked on at the end. The string of 3s is the calculator's approximation to one third. Theoretically, there should be an infinite number of 3s. But the calculator can hold only so many digits, thus the inherent inaccuracy of the machine. The same type of inaccuracy applies here: Certain floating-point values cannot be exactly represented inside the computer's memory.
- When displaying the values of float or double variables, you have the choice of three different formats. The `%f` characters are used to display values in a standard manner. Unless told otherwise, `printf()` always displays a float or double value to six decimal places rounded. You see later in this chapter how to select the number of decimal places that are displayed.
- The `%e` characters are used to display the value of a float or double variable in scientific notation. Once again, six decimal places are automatically displayed by the system.
- With the `%g` characters, `printf()` chooses between `%f` and `%e` and also automatically removes from the display any trailing zeroes. If no digits follow the decimal point, it doesn't display that either.
- In the next-to-last `printf()` statement, the `%c` characters are used to display the single character 'W' that you assigned to `charVar` when the variable was declared. Remember that whereas a character string (such as the first argument to `printf()`) is enclosed within a pair of double quotes, a character constant must always be enclosed within a pair of single quotes.
- The last `printf()` shows that a `_Bool` variable can have its value displayed using the integer format characters `%i`.

Summary

Type	Constant Examples	printf chars
char	'a', '\n'	%c
_Bool	0, 1	%i, %u
short int	—	%hi, %hx, %ho
unsigned short int	—	%hu, %hx, %ho
int	12, -97, 0xFFE0, 0177	%i, %x, %o
unsigned int	12u, 100U, 0xFFu	%u, %x, %o
long int	12L, -2001, 0xffffL	%li, %lx, %lo
unsigned long int	12UL, 100ul, 0xffeeUL	%lu, %lx, %lo
long long int	0xe5e5e5e5LL, 5001l	%lli, %llx, %llo
unsigned long long int	12ull, 0xffeeULL	%llu, %llx, %llo
float	12.34f, 3.1e-5f, 0x1.5p10, 0x1P-1	%f, %e, %g, %a
double	12.34, 3.1e-5, 0x.1p3	%f, %e, %g, %a
long double	12.34l, 3.1e-5l	%Lf, %Le, %Lg

Taken from “Programming in C”, Kochan

Command Line Arguments

By Jason Fedin

Overview

- There are times when a program is developed that requires the user to enter a small amount of information at the terminal
- This information might consist of a number indicating the triangular number that you want to have calculated or a word that you want to have looked up in a dictionary
- Two ways of handling this
 - Requesting the data from the user
 - supply the information to the program at the time the program is executed (command-line arguments)
- We know that the `main()` function is a special function in C
 - Entry point of the program
- When `main()` is called by the runtime system, two arguments are actually passed to the function
 - the first argument (`argc` for argument count) is an integer value that specifies the number of arguments typed on the command line
 - the second argument (`argv` for argument vector) is an array of character pointers (strings)
- The first entry in this array is a pointer to the name of the program that is executing

Overview (cont'd)

```
int main (int argc, char *argv[])  
{  
    ...  
}
```

Lets go through an example!

Example

```
#include <stdio.h>

int main (int  argc, char *argv[])
{
    int  number_of_arguments = argc;
    char *argument1 = argv[0];
    char *argument2 = argv[1];

    printf("Number of arguments: %d", argc);
    printf("Argument 1 is the program name: %s", argument1);
    printf("Argument 2 is the command line argument: %s" , argument2);

    return 0;
}
```

Challenge

By Jason Fedin

Requirements

- In this challenge, you are to create a C program that displays the perimeter and area of a rectangle
- The program should create 4 variables of type double
 - one variable to store the width of the rectangle
 - one variable to store the height of the rectangle
 - one variable to store the perimeter of the rectangle
 - one variable to store the area of the rectangle
- The program should perform the calculation for the perimeter of a rectangle
 - Use the + operator addition and the * operator for multiplication
 - perimeter is calculated by adding the height and width and then multiplying by two
 - Area is calculated by multiplying the width * height variables

Requirements

- The program should display the height, width, and perimeter variables in the correct format in one print statement
- The program should display the height, width, and area variables in the correct format in one print statement

Hints

- Create a project in code::blocks
- Edit the main.c file that is auto-generated for you as part of creating the project
- Declare and initialize the height and width variables to any value (need to be of type double)
- Declare the perimeter and area values to 0.0
- Assign to the perimeter and area values the correct data based on calculations
 - `perimeter = 2.0 * (height + width);`
 - `area = width * height;`

Next Steps

- Use the printf function and the correct format specifier to display the required output
- Compile and link the source code
- Run the program!!
- Analyze the output to confirm it is correct!!

Challenge

By Jason Fedin

Requirements

- In this challenge, you are to create a C program that defines an enum type and uses that type to display the values of some variables
- The program should create an enum type named Company
 - Valid values for this type are GOOGLE, FACEBOOK, XEROX, YAHOO, EBAY, MICROSOFT
- The program should create three variables of the above enum type that are assigned values: XEROX, GOOGLE, and EBAY
- The program should display as output, the value of the three variables with each variable separated by a newline
 - Correct output would be if XEROX, GOOGLE, and EBAY variables are printed in that order:
 - 2
 - 0
 - 4

Hints

- Define the enum type and its values
- Declare and initialize three variables with the values specified on the previous slide
- Use printf to display the value of the enum variables
 - Use the '\n' escape character to display a new line

Overview

By Jason Fedin

Overview

- Operators are functions that use a symbolic name
 - perform mathematical or logical functions
- Operators are predefined in C, just like they are in most other languages, and most operators tend to be combined with the infix style
- A *logical operator* (sometimes called a “Boolean operator”) is an operator that returns a Boolean result that’s based on the Boolean result of one or two other expressions
- An arithmetic operator is a mathematical function that takes two operands and performs a calculation on them
- Other operators include assignment, relational (<, >, !=), bitwise (<<, >>, ~)

Expressions and Statements

- Statements form the basic program steps of C, and most statements are constructed from expressions
- An *expression* consists of a combination of operators and operands
 - operands are what an operator operates on
 - operands can be constants, variables, or combinations of the two
 - every expression has a value

-6

4+21

$a * (b + c/d) / 20$

$q = 5 * 2$

$x = ++q \% 3$

$q > 3$

Expressions and Statements (cont'd)

- Statements are the building blocks of a program (declaration)
 - A program is a series of statements with special syntax ending with a semicolon (simple statements)
 - a complete instruction to the computer
- Declaration statement: **int Jason;**
- Assignment Statement: **Jason = 5;**
- Function call statement: **printf("Jason");**
- Structure Statement: **while (Jason < 20) Jason = Jason + 1;**
- Return Statement: **return 0;**
- C considers any expression to be a statement if you append a semicolon (expression statements)
 - So, 8; and 3-4; are perfectly valid in C

Compound Statements

- two or more statements grouped together by enclosing them in braces (*block*)

```
int index = 0;
while (index < 10)
{
    printf("hello");
    index = index + 1;
}
```

Basic Operators

By Jason Fedin

Overview

- Lets discuss, arithmetic, logical, assignment and relational operators
- An arithmetic operator is a mathematical function that takes two operands and performs a calculation on them
- A logical operator (sometimes called a “Boolean operator”) is an operator that returns a Boolean result that’s based on the Boolean result of one or two other expressions
- Assignment operators set variables equal to values
 - assigns the value of the expression at its right to the variable at its left
- A relational operator will compare variables against each other

Arithmetic Operators in C

Operator	Description	Example
+	Adds two operands.	$A + B = 30$
-	Subtracts second operand from the first.	$A - B = -10$
*	Multiplies both operands.	$A * B = 200$
/	Divides numerator by de-numerator.	$B / A = 2$
%	Modulus Operator and remainder of after an integer division.	$B \% A = 0$
++	Increment operator increases the integer value by one.	$A++ = 11$
--	Decrement operator decreases the integer value by one.	$A-- = 9$

***** All tables taken from tutorials points website *****

Logical Operators

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false.
	Called Logical OR Operator. If any of the two operands is non-zero, then the condition becomes true.	(A B) is true.
!	Called Logical NOT Operator. It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.	!(A && B) is true.

Assignment Operators

Operator	Description	Example
=	Simple assignment operator	C = A + B will assign the value of A + B to C
+=	Add AND assignment operator. It adds the right operand to the left operand and assign the result to the left operand.	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator. It subtracts the right operand from the left operand and assigns the result to the left operand.	C -= A is equivalent to C = C - A
*=	Multiply AND assignment operator. It multiplies the right operand with the left operand and assigns the result to the left operand.	C *= A is equivalent to C = C * A

Assignment Operators (cont'd)

Operator	Description	Example
<code>/=</code>	Divide AND assignment operator. It divides the left operand with the right operand and assigns the result to the left operand.	<code>C /= A</code> is equivalent to <code>C = C / A</code>
<code>%=</code>	Modulus AND assignment operator. It takes modulus using two operands and assigns the result to the left operand.	<code>C %= A</code> is equivalent to <code>C = C % A</code>
<code><<=</code>	Left shift AND assignment operator.	<code>C <<= 2</code> is same as <code>C = C << 2</code>
<code>>>=</code>	Right shift AND assignment operator.	<code>C >>= 2</code> is same as <code>C = C >> 2</code>
<code>&=</code>	Bitwise AND assignment operator.	<code>C &= 2</code> is same as <code>C = C & 2</code>
<code>^=</code>	Bitwise exclusive OR and assignment operator.	<code>C ^= 2</code> is same as <code>C = C ^ 2</code>
<code> =</code>	Bitwise inclusive OR and assignment operator.	<code>C = 2</code> is same as <code>C = C 2</code>

Relational Operators

Operator	Description	Example
==	Checks if the values of two operands are equal or not. If yes, then the condition becomes true.	(A == B) is not true.
!=	Checks if the values of two operands are equal or not. If the values are not equal, then the condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand. If yes, then the condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand. If yes, then the condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand. If yes, then the condition becomes true.	(A >= B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand. If yes, then the condition becomes true.	(A <= B) is true.

Bitwise Operators

Jason Fedin

Overview

- C offers bitwise logical operators and shift operators
 - look something like the logical operators you saw earlier but are quite different
 - operate on the bits in integer values
- Not used in the common program
- One major use of the bitwise AND, &, and the bitwise OR, |, is in operations to test and set individual bits in an integer variable
 - can use individual bits to store data that involve one of two choices
- You could use a single integer variable to store several characteristics of a person.
 - store whether the person is male or female with one bit
 - use three other bits to specify whether the person can speak French, German, or Italian
 - another bit to record whether the person's salary is \$50,000 or more
 - in just four bits you have a substantial set of data recorded

Binary Numbers

- a binary number is a number that includes only ones and zeroes.
- the number could be of any length
- the following are all examples of binary numbers

0 10101

1 0101010

10 1011110101

01 0110101110

111000 000111

Binary Numbers

- Every Binary number has a corresponding Decimal value (and vice versa)
- Examples:

Binary Number	Decimal Equivalent
---------------	--------------------

1	1
---	---

10	2
----	---

11	3
----	---

...	...
-----	-----

1010111	87
---------	----

Binary Numbers

- each position for a binary number has a value.
- for each digit, multiply the digit by its position value
- add up all of the products to get the final result
- in general, the "position values" in a binary number are the powers of two.
- The first position value is 2^0 , i.e. one
- The 2nd position value is 2^1 , i.e. two
- The 2nd position value is 2^2 , i.e. four
- The 2nd position value is 2^3 , i.e. eight
- The 2nd position value is 2^4 , i.e. sixteen
- etc.

Example

- The value of binary 01101001 is decimal 105. This is worked out below:

128	64	32	16	8	4	2	1	
0	1	1	0	1	0	0	1	
							1×1	$= 1$
						0×2		$= 0$
				0×4				$= 0$
			1×8					$= 8$
		0×16						$= 0$
	1×32							$= 32$
1×64								$= 64$
0×128								$= 0$
								Answer: 105

Another example

- The value of binary 10011100 is decimal 156. This is worked out below:

128	64	32	16	8	4	2	1		
1	0	0	1	1	1	0	0		
							0×1	= 0	
						0×2		= 0	
					1×4			= 4	
				1×8				= 8	
			1×16					= 16	
		0×32						= 0	
	0×64							= 0	
1×128								= 128	

								Answer:	156

Bitwise Operators (tutorials point)

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) = 12, i.e., 0000 1100
	Binary OR Operator copies a bit if it exists in either operand.	(A B) = 61, i.e., 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) = 49, i.e., 0011 0001
~	Binary Ones Complement Operator is unary and has the effect of 'flipping' bits.	(~A) = -61, i.e., 1100 0011 in 2's complement form.
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 = 240 i.e., 1111 0000
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 = 15 i.e., 0000 1111

Truth Table

p	q	p & q	p q	p ^ q
0	0	0	0	0
0	1	0	1	1
1	1	1	1	0
1	0	0	1	1

The Cast and sizeof operators

By Jason Fedin

Type Conversions

- conversion of data between different types can happen automatically (implicit conversion) by the language or explicit by the program
 - to effectively develop C programs, you must understand the rules used for the implicit conversion of floating-point and integer values in C
- Normally, you shouldn't mix types, but there are occasions when it is useful
 - remember, C is flexible, gives you the freedom, but, do not abuse it
- Whenever a floating-point value is assigned to an integer variable in C, the decimal portion of the number gets truncated

```
int x = 0;
```

```
float f = 12.125;
```

```
x = f; // value stored in x is the number 12, only the int portion is stored
```


Type Conversion (cont'd)

- assigning an integer variable to a floating variable does not cause any change in the value of the number
 - value is converted by the system and stored in the floating variable
- when performing integer arithmetic
 - if two operands in an expression are integers then any decimal portion resulting from a division operation is discarded, even if the result is assigned to a floating variable
 - If one operand is an int and the other is a float then the operation is performed as a floating point operation

The Cast Operator

- As mentioned, you should usually steer clear of automatic type conversions, especially of demotions
 - better to do an explicit conversion
- it is possible for you to demand the precise type conversion that you want
 - called a cast and consists of preceding the quantity with the name of the desired type in parentheses
 - parentheses and type name together constitute a cast operator, i.e. (type)
 - The actual type desired, such as long, is substituted for the word type

The Cast Operator

- The type cast operator has a higher precedence than all the arithmetic operators except the unary minus and unary plus

`(int) 21.51 + (int) 26.99`

- is evaluated in C as

`21 + 26`

sizeof operator

- You can find out how many bytes are occupied in memory by a given type by using the sizeof operator
 - sizeof is a special keyword in C
- sizeof is actually an operator, and not a function
 - evaluated at compile time and not at runtime, unless a variable-length array is used in its argument
- The argument to the sizeof operator can be a variable, an array name, the name of a basic data type, the name of a derived data type, or an expression

sizeof operator

- `sizeof(int)` will result in the number of bytes occupied by a variable of type `int`
- You can also apply the `sizeof` operator to an expression
 - result is the size of the value that results from evaluating the expression
- Use the `sizeof` operator wherever possible to avoid having to calculate and hard-code sizes into your program

Other Operators

- the asterisk “*” is an operator that represents a pointer to a variable.

*a;

- ? : is an operator used for comparisons
 - If Condition is true ? then value X : otherwise value Y
 - name is the ternary operator
- will discuss both of these operators when we talk about pointers and decision statements

Operator Precedence

By Jason Fedin

Overview

- Operator precedence determines the grouping of terms in an expression and decides how an expression is evaluated
 - dictates the order of evaluation when two operators share an operand
 - certain operators have higher precedence than others
 - for example, the multiplication operator has a higher precedence than the addition operator

$x = 7 + 3 * 2;$

- Can result in 13 or 20 depending on the order of each operands evaluation

Overview

- the order of executing the various operations can make a difference, so C needs unambiguous rules for choosing what to do first
- In C, x is assigned 13, not 20 because operator * has a higher precedence than +
 - first gets multiplied with $3*2$ and then adds into 7
- Each operator is assigned a *precedence* level
 - multiplication and division have a higher precedence than addition and subtraction, so they are performed first
- Whatever is enclosed in parentheses is executed first, should just always use () to group expressions

Associativity

- What if two operators have the same precedence?
 - Then associativity rules are applied
- If they share an operand, they are executed according to the order in which they occur in the statement
 - For most operators, the order is from left to right

1 == 2 != 3

Associativity

- operators `==` and `!=` have same precedence
 - associativity of both `==` and `!=` is left to right
 - the expression on the left is executed first and moves towards the right
- the expression above is equivalent to

`((1 == 2) != 3)`

- `(1 == 2)` executes first resulting into 0 (false), then, `(0 != 3)` executes resulting into 1 (true)

Table (highest to lowest) (tutorials point)

Category	Operator	Associativity
Postfix	() [] -> . ++ --	Left to right
Unary	+ - ! ~ ++ -- (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right

Table (highest to lowest)

Category	Operator	Associativity
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= =	Right to left
Comma	,	Left to right

Challenge

By Jason Fedin

Requirements

- In this challenge, you are to create a C program that converts the number of minutes to days and years
- The program should ask the user to enter the number of minutes via the terminal
- The program should display as output the minutes and then its equivalent in years and days
- The program should create variables to store (should all be of type double)
 - minutes (int)
 - minutes in year
 - Years
 - Days
- Need to perform a calculation and use arithmetic operators
 - You have to figure out the conversions 😊

Challenge

By Jason Fedin

Requirements

- In this challenge, you are to create a C program that displays the byte size of basic data types supported in c
 - The output varies depending on the system you are running the program
- Display the byte size of the following types
 - int
 - char
 - long
 - long long
 - double
 - long double
- You can use the %zd format specifier to format each size
- Use the sizeof operator
- Test on more than one computer to see the differences

Overview

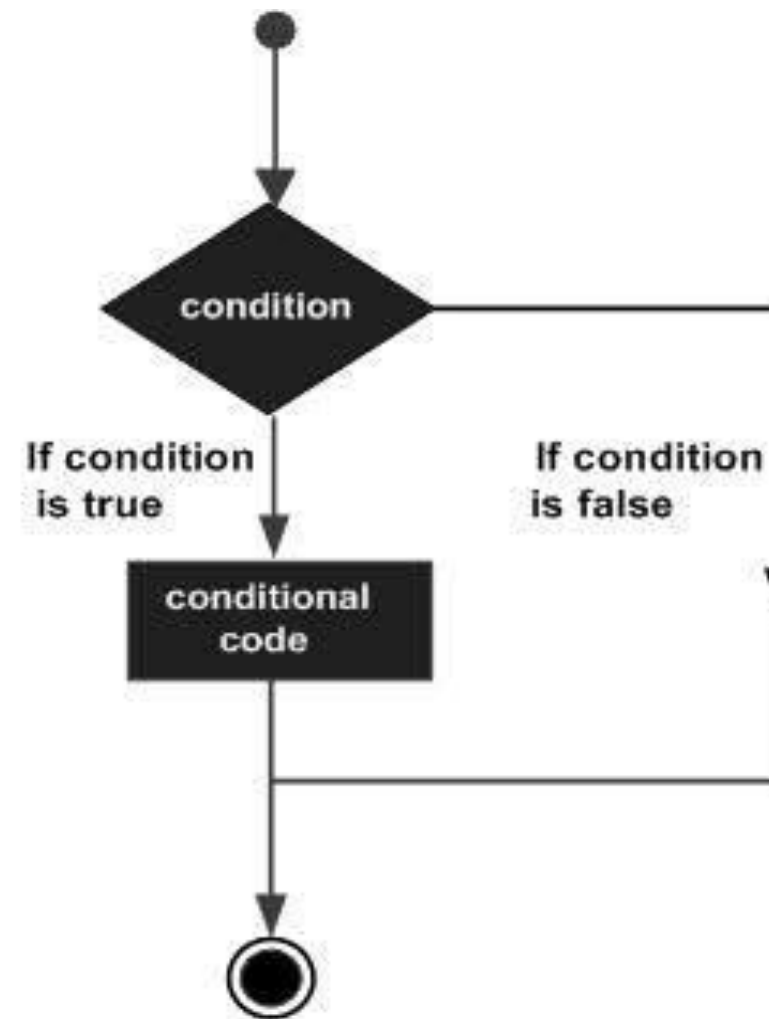
By Jason Fedin

Overview

- The statements inside your source files are generally executed from top to bottom, in the order that they appear
- Control flow statements, however, break up the flow of execution by employing decision making, looping, and branching, enabling your program to conditionally execute particular blocks of code
 - Decision-making statements (if-then, if-then-else, switch, goto)
 - Looping statements (for, while, do-while)
 - branching statements (break, continue, return)

Decision Making

- Structures require that the programmer specify one or more conditions to be evaluated or tested by the program
 - If a condition is true then a statement or statements are executed
 - If a condition is false then other statements are executed



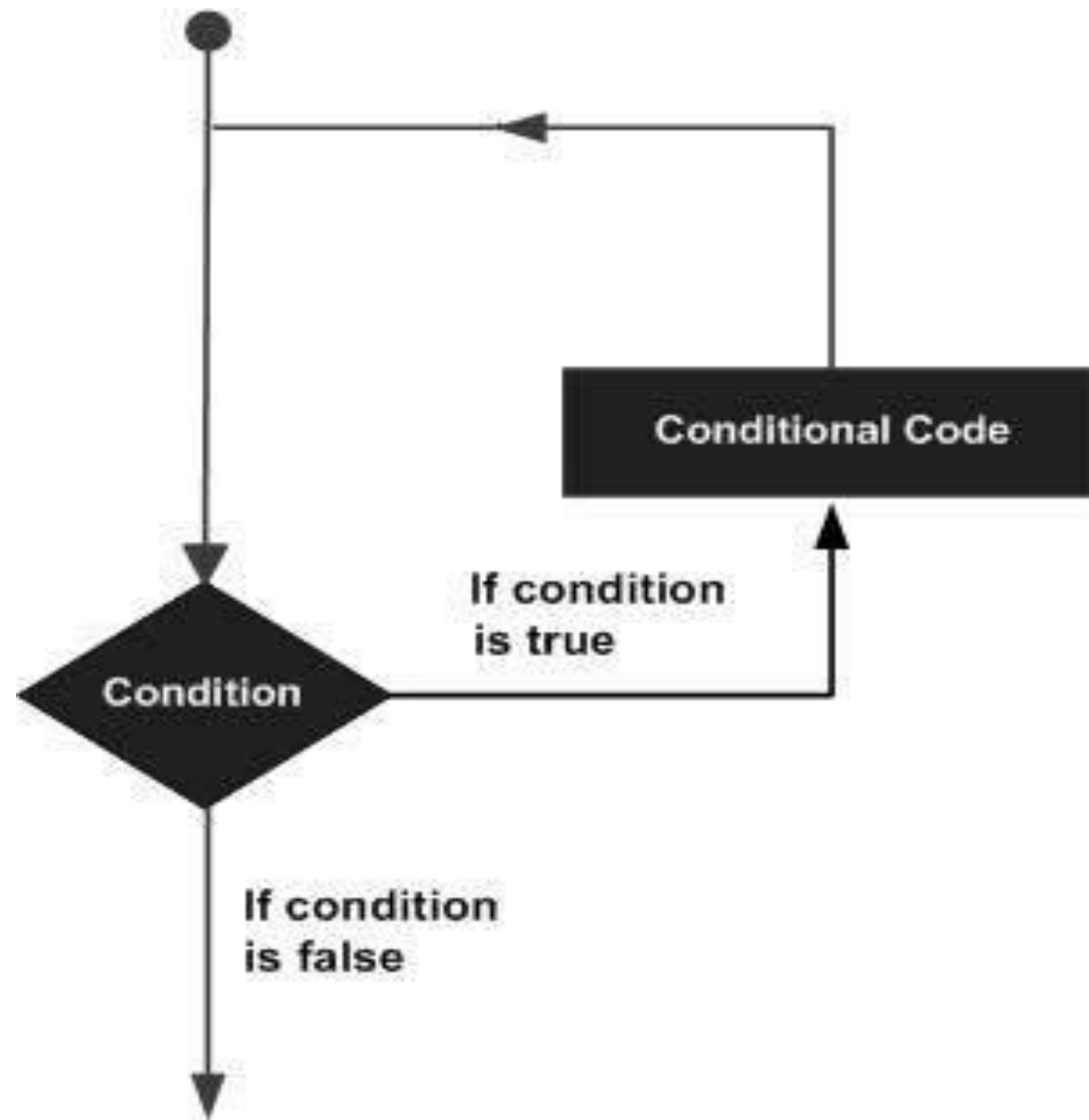
If statements

Statement	Description
if statement	An if statement consists of a boolean expression followed by one or more statements.
if...else statement	An if statement can be followed by an optional else statement, which executes when the boolean expression is false.
nested if statements	You can use one if or else if statement inside another if or else if statement(s)

Repeating Code

- There may be a situation, when you need to execute a block of code several number of times
 - the statements are executed sequentially: The first statement in a function is executed first, followed by the second, and so on
- A loop statement allows us to execute a statement or a group of statements multiple times
- Loop control statements change execution from its normal sequence
 - When execution leaves a scope, all automatic objects that were created in that scope are destroyed (break and continue)
- A loop becomes infinite loop if a condition never becomes false
 - The **for** loop is traditionally used for this purpose

Loop Dataflow



Loops

Loop Type	Description
while loop	It repeats a statement or a group of statements while a given condition is true. It tests the condition before executing the loop body.
for loop	It executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.
do...while loop	It is similar to a while statement, except that it tests the condition at the end of the loop body
nested loops	You can use one or more loop inside any another while, for or do..while loop.

If Statements

By Jason Fedin

Overview

- The C programming language provides a general decision-making capability in the form of a if statement

if (expression)
 program statement

- translating a statement such as “If it is not raining, then I will go swimming” into the C language is easy

if (it is not raining)
 I will go swimming

- The if statement is used to stipulate execution of a program statement/s based upon specified conditions
 - I will go swimming if it is not raining
- The curly brackets are required for compound statements inside the if block

If statement (example)

```
int score = 95;
```

```
int big = 90;
```

```
// simple statement if, no brackets
```

```
if (score > big)  
    printf("Jackpot!\n");
```

```
// compound statement if, brackets
```

```
if (score > big)  
{  
    score++;  
    printf("You win\n");  
}
```

If with an else

- You can extend the if statement with a small addition that gives you a lot more flexibility

If the rain today is worse than the rain yesterday,
I will take my umbrella.

Else

I will take my jacket.

Then I will go to work.

- This is exactly the kind of decision making the if-else statement provides

```
if(expression)
```

```
    Statement1;
```

```
else
```

```
    Statement2;
```

If with an else (example)

- // Program to determine if a number is even or odd

```
#include <stdio.h>
```

```
int main ()
```

```
{
```

```
    int  number_to_test, remainder;
```

```
    printf ("Enter your number to be tested: ");
```

```
    scanf ("%i", &number_to_test);
```

```
    remainder = number_to_test % 2;
```

```
    if ( remainder == 0 )
```

```
        printf ("The number is even.\n");
```

```
    else
```

```
        printf ("The number is odd.\n");
```

```
    return 0;
```

```
}
```


else if

- You can handle additional complex decision making by adding an if statement to your else clause

```
if ( expression 1 )  
    program statement 1  
else  
    if ( expression 2 )  
        program statement 2  
    else  
        program statement 3
```

- The above extends the if statement from a two-valued logic decision to a three-valued logic decision
 - formatted using the else if construct

else if

```
if ( expression 1 )  
    program statement 1  
else if ( expression 2 )  
    program statement 2  
else  
    program statement 3
```

else if example

// Program to implement the sign function

```
#include <stdio.h>
```

```
int main (void)
```

```
{
```

```
    int number, sign;
```

```
    printf ("Please type in a number: ");
```

```
    scanf ("%i", &number);
```

```
    if ( number < 0 )
```

```
        sign = -1;
```

```
    else if ( number == 0 )
```

```
        sign = 0;
```

```
    else // Must be positive
```

```
        sign = 1;
```

```
    printf ("Sign = %i\n", sign);
```

```
    return 0;
```

```
}
```


nested If-else statement

- A nested if-else statement means you can use one if or else if statement inside another if or else if statement(s)

```
if( boolean_expression 1) {
```

```
    /* Executes when the boolean expression 1 is true */
```

```
    if(boolean_expression 2) {
```

```
        /* Executes when the boolean expression 2 is true */
```

```
    }
```

```
}
```

Nested if statement code example

```
if ( gamelsOver == 0 )  
    if ( playerToMove == YOU )  
        printf ("Your Move\n");  
    else  
        printf ("My Move\n");  
else  
    printf ("The game is over\n");
```

The conditional operator (ternary statement)

- the conditional operator is a unique operator
 - unlike all other operators in C
 - most operators are either unary or binary operators
 - is a ternary operator (takes three operands)
- the two symbols that are used to denote this operator are the question mark (?) and the colon (:)
- the first operand is placed before the ?, the second between the ? and the :, and the third after the :
 - condition ? expression1 : expression2

The conditional operator (ternary statement)

- The conditional operator evaluates to one of two expressions, depending on whether a logical expression evaluates true or false
- Notice how the operator is arranged in relation to the operands
 - the ? character follows the logical expression, condition
 - on the right of ? are two operands, expression1 and expression2, that represent choices.
 - the value that results from the operation will be the value of expression1 if condition evaluates to true, or the value of expression2 if condition evaluates to false

Conditional operator (example)

```
x = y > 7 ? 25 : 50;
```

- results in x being set to 25 if y is greater than 7, or to 50 otherwise
- Same as:

```
if(y > 7)  
    x = 25;  
else  
    x = 50;
```

- An expression for the maximum or minimum of two variables can be written very simply using the conditional operator

Switch Statement

By Jason Fedin

Overview

- The conditional operator and the if else statements make it easy to write programs that choose between two alternatives
- However, many times a program needs to choose one of several alternatives
 - you can do this by using if else if...else
 - tedious, prone to errors
- when the value of a variable is successively compared against different values use the switch statement
 - more convenient and efficient

switch syntax

```
switch ( expression )
{
    case value1:
        program statement
        ...
        break;
    case valuen:
        program statement
        program statement
        ...
        break;
    default:
        program statement
        ...
        break;
}
```


switch statement details

- The expression enclosed within parentheses is successively compared against the values: value1, value2, ..., valuen
 - cases must be simple constants or constant expressions
- If a case is found whose value is equal to the value of expression then the statements that follow the case are executed
 - when more than one statement is included, they do not have to be enclosed within braces
- The break statement signals the end of a particular case and causes execution of the switch statement to be terminated
 - include the break statement at the end of every case
 - forgetting to do so for a particular case causes program execution to continue into the next case
- The special optional case called default is executed if the value of expression does not match any of the case values
 - same as a “fall through” else

Switch case example

```
enum Weekday {Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday};  
enum Weekday today = Monday;
```

```
switch(today)  
{  
    case Sunday:  
        printf("Today is Sunday.");  
        break;  
    case Monday:  
        printf("Today is Monday.");  
        break;  
    case Tuesday:  
        printf("Today is Tuesday.");  
        break;  
    default:  
        printf("whatever");  
        break;  
}
```

Another Switch statement example

```
#include <stdio.h>
```

```
int main (void)
```

```
{
```

```
    float value1, value2;
```

```
    char operator;
```

```
    printf ("Type in your expression.\n");
```

```
    scanf ("%f %c %f", &value1, &operator, &value2);
```

```
        switch (operator)
```

```
        {
```

```
            case '+':
```

```
                printf ("%f\n", value1 + value2);
```

```
                break;
```

```
            case '-':
```

```
                printf ("%f\n", value1 - value2);
```

```
                break;
```

```
            case '*':
```

```
                printf ("%f\n", value1 * value2);
```

```
                break;
```

```
            case '/':
```

```
                if ( value2 == 0 )
```

```
                    printf ("Division by zero.\n");
```

```
                else
```

```
                    printf ("%f\n", value1 / value2);
```

```
                break;
```

```
            default:
```

```
                printf ("Unknown operator.\n");
```

```
                break;
```

```
        }
```

```
        return 0;
```

```
    }
```

goto statement

- The goto statement is available in C
 - has two parts—the goto and a label name
 - label is named following the same convention used in naming a variable

goto part2;

- For the above to there must be another statement bearing the part2 label
- you should never need to use the goto statement
 - if you have a background in older versions of FORTRAN or BASIC, you might have developed programming habits that depend on using goto

goto example

- Form:

```
goto label;
```

```
.  
.   
.
```

```
label : statement
```

- Example:

```
top : ch = getchar();
```

```
.  
.   
.
```

```
if (ch != 'y')  
    goto top;
```


Challenge

By Jason Fedin

Requirements

- In this challenge, you are to create a C program that calculates your weekly pay.
- The program should ask the user to enter the number of hours worked in a week via the keyboard
- The program should display as output the gross pay, the taxes, and the net pay
- The following assumptions should be made:
 - Basic pay rate = \$12.00/hr
 - Overtime (in excess of 40 hours) = time and a half
 - Tax rate:
 - 15% of the first \$300
 - 20% of the next \$150
 - 25% of the rest
- You will need to utilize if/else statements

For loop

By Jason Fedin

Overview

- Lets discuss repeating code
 - the C programming language has a few constructs specifically designed to handle these situations when you need to use the same code repeatedly
 - you can repeat a block of statements until some condition is met or a specific number of times
 - repeating code without a condition is a forever/infinite loop
- the number of times that a loop is repeated can be controlled simply by a count
 - repeating the statement block a given number of times (counter controlled loop)
- the number of times that a loop is repeated can depend on when a condition is met
 - the user entering "quit"

for loop

- You typically use the for loop to execute a block of statements a given number of times
- If you want to display the numbers from 1 to 10
 - Instead of writing ten statements that call printf(), you would use a for loop

```
for(int count = 1 ; count <= 10 ; ++count)
{
    printf(" %d", count);
}
```

- The for loop operation is controlled by what appears between the parentheses that follow the keyword for
 - the three control expressions that are separated by semicolons control the operation of the loop
- The action that you want to repeat each time the loop repeats is the block containing the statement that calls printf() (body of the loop)
 - for single statements, you can omit the braces

For syntax (cont'd)

- The general pattern for the for loop is:

```
for(starting_condition; continuation_condition ; action_per_iteration)
    loop_statement;
```

- The statement to be repeated is represented by loop_statement
 - could equally well be a block of several statements enclosed between braces
- The starting_condition usually (but not always) sets an initial value to a loop control variable
 - the loop control variable is typically a counter of some kind that tracks how often the loop has been repeated
 - can also declare and initialize several variables of the same type here with the declarations separated by commas
 - variables will be local to the loop and will not exist once the loop ends

Overview

```
for(starting_condition; continuation_condition ; action_per_iteration)
    loop_statement;
```

- The continuation_condition is a logical expression evaluating to true or false
 - determines whether the loop should continue to be executed
 - as long as this condition has the value true, the loop continues
 - typically checks the value of the loop control variable
 - you can put any logical or arithmetic expression here as long as you know what you are doing
- the continuation_condition is tested at the beginning of the loop rather than at the end
 - means that the loop_statement will not be executed at all if the continuation_condition starts out as false
- The action_per_iteration is executed at the end of each loop iteration
 - usually an increment or decrement of one or more loop control variables
 - can modify several variables here, just need to use commas to separate

Another Example

```
for(int i = 1, j = 2 ; i <= 5 ; ++i, j = j + 2)
    printf(" %5d", i*j);
```

- The output produced by this fragment will be the values 2, 8, 18, 32, and 50 on a single line

For syntax

This expression executes once, when the loop starts.
It declares *count* and initializes it to 1.

This expression is executed at the end of every loop cycle.
It increments *count*.

```
for( int count = 1 ; count <= 10 ; ++count )  
{  
    printf(" %d", count);  
}
```

This expression is evaluated at the beginning of each loop cycle. If it is *true*, the loop continues, and if it is *false*, the loop ends.

for example (flexibility)

```
unsigned long long sum = 0LL;           // Stores the sum of the integers
unsigned int count = 0;                 // The number of integers to be summed
```

```
// Read the number of integers to be summed
printf("\nEnter the number of integers you want to sum: ");
scanf(" %u", &count);
```

```
// Sum integers from 1 to count
for(unsigned int i = 1 ; i <= count ; ++i)
    sum += i;
```

```
// OR
for(unsigned int i = 1 ; i <= count ; sum += i++);
```

```
printf("\nTotal of the first %u numbers is %llu\n", count, sum);
```

Infinite loop

- you have no obligation to put any parameters in the for loop statement

```
for( ;; )  
{  
    /* statements */  
}
```

- the condition for continuing the loop is absent, the loop will continue indefinitely
 - sometimes useful for monitoring data or listening for connections

while and do-while loops

By Jason Fedin

While loop

- the mechanism for repeating a set of statements allows execution to continue for as long as a specified logical expression evaluates to true

While this condition is true
Keep on doing this

OR While you are hungry
 Eat sandwiches

- The general syntax for the while loop is as follows (one statement in body):

while(expression)	or	while (expression)
statement1;		{
		statement1;
		statement2;
		}

While loop (cont'd)

- the condition for continuation of the while loop is tested at the start (top of the loop)
 - pre-test loop
- if expression starts out false, none of the loop statements will be executed
 - If you answer the first question “No, I’m not hungry,” then you don’t get to eat any sandwiches at all, and you move straight to the coffee
- if the loop condition starts out as true, the loop body must contain a mechanism for changing this if the loop is to end

Counter Controlled While loop Example

```
// Program to introduce the while statement
```

```
#include <stdio.h>
```

```
int main (void)
```

```
{
```

```
    int count = 1;
```

```
    while ( count <= 5 ) {
```

```
        printf ("%i\n", count);
```

```
        ++count;
```

```
    }
```

```
    return 0;
```

```
}
```

Logic controlled while loop example

```
int num = 0;
scanf("%d", &num);

while (num != -1)
{
    /* loop actions */
    scanf("%d", &num);
}
```


do-while loop

- In the while loop, the body is executed while the condition is true
- the do-while loop is a loop where the body is executed for the first time unconditionally
 - always guaranteed to execute at least once
 - condition is at the bottom (post-test loop)
- After initial execution, the body is only executed while the condition is true

```
do  
    statement  
while ( expression );
```

```
do  
{  
    prompt for password  
    read user input  
} while (input not equal to password);
```

do-while loop example

```
do
    scanf("%d", &number);
while (number != 20);
```

Or counter controlled

```
int number = 4;
do
{
    printf("\nNumber = %d", number);
    number++;
}
while(number < 4);
```

which loop to use??

- First, decide whether you need a pre or post test loop
 - usually will be a pre test loop (for or while), a bit better option in most cases
 - it is better to look before you leap (or loop) than after
 - easier to read if the loop test is found at the beginning of the loop
 - in many uses, it is important that the loop be skipped entirely if the test is not initially met
- So, should you use a for or a while
 - a matter of taste, because what you can do with one, you can do with the other
 - To make a for loop like a while, you can omit the first and third expressions

`for ( ;test; )`

is the same as

`while (test)`

do-while loop example

- To make a while like a for, preface it with an initialization and include update statements

```
initialize;  
while (test)  
{  
    body;  
    update;  
}
```

is the same as

```
for (initialize; test; update)  
    body;
```

do-while loop example

- a for loop is appropriate when the loop involves initializing and updating a variable
- a while loop is better when the conditions are otherwise
- I usually use the while loop for logic controlled loops and the for loop for counter controlled loops

```
while (scanf("%l", &num) == 1)
```

```
for (count = 1; count <= 100; count++)
```

Nested loops and loop control

By Jason Fedin

Nested Loops

- Sometimes you may want to place one loop inside another
- You might want to count the number of occupants in each house on a street
 - step from house to house, and for each house you count the number of occupants
- Going through all the houses could be an outer loop, and for each iteration of the outer loop you would have an inner loop that counts the occupants

Example

```
for(int i = 1 ; i <= count ; ++i)
{
    sum = 0;                // Initialize sum for the inner loop

    // Calculate sum of integers from 1 to i
    for(int j = 1 ; j <= i ; ++j)
        sum += j;

    printf("\n%d\t%d", i, sum);    // Output sum of 1 to i
}
```


Another Example (while inside a for)

```
for(int i = 1 ; i <= count ; ++i)
{
    sum = 1;                // Initialize sum for the inner loop
    j=1;                    // Initialize integer to be added
    printf("\n1");

    // Calculate sum of integers from 1 to i
    while(j < i)
    {
        sum += ++j;
        printf(" + %d", j);    // Output +j – on the same line
    }
    printf(" = %d", sum);      // Output = sum
}
```

Continue statements

- Sometimes a situation arises where you do not want to end a loop, but you want to skip the current iteration
- The continue statement in the body of a loop does this
 - All you need to do is use the keyword “continue;” in the body of the loop
- An advantage of using continue is that it can sometimes eliminate nesting or additional blocks of code
 - can enhance readability when the statements are long or are deeply nested already
- Don't use continue if it complicates rather than simplifies the code

Continue Example

```
enum Day { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday};
```

```
for(enum Day day = Monday; day <= Sunday ; ++day)
```

```
{
```

```
    if(day == Wednesday)
```

```
        continue;
```

```
    printf("It's not Wednesday!\n");
```

```
    /* Do something useful with day */
```

```
}
```


Break statement

- normally, after the body of a loop has been entered, a program executes all the statements in the body before doing the next loop test
 - we learned how continue works
 - another statement named break alters this behavior
- the break statement causes the program to immediately exit from the loop it is executing
 - statements in the loop are skipped, and execution of the loop is terminated
 - if the break statement is inside nested loops, it affects only the innermost loop containing it
 - use the keyword “break;”
- break is often used to leave a loop when there are two separate reasons to leave
- break is also used in switch statements

Break example

```
while ( p > 0)
{
    printf("%d\n", p);
    scanf("%d", &q);
    while( q > 0)
    {
        printf("%d\n",p*q);
        if (q > 100)
            break;          // break from inner loop
        scanf("%d", &q);
    }
    if (q > 100)
        break;              // break from outer loop
    scanf("%d", &p);
}
```

Challenge

By Jason Fedin

Requirements

- In this challenge, you are going to create a “Guess the Number” C program
- Your program will generate a random number from 0 to 20
- You will then ask the user to guess it
 - User should only be able to enter numbers from 0-20
- The program will indicate to the user if each guess is too high or too low
- The player wins the game if they can guess the number within five tries

Sample Output

This is a guessing game.

I have chosen a number between 0 and 20 which you must guess.

You have 5 tries left.

Enter a guess: 12

Sorry, 12 is wrong. My number is less than that.

You have 4 tries left.

Enter a guess: 8

Sorry, 8 is wrong. My number is less than that.

You have 3 tries left.

Enter a guess: 4

Sorry, 4 is wrong. My number is less than that.

You have 2 tries left.

Enter a guess: 2

Congratulations. You guessed it!

Generating a random number

- To generate a random number from 0-20
 - include the correct system libraries
 - `#include <stdlib.h>`
 - `#include <time.h>`
 - Create a time variable
 - `time_t t;`
 - Initialize the random number generator
 - `srand((unsigned) time(&t));`
 - Get the random number (0-20) and store in an int variable
 - `int randomNumber = rand() % 21;`

Creating and Using Arrays

By Jason Fedin

Arrays

- it is very common to in a program to store many data values of a specified type
 - In a sports program, you might want to store the scores for all games or the scores for each player
 - you could write a program that does this using a different variable for each score
 - If there are a lot of games to store then this is very tedious
 - using an array will solve this problem
- arrays allow you to group values together under a single name
 - you do not need separate variables for each item of data
- an array is a fixed number of data items that are all of the same type

Declaring an Array

- the data items in an array are referred to as elements
- the elements in an array have to be the same type (int, long, double, etc)
 - You cannot “mix” data types, no such thing as a single array of ints and doubles
- declaring to use an array in a program is similar to a normal variable that contains a single value
 - difference is that you need a number between square brackets [] following the name

`long numbers[10];`

- the number between square brackets defines how many elements the array contains
 - called the size of the array or the array dimension.

Accessing an array's elements

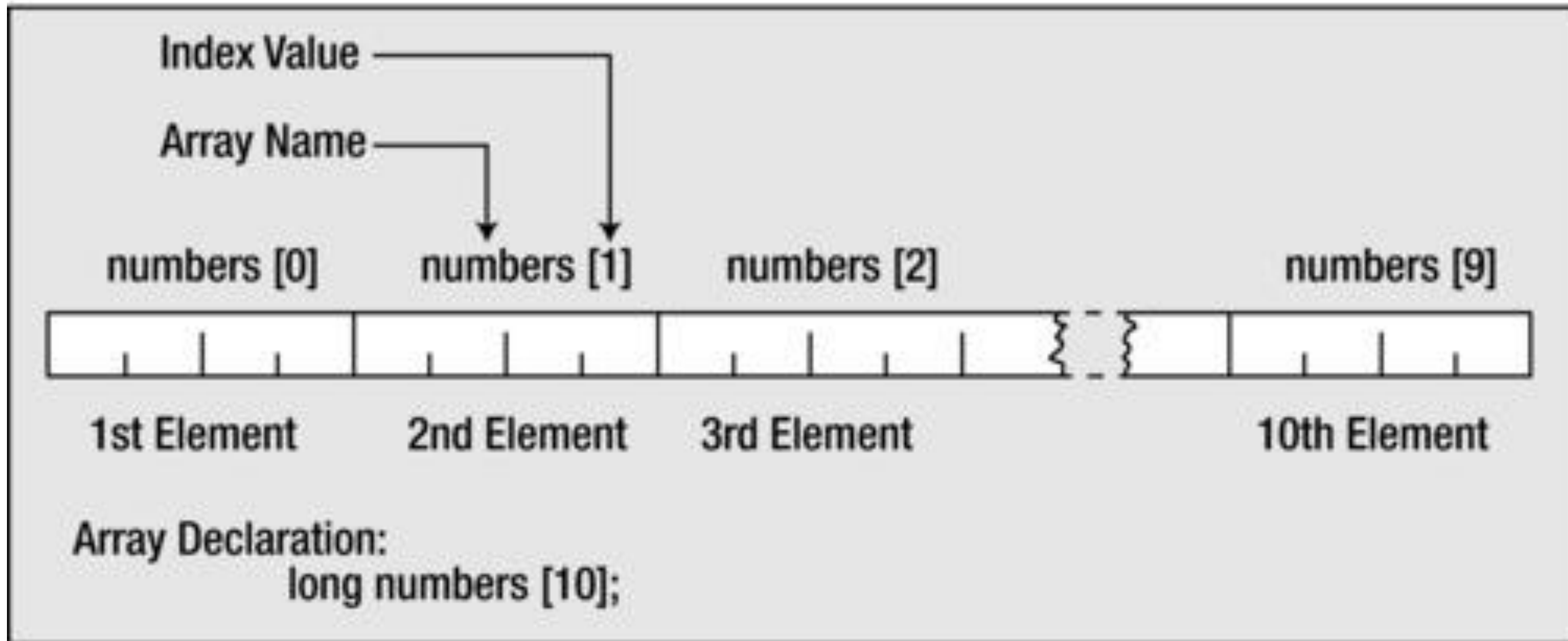
- each of the data items stored in an array is accessed by the same name
- you select a particular element by using an index (subscript) value between square brackets following the array name
- index values are sequential integers that start from zero
 - index values for elements in an array of size 10 would be from 0-9
 - Arrays are zero based
 - 0 is the index value for the first array element
 - for an array of 10 elements, index value 9 refers to the last element

Accessing an array's elements

- It is a very common mistake to assume that arrays start from one
 - referred to as the off-by-one error
 - you can use a simple integer to explicitly reference the element that you want to access
 - to access the fourth value in an array, you use the expression `arrayName[3]`
- You can also specify an index for an array element by an expression in the square brackets following the array name
 - the expression must result in an integer value that corresponds to one of the possible index values
- it is very common to use a loop to access each element in an array

```
for ( i = 0; i < 10; ++i )  
    printf("Number is %d", numbers[i]);
```

Accessing an array's elements



Array out of bounds

- if you use an expression or a variable for an index value that is outside the range for the array, your program may crash or the array can contain garbage data
 - referred to as an out of bounds error
- the compiler cannot check for out of bounds errors so your program will still compile
- very important to ensure that your array indexes are always within bounds

Assigning values to an Array

- a value can be stored in an element of an array simply by specifying the array element on the left side of an equal sign

```
grades[100] = 95;
```

- the value 95 is stored in element number 100 of the grades array
- can also use variables to assign values to an array

Example of using an array

```
int main(void)
{
    int grades[10];           // Array storing 10 values
    int count = 10;           // Number of values to be read
    long sum = 0;              // Sum of the numbers
    float average = 0.0f;      // Average of the numbers

    printf("\nEnter the 10 grades:\n"); // Prompt for the
    input

    // Read the ten numbers to be averaged
    for(int i = 0 ; i < count ; ++i)
    {
        printf("%2u> ", i + 1);
        scanf("%d", &grades[i]);           // Read a grade
        sum += grades[i];                   // Add it to sum
    }

    average = (float)sum/count;             // average
    printf("\nAverage of the ten grades entered is:
    %.2f\n", average);

    return 0;
}
```

Initializing an array

Jason Fedin

Initializing an array

- you will want to assign initial values for the elements of your array most of the time
 - defining initial values for array elements makes it easier to detect when things go wrong
- just as you can assign initial values to variables when they are declared, you can also assign initial values to an array's elements
- to initialize an array's values, simply provide the values in a list
 - values in the list are separated by commas and the entire list is enclosed in a pair of braces

```
int counters[5] = { 0, 0, 0, 0, 0 };
```

- declares an array called counters to contain five integer values and initializes each of these elements to zero

```
int integers[5] = { 0, 1, 2, 3, 4 };
```

- declares an array named integers and sets the value of integers[0] to 0, integers[1] to 1, integers[2] to 2, and so on

Initializing an array (cont'd)

- It is not necessary to completely initialize an entire array
- If fewer initial values are specified, only an equal number of elements are initialized
 - remaining values in the array are set to zero

```
float sample_data[500] = { 100.0, 300.0, 500.5 };
```

- initializes the first three values of sample_data to 100.0, 300.0, and 500.5, and sets the remaining 497 elements to zero

Designated Initializers

- C99 added a feature called designated initializers
 - allows you to pick and choose which elements are initialized
- by enclosing an element number in a pair of brackets, specific array elements can be initialized in any order

```
float sample_data[500] = { [2] = 500.5, [1] = 300.0, [0] = 100.0 };
```

- initializes the sample_data array to 100.0, 300.0, and 500.5 for the first three values

```
int arr[6] = {[5] = 212}; // initialize arr[5] to 212
```

Example of traditional initialization

```
#define MONTHS 12
```

```
int main(void)
```

```
{
```

```
    int days[MONTHS] = {31,28,31,30,31,30,31,31,30,31,30,31};
```

```
    int index;
```

```
    for (index = 0; index < MONTHS; index++)
```

```
        printf("Month %d has %2d days.\n", index +1, days[index]);
```

```
    return 0;
```

```
}
```

Example using designated initializers

```
#include <stdio.h>
#define MONTHS 12
int main(void)
{
    int days[MONTHS] = {31,28, [4] = 31,30,31, [1] = 29};
    int i;

    for (i = 0; i < MONTHS; i++)
        printf("%2d  %d\n", i + 1, days[i]);

    return 0;
}
```


Repeating an initial value

- C does not provide any shortcut mechanisms for initializing array elements
- no way to specify a repeat count
- if it were desired to initially set all 500 values of `sample_data` to 1, all 500 would have to be explicitly assigned
- to solve this problem, you will want to initialize the array inside the program using a loop

Initializing all elements to the same value

```
int main (void)
{
    int array_values[10] = { 0, 1, 4, 9, 16 };
    int i;

    for ( i = 5; i < 10; ++i )
        array_values[i] = i * i;

    for ( i = 0; i < 10; ++i )
        printf ("array_values[%i] = %i\n", i, array_values[i]);

    return 0;
}
```

Multidimensional arrays

By Jason Fedin

Overview

- the types of arrays that you have been exposed to so far are all linear arrays
 - a single dimension
- the C language allows arrays of any dimension to be defined
 - two dimensional arrays are the most common
- you can visualize a two-dimensional array as a rectangular arrangement like rows and columns in a spreadsheet
- one of the most natural applications for a two-dimensional array arises in the case of a matrix
- two-dimensional arrays are declared the same way that one-dimensional arrays are

```
int matrix[4][5];
```

- declares the array matrix to be a two-dimensional array consisting of 4 rows and 5 columns, for a total of 20 elements
 - note how each dimension is between its own pair of square brackets

Initializing a two dimensional array

- two-dimensional arrays can be initialized in the same manner of a one-dimensional array
- When listing elements for initialization, the values are listed by row
 - the difference is that you put the initial values for each row between braces, {}, and then enclose all the rows between braces

```
int numbers[3][4] = {  
    { 10, 20, 30, 40 },    // Values for first row  
    { 15, 25, 35, 45 },    // Values for second row  
    { 47, 48, 49, 50 }     // Values for third row  
};
```

- commas are required after each brace that closes off a row, except in the case of the final row
- the use of the inner pairs of braces is actually optional, but, should be used for readability

Initializing a 2D array

- as with one-dimensional arrays, it is not required that the entire array be initialized

```
int matrix[4][5] = {  
    { 10, 5, -3 },  
    { 9, 0, 0 },  
    { 32, 20, 1 },  
    { 0, 0, 8 }  
};
```

- only initializes the first three elements of each row of the matrix to the indicated values
 - remaining values are set to 0.
 - in this case, the inner pairs of braces are required to force the correct initialization

Designated initializers

- subscripts can also be used in the initialization list, in a like manner to single-dimensional arrays

```
int matrix[4][3] = { [0][0] = 1, [1][1] = 5, [2][2] = 9 };
```

- initializes the three indicated elements of matrix to the specified values
 - unspecified elements are set to zero by default
 - each set of values that initializes the elements in a row is between braces
 - the entire initialization goes between another pair of braces
 - the values for a row are separated by commas
 - each set of row values is separated from the next set by a comma

Other dimensions

- everything mentioned so far about two-dimensional arrays can be generalized to three-dimensional arrays and further
- you can declare a three-dimensional array this way:

```
int box[10][20][30];
```

- you can visualize a one-dimensional array as a row of data
- you can visualize a two-dimensional array as a table of data, matrix, or a spreadsheet
- you can visualize a three-dimensional array as a stack of data tables
- typically, you would use three nested loops to process a three-dimensional array, four nested loops to process a four-dimensional array, and so on

Initializing an array of more than 2 dimensions

- for arrays of three or more dimensions, the process of initialization is extended
- a three-dimensional array will have three levels of nested braces, with the inner level containing sets of initializing values for a row

```
int numbers[2][3][4] = {  
    {           // First block of 3 rows  
        { 10, 20, 30, 40 },  
        { 15, 25, 35, 45 },  
        { 47, 48, 49, 50 }  
    },  
    {           // Second block of 3 rows  
        { 10, 20, 30, 40 },  
        { 15, 25, 35, 45 },  
        { 47, 48, 49, 50 }  
    }  
};
```

Processing elements in a n dimensional array

- you need a nested loop to process all the elements in a multidimensional array
 - the level of nesting will be the number of array dimensions
 - each loop iterates over one array dimension

```
int sum = 0;
for(int i = 0 ; i < 2 ; ++i)
{
    for(int j = 0 ; j < 3 ; ++j)
    {
        for(int k = 0 ; k < 4 ; ++k)
        {
            sum += numbers[i][j][k];
        }
    }
}
```

Variable Length Arrays

Jason Fedin

Variable length arrays

- so far, all the sizes of an array have been specified using a number
- the term variable in variable-length array does not mean that you can modify the length of the array after you create it
- a VLA keeps the same size after creation
- variable length arrays allow you to specify the size of an array with a variable when creating an array
- C99 introduced variable-length arrays primarily to allow C to become a better language for numerical computing
- VLAs make it easier to convert existing libraries of FORTRAN numerical calculation routines to C
- you can not initialize a VLA in its declaration

Valid and invalid declarations of an array

```
int n = 5;
int m = 8;
float a1[5];           // yes
float a2[5*2 + 1];     // yes
float a3[sizeof(int) + 1]; // yes
float a4[-4];          // no, size must be > 0
float a5[0];           // no, size must be > 0
float a6[2.5];         // no, size must be an integer
float a7[(int)2.5];     // yes, typecast float to int constant
float a8[n];           // not allowed before C99, VLA
float a9[m];           // not allowed before C99, VLA
```

Challenge

By Jason Fedin

Requirements

- In this challenge, you are going to create a program that will find all the prime numbers from 3-100
- there will be no input to the program
- The output will be each prime number separated by a space on a single line
- You will need to create an array that will store each prime number as it is generated
- You can hard-code the first two prime numbers (2 and 3) in the primes array
- You should utilize loops to only find prime numbers up to 100 and a loop to print out the primes array

Hints

- The criteria that can be used to identify a prime number is that a number is considered prime if it is not evenly divisible by any other previous prime numbers
- Can use the following as an exit condition in the innermost loop
 - $p / \text{primes}[i] \geq \text{primes}[i]$
 - a test to ensure that the value of p does not exceed the square root of $\text{primes}[i]$
- Your program can be more efficient by skipping any checks for even numbers (as they cannot be prime)

Challenge

By Jason Fedin

Requirements

- In this challenge, you are to create a C program that uses a two-dimensional array in a weather program.
- This program will find the total rainfall for each year, the average yearly rainfall, and the average rainfall for each month
- Input will be a 2D array with hard-coded values for rainfall amounts for the past 5 years
 - The array should have 5 rows and 12 columns
 - rainfall amounts can be floating point numbers

Example output

YEAR	RAINFALL (inches)
2010	32.4
2011	37.9
2012	49.8
2013	44.0
2014	32.9

The yearly average is 39.4 inches.

MONTHLY AVERAGES:

Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
7.3	7.3	4.9	3.0	2.3	0.6	1.2	0.3	0.5	1.7	3.6	6.7

hints

- Initialize your 2D array with hard-coded rainfall amounts
- remember, to iterate through a 2D array you will need a nested loop
- the key to this solution will be to visualize a 2D array and understand how to iterate through one, via a nested loop
- as you are iterating, you can keep a running total (outer loop iterate by year, inner loop iterate by month) to get the total rainfall for all years
- to get the average monthly rainfalls, iterate through the 2D array by having the outer loop go through each month and the inner loop go through each year

Function Basics

By Jason Fedin

Overview

- A function is a self-contained unit of program code designed to accomplish a particular task
- Syntax rules define the structure of a function and how it can be used
- A function in C is the same as subroutines or procedures in other programming languages
- Some functions cause an action to take place
 - `printf()` causes data to be printed on your screen
- Some functions find a value for a program to use
 - `strlen()` tells a program how long a certain string is

Advantages

- allow for the divide and conquer strategy
 - it is very difficult to write an entire program as a single large main function
 - difficult to test, debug and maintain
- with divide and conquer, tasks can be divided into several independent subtasks
 - reduces the overall complexity
 - separate functions are written for each subtask
 - we can further divide each subtask into smaller subtasks, further reducing the complexity
- reduce duplication of code
 - saves you time when writing, testing, and debugging code
 - reduces the size of the source code
- If you have to do a certain task several times in a program, you only need to write an appropriate function once
 - program can then use that function wherever needed
 - you can also use the same function in different programs (printf)

Advantages (cont'd)

- helps with readability
 - program is better organized
 - easier to read and easier to change or fix
- the divide and conquer approach also allows the parts of a program to be developed, tested and debugged independently
 - reduces the overall development time
- the functions developed for one program can be used in another program
 - software reuse
- many programmers like to think of a function as a “black box”
 - information that goes in (its input)
 - the value or action it produces (its output)
- using this “black box” thinking helps you concentrate on the program’s overall design rather than the details
 - what the function should do and how it relates to the program as a whole before worrying about writing the code

Examples

- You have already used built-in functions such as `printf()` and `scanf()`
- You should have noticed how to invoke these functions and pass data to them
 - arguments between parentheses following the function name
- e.g.
 - `printf()`
 - first argument is usually a string literal, and the succeeding arguments (of which there may be none) are a series of variables or expressions whose values are to be displayed
- You also should have noticed how you can receive information back from a function in two ways
 - through one of the function arguments (`scanf`)
 - the input is stored in an address that you supply as an argument
 - as a return value

Example

```
#define SIZE 50
int main(void)
{
    float list[SIZE];

    readlist(list, SIZE);
    sort(list, SIZE);
    average(list, SIZE);
    return 0;
}
```

Implementing functions

- remember, just calling functions does not work unless we implement the function itself
 - user defined functions
 - we would have to write the code for the three functions `readlist()`, `sort()`, and `average()` in our previous examples
- always use descriptive function names to make it clear what the program does and how it is organized
 - If you can make the functions general enough, you can reuse them in other programs
- in the upcoming lectures we will understand
 - how to define them
 - how to invoke them
 - how to pass and return data from them

main() function

- as a reminder, the main() is a specially recognized name in the C system
 - indicates where the program is to begin execution
 - all C programs must always have a main()
 - can pass data to it (command line arguments)
 - returning data optional (error code)

Defining Functions

By Jason Fedin

Defining a function

- when you create a function, you specify the function header as the first line of the function definition
 - followed by a starting curly brace {
 - The executable code in between the starting and ending braces
 - The ending curly brace }
 - the block of code between braces following the function header is called the function body.
- the function header defines the name of the function
 - parameters (which specify the number and types of values that are passed to the function when it's called)
 - the type for the value that the function returns
- the function body contains the statements that are executed when the function is called
 - have access to any values that are passed as arguments to the function

```
Return_type Function_name( Parameters - separated by commas )  
{  
    // Statements...  
}
```

Defining a function (cont'd)

- the first line of a function definition tells the compiler (in order from left to right) three things about the function
 - the type of value it returns
 - its name
 - the arguments it takes
- choosing meaningful function names is just as important as choosing meaningful variable names
 - greatly affects the program's readability

```
void printMessage (void)
{
    printf ("Programming is fun.\n");
}
```

Example

```
void printMessage (void)
{
    printf ("Programming is fun.\n");
}
```

- the first line of the printMessage() function definition tells the compiler that the function returns no value
 - keyword void
- next is its name: printMessage
- after that is that it takes no arguments (the second use of the keyword void)

Defining a function

- the statements in the function body can be absent, but the braces must be present
- If there are no statements in the body of a function, the return type must be void, and the function will not do anything
 - defining a function with an empty body is often useful during the testing phase of a complicated program
 - allows you to run the program with only selected functions actually doing something
 - you can then add the detail for the function bodies step by step, testing at each stage, until the whole thing is implemented and fully tested

Naming functions

- the name of a function can be any legal name
 - not a reserved word (such as int, double, sizeof, and so on)
 - Is not the same name as another function in your program.
 - Is not the same name as any of the standard library functions
 - would prevent you from using the library function
- a legal name has the same form as that of a variable
 - a sequence of letters and digits
 - first character must be a letter
 - underline character counts as a letter
- the name that you choose should be meaningful and relevant to what the function does

Naming functions (cont'd)

- you will often define function names (and variable names, too) that consist of more than one word
- there are three common approaches you can adopt
 - separate each of the words in a function name with an underline character
 - capitalize the first letter of each word
 - capitalize words after the first (camelCase)
- can pick any one you want, but, use the same approach throughout your program

Function prototypes

- a function prototype is a statement that defines a function
 - defines its name, its return value type, and the type of each of its parameters
 - provides all the external specifications for the function
- you can write a prototype for a function exactly the same as the function header
 - only difference is that you add a semicolon at the end

`void printMessage (void);`

- a function prototype enables the compiler to generate the appropriate instructions at each point where you call the function
 - It also checks that you are using the function correctly in each invocation
- when you include a standard header file in a program, the header file adds the function prototypes for that library to your program
 - the header file `stdio.h` contains function prototypes for `printf()`, among others

Function prototypes (cont'd)

- generally appear at the beginning of a source file prior to the implementations of any functions or in a header file
- allows any of the functions in the file to call any function regardless of where you have placed the implementation of the functions
- parameter names do not have to be the same as those used in the function definition
 - not required to include the names of parameters in a function prototype
- its good practice to always include declarations for all of the functions in a program source file, regardless of where the are called
 - will help keep your programs more consistent in design
 - prevent any errors from occurring if, at any stage, you choose to call a function from another part of your program

Example

```
// #include & #define directives...
```

```
// Function prototypes
```

```
double Average(double data_values[], size_t count);
```

```
double Sum(double *x, size_t n);
```

```
size_t GetData(double*, size_t);
```

```
int main(void)
```

```
{
```

```
    // Code in main() ...
```

```
}
```

```
// Definitions/implementations for Average(), Sum() and GetData()...
```

Arguments and Parameters

By Jason Fedin

Arguments and parameters

- a parameter is a variable in a function declaration and function definition/implementation
- when a function is called, the arguments are the data you pass into the functions parameters.
 - the actual value of a variable that gets passed to the function
- function parameters are defined within the function header
 - are placeholders for the arguments that need to be specified when the function is called
- the parameters for a function are a list of parameter names with their types
 - each parameter is separated by a comma
 - entire list of parameters is enclosed between the parentheses that follow the function name
- a function can have no parameters, in which case you should put void between the parentheses

Arguments and parameters (cont'd)

- parameters provide the means to pass data to a function
 - data passed from the calling function to the function that is called
- the names of the parameters are local to the function
 - they will assume the values of the arguments that are passed when the function is called
- the body of the function should use these parameters in its implementation
- a function body may have additional locally defined variables that are needed by the function's implementation
- when passing an array as an argument to a function
 - you must also pass an additional argument specifying the size of the array
 - the function has no means of knowing how many elements there are in the array

Example

- when the `printf()` function is called, you always supply one or more values as arguments
 - first value being the format string
 - the remaining values being any variables to displayed
- parameters greatly increase the usefulness and flexibility of a function
 - the `printf()` function displays whatever you tell it to display via the parameters and arguments passed
- It is a good idea to add comments before each of your own function definitions
 - help explain what the function does and how the arguments are to be used

Example

```
#include <stdio.h>
```

```
void multiplyTwoNumbers (int x, int y)
{
    int result = x * y;
    printf ("The product of %d multiplied by %d is: %d\n", x, y, result);
}
```

```
int main (void)
{
    multiplyTwoNumbers (10, 20);
    multiplyTwoNumbers (20, 30);
    multiplyTwoNumbers (50, 2);

    return 0;
}
```


returning data from functions

By Jason Fedin

Returning data

- In our prior example of multiply two numbers, our multiply function displayed the results of the calculation at the terminal
- you might not always want to have the results of your calculations displayed
 - functions can return data using specific syntax
 - should be familiar from previous experience with the main function
- let's take another look at the general form of a function

```
Return_type Function_name(List of Parameters - separated by commas)
{
    // Statements...
}
```

- the Return_type specifies the type of the value returned by the function
-

The return type

```
Return_type Function_name(List of Parameters - separated by commas)
{
    // Statements...
}
```

- you can specify the type of value to be returned by a function as any of the legal types in C
 - includes enumeration types and pointers
- the return type can also be type void which means no value is returned

The return statement

- the return statement provides the means of exiting from a function

`return;`

- this form of the return statement is used exclusively in a function where the return type has been declared as void

- does not return a value

- the more general form of the return statement is:

`return expression;`

- this form of return statement must be used when the return value type for the function has been declared as some type other than void

- the value that is returned to the calling program is the value that results when expression is evaluated
 - should be of the return type specified for the function

Returning data

- a function that has statements in the function body but does not return a value must have the return type as void
 - will get an error message if you compile a program that contains a function with a void return type that tries to return a value
- a function that does not have a void return type must return a value of the specified return type
 - will get an error message from the compiler if return type is different than specified
- If expression results in a value that's a different type from the return type in the function header, the compiler will insert a conversion from the type of expression to the one required
 - If conversion is not possible then the compiler will produce an error message
- there can be more than one return statement in a function
 - each return statement must supply a value that is convertible to the type specified in the function header for the return value

Invoking a function

- you call a function by using the function name followed by the arguments to the function between parentheses
- when you call the function, the values of the arguments that you specify in the call will be assigned to the parameters in the function
- when the function executes, the computation proceeds using the values you supplied as arguments
- the arguments you specify when you call a function should agree in type, number, and sequence with the parameters in the function header.

Invoking a function and assigning data returned

- If the function is used as the right side of an assignment statement, the return value supplied by the function will be substituted for the function
 - will also work with an expression

```
int x = myFunctionCall();
```

- the calling function doesn't have to recognize or process the value returned from a called function
 - up to you how you use any values returned from function calls

Example

```
int multiplyTwoNumbers (int x, int y)
{
    int result = x * y;
    return result;
}
```

```
int main (void)
{
    int result = 0;
    result = multiplyTwoNumbers (10, 20);

    printf("result is %d\n", result);
    return 0;
}
```


Local and Global Variables

By Jason Fein

Local

Variables

Variables defined inside a function are known as automatic local variables

- they are automatically “created” each time the function is called
- their values are local to the function
- the value of a local variable can only be accessed by the function in which the variable is defined
 - its value cannot be accessed by any other function
- if an initial value is given to a variable inside a function, that initial value is assigned to the variable each time the function is called
- can use the auto keyword to be more precise, but, not necessary, as the compiler adds this by default
- local variables are also applicable to any code where the variable is created in a block (loops, if statements)

Global

Variables

the opposite of a local variable

- global variables value can be accessed by any function in the program
- A global variable has the lifetime of the program
- global variables are declared outside of any function
 - does not belong to any particular function
- any function in the program can change the value of a global variable
- if there is a local variable declared in a function with the same name, then, within that function the local variable will mask the global variable
 - global variable is not accessible and prevent it from being accessed normally.

Example

```
int myglobal = 0;    // global variable
```

```
int main ()  
{  
    int myLocalMain = 0;    // local variable  
    // can access my global and myLocal  
    return 0;  
}
```

```
void myFunction()  
{  
    int x;    // local variable  
    // can access myGlobal and x, cannot access myLocal  
}
```

Avoid using global variables

- in general, global variables are a “bad” thing and should be avoided
 - promotes coupling between functions (dependencies)
 - hard to find the location of a bug in a program
 - hard to fix a bug once its found
- use parameters in functions instead
 - If a lot of data, use a struct

Challenge

By Jason Fedin

Requirements

- we need to get some practice writing functions
 - better organized code
 - avoid duplication
- for this challenge you are to write three functions in a single program
- write a function which finds the greatest common divisor of two non-negative integer values and to return the result
 - gcd, takes two ints as parameters
- write a function to calculate the absolute value of a number
 - should take as a parameter a float and return a float
 - test this function with ints and floats
- write a function to compute the square root of a number
 - if a negative argument is passed then a message is display and -1.0 should be returned
 - should use the absoluteValue function as implemented in the above step
- Test! Test! Test!

Challenge

By Jason Fedin

Requirements

- write a program that plays tic-tac-toe
 - game is played on a 3x3 grid the game is played by two players, who take turns
- you should create an array to represent the board
 - can be of type char and consist of 10 elements (do not use zero)
 - each element represents a coordinate on the board that the user can select
- some functions that you should probably create
 - checkForWin - checks to see if a player has won or the game is a draw
 - drawboard – redraws the board for each player turn
 - markBoard - sets the char array with a selection and check for an invalid selection

Hints

- Demo game play

Overview

Jason Fedin

Strings

- we have learned all about the char data type
 - contains a single character.
- to assign a single character to a char variable, the character is enclosed within a pair of single quotation marks

```
plusSign = '+';
```

- you have also learned that there is a distinction made between the single quotation and double quotation marks
 - `plusSign = "+";` // incorrect if plusSign is a char
- a string constant or string literal is a sequence of characters or symbols between a pair of double-quote characters
 - anything between a pair of double quotes is interpreted by the compiler as a string
 - includes any special characters and embedded spaces

Strings (cont'd)

- every time you have displayed a message using the `printf()` function, you have defined the message as a string constant

```
printf("This is a string.");  
printf("This is on\ntwo lines!");  
printf("For \" you write \\\").");
```

- understand the difference between single quotation and double quotation marks
 - both are used to create two different types of constants in C
- for the third example above, you must specify a double quote within a string as the escape sequence `\"`
 - the compiler will interpret an explicit double quote without a preceding backslash as a string delimiter
- also, you must also use the escape sequence `\\` when you want to include a backslash in a string
 - a backslash in a string always signals the start of an escape sequence to the compiler

String in Memory

"This is a string."

T	h	i	s		i	s		a		s	t	r	i	n	g	.	\0
84	104	105	115	32	105	115	32	97	32	115	116	114	105	110	103	46	0

"This is on\ntwo lines."

T	h	i	s		i	s		o	n	\n	t	w	o		l	i	n	e	s	.	\0
84	104	105	115	32	105	115	32	111	110	10	116	119	111	32	108	105	110	101	115	46	0

"For \" you write \\\"."

F	o	r		"		w	r	i	t	e		\	"	.	\0
70	111	114	32	34	32	119	114	105	116	101	32	92	34	46	0

Taken from Beginning C, Horton

Null Character

- a special character with the code value 0 is added to the end of each string to mark where it ends
 - this character is known as the null character and you write it as `\0`
- a string is always terminated by a null character, so the length of a string is always one greater than the number of characters in the string
- don't confuse the null character with NULL
 - null character is a string terminator
 - NULL is a symbol that represents a memory address that doesn't reference anything
- you can add a `\0` character to the end of a string explicitly
 - this will create two strings

Null Character Example

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    printf("The character \0 is used to terminate a string.");
```

```
    return 0;
```

```
}
```

- If you compile and run this program, you'll get this output:
 - The character
 - only the first part of the string has been displayed
 - output ends after the first two words because the function stops outputting the string when it reaches the first null character
 - the second \0 at the end of the string will never be reached
- The first \0 that's found in a character sequence always marks the end of the string

Creating a String

By Jason Fedin

Character Strings

- C has no special variable type for strings
 - this means there are no special operators in the language for processing strings
 - the standard library provides an extensive range of functions to handle strings
- strings in C are stored in an array of type char
 - characters in a string are stored in adjacent memory cells, one character per cell
- to declare a string in C, simply use the char type and the brackets to indicate the size

```
char myString[20];
```

- this variable can accommodate a string that contains up to 19 characters
 - you must allow one element for the termination character (null character)
- when you specify the dimension of an array that you intend to use to store a string, it must be at least one greater than the number of characters in the string that you want to store
 - the compiler automatically adds `\0` to the end of every string constant

Initializing a String

- You can initialize a string variable when you declare it

```
char word [] = { 'H', 'e', 'l', 'l', 'o', '!' };
```

- to initialize a string, it is the same as any other array initialization
 - in the absence of a particular array size, the C compiler automatically computes the number of elements in the array
 - based upon the number of initializers
 - this statement reserves space in memory for exactly seven characters
 - automatically adds the null terminator

word[0]	'H'
word[1]	'e'
word[2]	'l'
word[3]	'l'
word[4]	'o'
word[5]	'!'
word[6]	'\0'

Taken from Programming in C,

Initializing a String (cont'd)

- you can specify the size of the string explicitly, just make sure you leave enough space for the terminating null character

```
char word[7] = { "Hello!" };
```

- If the size specified is too small, then the compiler can't fit a terminating null character at the end of the array, and it doesn't put one there (and it doesn't complain about it either)

```
char word[6] = { "Hello!" };
```

- So....., do not specify the size, let the compiler figure out, you can be sure it will be correct
- you can initialize just part of an array of elements of type char with a string

```
char str[40] = "To be";
```

- the compiler will initialize the first five elements, str[0] to str[4], with the characters of the string constant
 - str[5] will contain the null character, '\0'
 - space is allocated for all 40 elements of the array

Assigning a value to a string after initializing

- since you can not assign arrays in C, you can not assign strings either
- the following is an error:

```
char s[100]; // declare  
s = "hello"; // initialize - DOESN'T WORK ('lvalue required' error)
```

- you are performing an assignment operation, and you cannot assign one array of characters to another array of characters like this
 - you have to use `strncpy()` to assign a value to a char array after it has been declared or initialized

- The below is perfectly valid

```
s[0] = 'h';  
s[1] = 'e';  
s[2] = 'l';  
s[3] = 'l';  
s[4] = 'o';  
s[5] = '\0';
```


Displaying a string

- when you want to refer to a string stored in an array, you just use the array name by itself
- to display a string as output using the printf function, you do the following

```
printf("\nThe message is: %s", message);
```

- the %s format specifier is for outputting a null-terminated string
- the printf() function assumes when it encounters the %s format characters that the corresponding argument is a character string that is terminated by a null character

Inputting a string

- to input a string via the keyboard, use the scanf function

```
char input[10];  
printf("Please input your name: ");  
scanf("%s", input);
```

- the %s format specifier is for inputting string
- no need to use the & (address of operator) on a string

Testing if two strings are equal

- you cannot directly test two strings to see if they are equal with a statement such as `if ( string1 == string2 )`
- the equality operator can only be applied to simple variable types, such as floats, ints, or chars
 - does not work on structures or arrays
- to determine if two strings are equal, you must explicitly compare the two character strings character by character
 - we will discuss an easier way with the `strcmp` function
- Reminder::**
 - the string constant `"x"` is not the same as the character constant `'x'`
 - `'x'` is a basic type (`char`)
 - `"x"` is a derived type, an array of `char`
 - `"x"` really consists of two characters, `'x'` and `'\0'`, the null character

Example

```
#include <stdio.h>

int main(void)
{
    char str1[] = "To be or not to be";
    char str2[] = ",that is the question";
    unsigned int count = 0;          // Stores the string length

    while (str1[count] != '\0')      // Increment count till we reach the
        ++count;                    // terminating character.

    printf("The length of the string \"%s\" is %d characters.\n", str1, count);

    count = 0;                      // Reset count for next string
    while (str2[count] != '\0')      // Count characters in second string
        ++count;

    printf("The length of the string \"%s\" is %d characters.\n", str2, count);
    return 0;
}
```

#define and const

By Jason Fedin

Constant Strings

- sometimes you need to use a constant in a program

```
circumference = 3.14159 * diameter;
```

- the constant 3.14159 represents the world-famous constant pi (π)
- there are good reasons to use a symbolic constant instead of just typing in the number
 - a name tells you more than a number does

```
owed = 0.015 * housevalue;
```

```
owed = taxrate * housevalue;
```

- If you read through a long program, the meaning of the second version is plainer.
- suppose you have used a constant in several places, and it becomes necessary to change its value
 - you only need to alter the definition of the symbolic constant, rather than find and change every occurrence of the constant in the program

#define

- the preprocessor lets you define constants

```
#define TAXRATE 0.015
```

- when your program is compiled, the value 0.015 will be substituted everywhere you have used TAXRATE
 - compile-time substitution
- a defined name is not a variable
 - you cannot assign a value to it
- notice that the #define statement has a special syntax
 - no equal sign used to assign the value 0.015 to TAXRATE
 - no semicolon

#define (cont'd)

- #define statements can appear anywhere in a program
 - no such thing as a local define
 - most programmers group their #define statements at the beginning of the program (or inside an include file) where they can be quickly referenced and shared by more than one source file
- the #define statement helps to make programs more portable
 - it might be necessary to use constant values that are related to the particular computer on which the program is running
- the #define statement can be used for character and string constants

```
#define BEEP '\a'
```

```
#define TEE 'T'
```

```
#define ESC '\033'
```

```
#define OOPS "Now you have done it!"
```


const

- C90 added a second way to create symbolic constants
 - using the const keyword to convert a declaration for a variable into a declaration for a constant

```
const int MONTHS = 12; // MONTHS a symbolic constant for 12
```

- const makes MONTHS into a read-only value
 - you can display MONTHS and use it in calculations
 - you cannot alter the value of MONTHS
- const is a newer approach and is more flexible than using #define
 - it lets you declare a type
 - it allows better control over which parts of a program can use the constant
- C has yet a third way to create symbolic constants
 - enums

const (cont'd)

- initializing a char array and declaring it as constant is a good way of handling standard messages

```
const char message[] = "The end of the world is nigh.";
```

- because you declare message as const, it's protected from being modified explicitly within the program
 - any attempt to do so will result in an error message from the compiler
- this technique for defining standard messages is particularly useful if they are used in many places within a program
 - prevents accidental modification of such constants in other parts of the program

String Functions

By Jason Fedin

String Functions

- you already know that a character string is a char array terminated with a null character (`\0`)
 - character strings are commonly used
 - C provides many functions specifically designed to work with strings
- some of the more commonly performed operations on character strings include
 - getting the length of a string
 - `strlen`
 - copying one character string to another
 - `strcpy()` and `strncpy()`
 - combining two character strings together (concatenation)
 - `strcat()` and `strncat()`
 - determining if two character strings are equal
 - `strcmp()` and `strncmp()`
- the C library supplies these string-handling function prototypes in the `string.h` header file

Getting the length of a string

- the strlen() function finds the length of a string
 - returned as a size_t

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main(){
```

```
    char myString[] = "my string";
```

```
    printf("The length of this string is: %d", strlen(myString));
```

```
    return 0;
```

```
}
```

- this function does change the string
 - the function header does not use const in declaring the formal parameter string

Copying strings

- since you can not assign arrays in C, you can not assign strings either

```
char s[100]; // declare
```

```
s = "hello"; // initialize - DOESN'T WORK ('lvalue required' error)
```

- you can use the strcpy() function to copy a string to an existing string
 - the string equivalent of the assignment operator

```
char src[50], dest[50];
```

```
strcpy(src, "This is source");
```

```
strcpy(dest, "This is destination");
```

Copying strings (cont'd)

- the strcpy() function does not check to see whether the source string actually fits in the target string
 - safer way to copy strings is to use strncpy()
- strncpy() takes a third argument
 - the maximum number of characters to copy

```
char src[40];  
char dest[12];
```

```
memset(dest, '\0', sizeof(dest));  
strcpy(src, "Hello how are you doing");  
strncpy(dest, src, 10);
```

String concatenation

- the `strcat()` function takes two strings for arguments
 - a copy of the second string is tacked onto the end of the first
 - this combined version becomes the new first string
 - the second string is not altered
- it returns the value of its first argument
 - the address of the first character of the string to which the second string is appended

```
char myString[] = "my string";  
char input[80];
```

```
printf("Enter a string to be concatenated: ");  
scanf("%s", input);
```

```
printf("\nThe string %s concatenated with %s is::::\n", myString, input);  
printf("%s", strcat(input, myString));
```


String concatenation (cont'd)

- the `strcat()` function does not check to see whether the second string will fit in the first array
 - if you fail to allocate enough space for the first array, you will run into problems as excess characters overflow into adjacent memory locations
- use `strncat()` instead
 - takes a second argument indicating the maximum number of characters to add
- for example, `strncat(bugs, addon, 13)` will add the contents of the `addon` string to `bugs`, stopping when it reaches 13 additional characters or the null character, whichever comes first

```
char src[50], dest[50];
```

```
strcpy(src, "This is source");
```

```
strcpy(dest, "This is destination");
```

```
strncat(dest, src, 15);
```

```
printf("Final destination string : |%s|", dest);
```

comparing Strings

- suppose you want to compare someone response to a stored string
 - cannot use `==`, will only check to see if the string has the same address
- there is a function that compares string contents, not string addresses
 - it is the `strcmp()` (for string comparison) function
 - does not compare arrays, so it can be used to compare strings stored in arrays of different sizes
 - does not compare characters
 - you can use arguments such as "apples" and "A", but you cannot use character arguments, such as 'A'
- this function does for strings what relational operators do for numbers
 - it returns 0 if its two string arguments are the same and nonzero otherwise
 - if return value < 0 then it indicates str1 is less than str2
 - if return value > 0 then it indicates str2 is less than str1

comparing strings example

```
printf("strcmp(\"A\", \"A\") is ");  
printf("%d\n", strcmp("A", "A"));
```

```
printf("strcmp(\"A\", \"B\") is ");  
printf("%d\n", strcmp("A", "B"));
```

```
printf("strcmp(\"B\", \"A\") is ");  
printf("%d\n", strcmp("B", "A"));
```

```
printf("strcmp(\"C\", \"A\") is ");  
printf("%d\n", strcmp("C", "A"));
```

```
printf("strcmp(\"Z\", \"a\") is ");  
printf("%d\n", strcmp("Z", "a"));
```

```
printf("strcmp(\"apples\", \"apple\") is ");  
printf("%d\n", strcmp("apples", "apple"));
```


comparing strings (cont'd)

- the strcmp() function compares strings until it finds corresponding characters that differ
 - could take the search to the end of one of the strings
- the strncmp() function compares the strings until they differ or until it has compared a number of characters specified by a third argument
 - if you wanted to search for strings that begin with "astro", you could limit the search to the first five characters

```
if (strncmp("astronomy","astro", 5) == 0)
{
    printf("Found: astronomy");
}
```

```
if (strncmp("astounding","astro", 5) == 0)
{
    printf("Found: astounding");
}
```

Searching, Tokenizing, and Analyzing Strings

By Jason Fedin

Overview

- lets discuss some more string functions
- searching a string
 - the string.h header file declares several string-searching functions for finding a single character or a substring
 - strchr() and strstr()
- tokenizing a string
 - a token is a sequence of characters within a string that is bounded by a delimiter (space, comma, period, etc)
 - breaking a sentence into words is called *tokenizing*
 - strtok()
- analyzing strings
 - islower(), isupper(), isalpha(), isdigit(), etc.

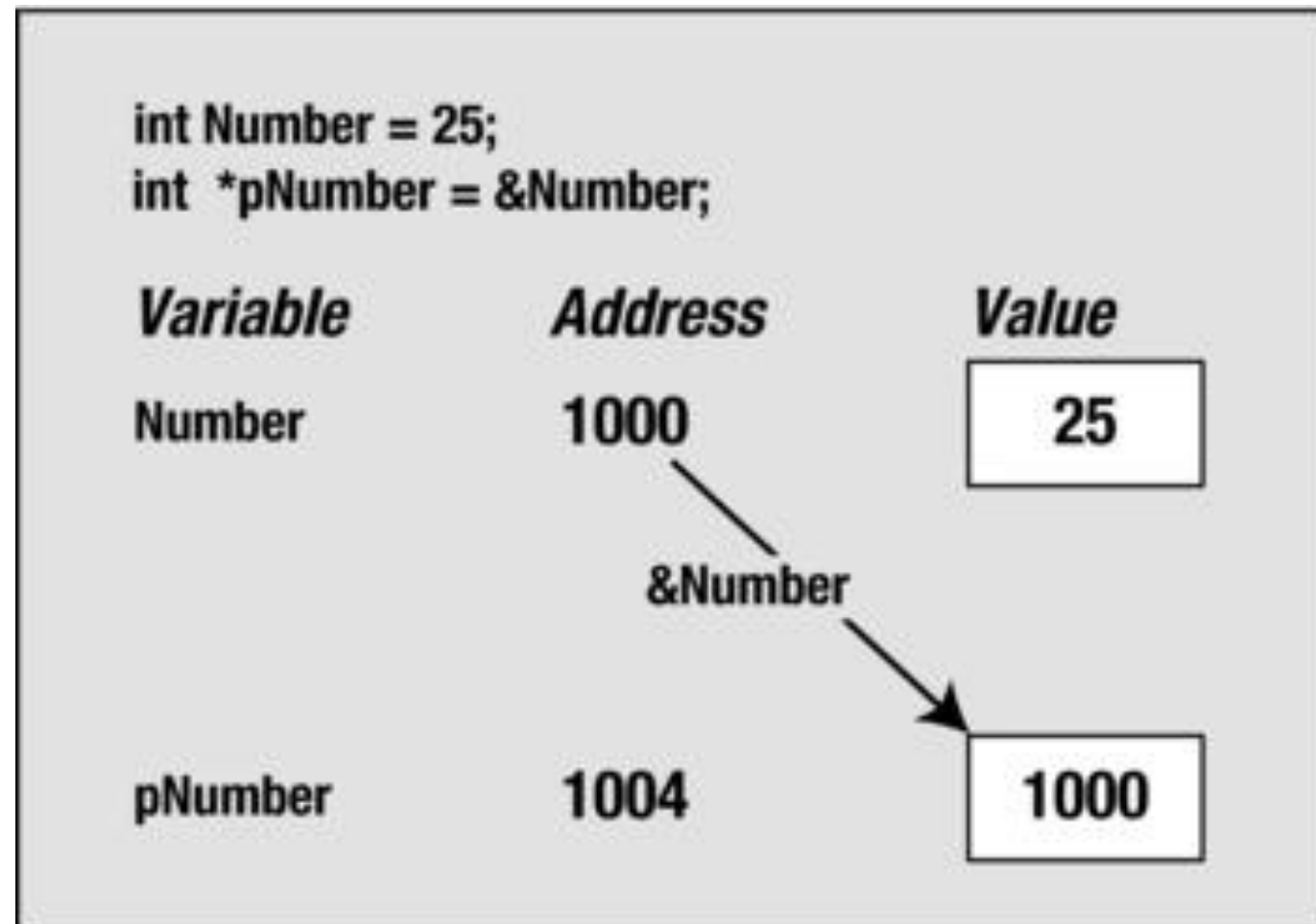
concept of a pointer

- we are going to discuss in detail, the concept of a pointer in an upcoming section
 - however, in order to understand some of these string functions, I want to give you a quick peek on this concept
- C provides a remarkably useful type of variable called a pointer
 - a variable that stores an address
 - its value is the address of another location in memory that can contain a value
 - we have used addresses in the past with the `scanf()` function

```
int Number = 25;  
int *pNumber = &Number;
```

- above, we declared a variable, `Number`, with the value 25
- we declared a pointer, `pNumber`, which contains the address of `Number`
 - asterisk used in declaring a pointer
- to get the value of the variable `pNumber`, you can use the asterisk to dereference the pointer
 - `*pNumber = 25`
 - `*` is the dereference operator, and its effect is to access the data stored at the address specified by a pointer

pointer (cont'd)



(taken from Beginning C,
Horton)

- the value of `&Number` is the address where `Number` is located
 - this value is used to initialize `pNumber` in the second statement
- many of the string functions return pointers
 - this is why I wanted to briefly mention them
 - do not worry if this concept does not sink in right now, we are going to cover points in a ton of detail in an upcoming section

Searching a string for a character

- the `strchr()` function searches a given string for a specified character
 - first argument to the function is the string to be searched (which will be the address of a char array)
 - second argument is the character that you are looking for
- the function will search the string starting at the beginning and return a pointer to the first position in the string where the character is found
 - the address of this position in memory
 - is of type `char*` described as the “pointer to char.”
- to store the value that’s returned, you must create a variable that can store the address of a character
- if the character is not found, the function returns a special value `NULL`
 - `NULL` is the equivalent of 0 for a pointer and represents a pointer that does not point to anything

strchr()

- you can use the strchr() function like this

```
char str[] = "The quick brown fox"; // The string to be searched
char ch = 'q';                      // The character we are looking for
char *pGot_char = NULL;             // Pointer initialized to NULL
pGot_char = strchr(str, ch);        // Stores address where ch is found
```

- the first argument to strchr() is the address of the first location to be searched
 - second argument is the character that is sought (ch, which is of type char)
 - expects its second argument to be of type int, so the compiler will convert the value of ch to this type
 - could just as well define ch as type int (int ch = 'q';)
 - pGot_char will point to the value (“quick brown fox”)

searching for a substring

- the `strstr()` function is probably the most useful of all the searching functions
 - searches one string for the first occurrence of a substring
 - returns a pointer to the position in the first string where the substring is found
 - if no match, returns `NULL`
- the first argument is the string that is to be searched
- the second argument is the substring you're looking for

```
char text[] = "Every dog has his day";  
char word[] = "dog";  
char *pFound = NULL;  
pFound = strstr(text, word);
```

- searches `text` for the first occurrence of the string stored in `word`
 - the string "dog" appears starting at the seventh character in `text`
 - `pFound` will be set to the address `text + 6` ("dog has his day")
 - search is case sensitive, "Dog" will not be found

Tokenizing a string

- a token is a sequence of characters within a string that is bound by a delimiter
- a delimiter can be anything, but, should be unique to the string
 - spaces, commas, and a period are good examples
- breaking a sentence into words is called tokenizing
- the `strtok()` function is used for tokenizing a string
- It requires two arguments
 - string to be tokenized
 - a string containing all the possible delimiter characters

strtok example

```
int main () {  
    char str[80] = "Hello how are you – my name is - jason";  
    const char s[2] = "-";  
    char *token;  
  
    /* get the first token */  
    token = strtok(str, s);  
  
    /* walk through other tokens */  
    while( token != NULL ) {  
        printf( " %s\n", token );  
  
        token = strtok(NULL, s);  
    }  
  
    return(0);  
}
```

Analyzing strings

Function	Tests for
<code>islower()</code>	Lowercase letter
<code>isupper()</code>	Uppercase letter
<code>isalpha()</code>	Uppercase or lowercase letter
<code>isalnum()</code>	Uppercase or lowercase letter or a digit
<code>isctrl()</code>	Control character
<code>isprint()</code>	Any printing character including space
<code>isgraph()</code>	Any printing character except space
<code>isdigit()</code>	Decimal digit ('0' to '9')
<code>isxdigit()</code>	Hexadecimal digit ('0' to '9', 'A' to 'F', 'a' to 'f')
<code>isblank()</code>	Standard blank characters (space, '\t')
<code>isspace()</code>	Whitespace character (space, '\n', '\t', '\v', '\r', '\f')
<code>ispunct()</code>	Printing character for which <code>isspace()</code> and <code>isalnum()</code> return false

- the argument to each of these functions is the character to be tested
- all these functions return a nonzero value of type `int` if the character is within the set that's being tested for
- these return values convert to `true` and `false`, respectively, so you can use them as Boolean values.

Example

```
char buf[100];           // Input buffer
int nLetters = 0;        // Number of letters in input
int nDigits = 0;         // Number of digits in input
int nPunct = 0;          // Number of punctuation characters

printf("Enter an interesting string of less than %d characters:\n", 100);
scanf("%s", buf); // Read a string into buffer

int i = 0;               // Buffer index
while(buf[i])
{
    if(isalpha(buf[i]))
        ++nLetters;      // Increment letter count
    else if(isdigit(buf[i]))
        ++nDigits;       // Increment digit count
    else if(ispunct(buf[i]))
        ++nPunct;
    ++i;
}

printf("\nYour string contained %d letters, %d digits and %d punctuation
characters.\n", nLetters, nDigits, nPunct);
```

Converting Strings

By Jason Fedin

Converting Strings

- it is very common to convert character case
 - to all upper case or all lower case
- the `toupper()` function converts from lowercase to uppercase
- the `tolower()` function converts from uppercase to lowercase
- both functions return either the converted character or the same character for characters that are already in the correct case or are not convertible such as punctuation characters
- this is how you convert a string to uppercase

```
for(int i = 0 ; (buf[i] = (char)toupper(buf[i])) != '\0' ; ++i);
```

- this loop will convert the entire string in the `buf` array to uppercase by stepping through the string one character at a time
 - loop stops when it reaches the string termination character `'\0'`
 - the cast to type `char` is there because `toupper()` returns type `int`
- you can use the function `toupper()` in combination with the `strstr()` function to find out whether one string occurs in another, ignoring case

Case conversion example

```
char text[100];           // Input buffer for string to be searched
char substring[40];       // Input buffer for string sought

printf("Enter the string to be searched (less than %d characters):\n", 100);
scanf("%s", text);

printf("\nEnter the string sought (less than %d characters):\n", 40);
scanf("%s", substring);

printf("\nFirst string entered:\n%s\n", text);
printf("Second string entered:\n%s\n", substring);

// Convert both strings to uppercase.
for(i = 0 ; (text[i] = (char)toupper(text[i])) != '\0' ; ++i);
for(i = 0 ; (substring[i] = (char)toupper(substring[i])) != '\0' ; ++i);

printf("The second string %s found in the first.\n", ((strstr(text, substring) == NULL) ? "was not" : "was"));
```


Converting Strings to numbers

the `stdlib.h` header file declares functions that you can use to convert a string to a numerical

Function	Returns
<code>atof()</code>	A value of type <code>double</code> that is produced from the string argument. Infinity as a <code>double</code> value is recognized from the strings <code>"INF"</code> or <code>"INFINITY"</code> where any character can be in uppercase or lowercase and 'not a number' is recognized from the string <code>"NAN"</code> in uppercase or lowercase.
<code>atoi()</code>	A value of type <code>int</code> that is produced from the string argument.
<code>atol()</code>	A value of type <code>long</code> that is produced from the string argument.
<code>atoll()</code>	A value of type <code>long long</code> that is produced from the string argument.

Taken from Beginning C, Horton

For all four functions, leading whitespace is ignored

```
char value_str[] = "98.4";
double value = atof(value_str);
```


Converting Strings to Numbers

Taken from Beginning C, Horton

Function	Returns
<code>strtod()</code>	A value of type <code>double</code> is produced from the initial part of the string specified by the first argument. The second argument is a pointer to a variable, <code>ptr</code> say, of type <code>char*</code> in which the function will store the address of the first character following the substring that was converted to the <code>double</code> value. If no string was found that could be converted to type <code>double</code> , the variable <code>ptr</code> will contain the address passed as the first argument.
<code>strtof()</code>	A value of type <code>float</code> . In all other respects it works as <code>strtod()</code> .
<code>strtold()</code>	A value of type <code>long double</code> . In all other respects it works as <code>strtod()</code> .

Example

```
double value = 0;
char str[] = "3.5 2.5 1.26";    // The string to be converted
char *pstr = str;               // Pointer to the string to be converted
char *ptr = NULL;               // Pointer to character position after conversion
```

```
while(true)
{
    value = strtod(pstr, &ptr);    // Convert starting at pstr
    if(pstr == ptr)               // pstr stored if no conversion...
        break;                   // ...so we are done
    else
    {
        printf(" %f", value);     // Output the resultant value
        pstr = ptr;               // Store start for next conversion
    }
}
```

Challenge

By Jason Fedin

Requirements

- In this challenge, you are going to write a program that tests your understanding of char arrays
- write a function to count the number of characters in a string (length)
 - cannot use the strlen library function
 - function should take a character array as a parameter
 - should return an int (the length)
- write a function to concatenate two character strings
 - cannot use the strcat library function
 - function should take 3 parameters
 - char result[]
 - const char str1[]
 - const char str2[]
 - can return void
- write a function that determines if two strings are equal
 - cannot use strcmp library function
 - function should take two const char arrays as parameters and return a Boolean of true if they are equal and false otherwise

Challenge

By Jason Fedin

Requirements

- this challenge will help you better understand how to use the most common string functions in the string library
- write a program that displays a string in reverse order
 - should read input from the keyboard
 - need to use the strlen string function

Requirements

- write a program that sorts the strings of an array using a bubble sort
 - need to use the strcmp and strcpy functions

Input number of strings :3

Input string 3 :

zero

one

two

Expected Output :

The strings appears after sorting :

one

two

zero

Debugging

Jason Fedin

Overview

- debugging is the process of finding and fixing errors in a program (usually logic errors, but, can also include compiler/syntax errors)
- for syntax errors, understand what the compiler is telling you
- always focus on fixing the first problem detected

- can range in complexity from fixing simple errors to collecting large amounts of data for analysis

- the ability to debug by a programmer is an essential skill (problem solving) that can save you tremendous amounts of time (and money)

- maintenance phase is the most expensive phase of the software life cycle

- understand that bugs are unavoidable

Common Problems

- Logic Errors
- Syntax Errors
- Memory Corruption
- Performance / Scalability
- Lack of Cohesion
- Tight Coupling (dependencies)

Debugging Process

- Understand the problem (sit down with tester, understand requirements)
- Reproduce the problem
- Sometimes very difficult as problems can be intermittent or only happen in very rare circumstances
- Parallel processes or threading problems
- Simplify the problem / Divide and conquer / isolate the source
- Remove parts of the original test case
- Comment out code / back out changes
- Turn a large program into a lot of small programs (unit testing)

Debugging Process (cont'd)

- Identify origin of the problem (in the code)
- Use Debugging Tools if necessary
- Solve the problem
- Experience and practice
- Sometimes includes redesign or refactor of code
- Test!, Test!, Test!

Techniques and

Tools

- Tracing / using print statements
 - Output values of variables at certain points of a program
 - Show the flow of execution
 - Can help isolate the error
- Debuggers – monitor the execution of a program, stop it, restart it, set breakpoints and watch variables in memory
- Log Files – can be used for analysis, add “good” log statements to your code
- Monitoring Software – run-time analysis of memory usage, network traffic, thread and object information

Common Debugging Tools

- Exception Handling helps a great deal to identify catastrophic errors
- Static Analyzers – analyze source code for specific set of known problems
- Semantic checker, does not analyze syntax
- Can detect things like uninitialized variables, memory leaks, unreachable code, deadlocks or race conditions
- Test Suites – run a set of comprehensive system end-to-end tests
- Debugging the program after it has crashed
- Analyze the call stack
- Analyze memory dump (core file)

Preventing Errors

- write high quality code (follow good design principles and good programming practices)
- Unit Tests – automatically executed when compiling
- Helps avoid regression
- Finds errors in new code before it is delivered
- TDD (Test Driven Development)
- Provide good documentation and proper planning (write down design on paper and utilize pseudocode)
- Work in Steps and constantly test after each step
- Avoid too many changes at once
- When making changes, apply them incrementally. Add one change, then test thoroughly before starting the next step
- Helps reduce the possible sources of bugs, limits problem set

Debugging

Jason Fedin

Overview

- debugging is the process of finding and fixing errors in a program (usually logic errors, but, can also include compiler/syntax errors)
 - for syntax errors, understand what the compiler is telling you
 - always focus on fixing the first problem detected
- can range in complexity from fixing simple errors to collecting large amounts of data for analysis
- the ability to debug by a programmer is an essential skill (problem solving) that can save you tremendous amounts of time (and money 😊)
- maintenance phase is the most expensive phase of the software life cycle
- understand that bugs are unavoidable

Common Problems

- Logic Errors
- Syntax Errors
- Memory Corruption
- Performance / Scalability
- Lack of Cohesion
- Tight Coupling (dependencies)

Debugging Process

- Understand the problem (sit down with tester, understand requirements)
- Reproduce the problem
 - Sometimes very difficult as problems can be intermittent or only happen in very rare circumstances
 - Parallel processes or threading problems
- Simplify the problem / Divide and conquer / isolate the source
 - Remove parts of the original test case
 - Comment out code / back out changes
 - Turn a large program into a lot of small programs (unit testing)

Debugging process (cont'd)

- Identify origin of the problem (in the code)
 - Use Debugging Tools if necessary
- Solve the problem
 - Experience and practice
 - Sometimes includes redesign or refactor of code
- Test!, Test!, Test!

Techniques and Tools

- Tracing / using print statements
 - Output values of variables at certain points of a program
 - Show the flow of execution
 - Can help isolate the error
- Debuggers – monitor the execution of a program, stop it, restart it, set breakpoints and watch variables in memory
- Log Files – can be used for analysis, add “good” log statements to your code
- Monitoring Software – run-time analysis of memory usage, network traffic, thread and object information

Common Debugging Tools

- Exception Handling helps a great deal to identify catastrophic errors
- Static Analyzers – analyze source code for specific set of known problems
 - Semantic checker, does not analyze syntax
 - Can detect things like uninitialized variables, memory leaks, unreachable code, deadlocks or race conditions
- Test Suites – run a set of comprehensive system end-to-end tests
- Debugging the program after it has crashed
 - Analyze the call stack
 - Analyze memory dump (core file)

Preventing Errors

- write high quality code (follow good design principles and good programming practices)
- Unit Tests – automatically executed when compiling
 - Helps avoid regression
 - Finds errors in new code before it is delivered
 - TDD (Test Driven Development)
- Provide good documentation and proper planning (write down design on paper and utilize pseudocode)
- Work in Steps and constantly test after each step
 - Avoid too many changes at once
 - When making changes, apply them incrementally. Add one change, then test thoroughly before starting the next step
 - Helps reduce the possible sources of bugs, limits problem set

Understanding the Call Stack

Jason Fedin

Overview

- A stack trace (call stack) is generated whenever your app crashes because of a fatal error
- A stack trace shows a list of the function calls that lead to the error
 - Includes the filenames and line numbers of the code that cause the exception or error to occur
 - Top of the stack contains the last call that caused the error (nested calls)
 - Bottom of the stack contains the first call that started the chain of calls to cause the error
 - You need to find the call in your application that is causing the crash
- A programmer can also dump the stack trace

Example

Call stack		
Nr	Address	Function
0	(	GcmdGtkFoldview::Model::iter_refresh(this=0xb442b8, _iter=0x7fff24e606f0)
1	0x4b332f	GcmdGtkFoldview::control_refresh(this=0xb44000, ctxdata=0xb2ad50)
2	0x4b3352	GcmdGtkFoldview::Control_refresh(menu_item=0xaed540, ctxdata=0xb2ad50)
3	0x7f521b69fe9d	IA__g_closure_invoke(closure=0xb6d800, return_value=0x0, n_param_values=1, param_values=0x7fff24e60990, invocation_hint=0x7fff24e60990)
4	0x7f521b6b2bfd	signal_emit_unlocked_R(node=0x988f00, detail=0, instance=0xaed540, emission_return=0x0, instance_and_params=0x7fff24e60990)
5	0x7f521b6b40ee	IA__g_signal_emit_valist(instance=0xaed540, signal_id=619054832, detail=0, var_args=0x7fff24e60bf0)
6	0x7f521b6b45f3	IA__g_signal_emit(instance=0xb442b8, signal_id=619054832, detail=11900256)
7	0x7f521c8e2ceb	gtk_widget_activate()
8	0x7f521c7d63ad	gtk_menu_shell_activate_item()
9	0x7f521c7d8085	??()
10	0x7f521c7c9848	??()
11	0x7f521b69fe9d	IA__g_closure_invoke(closure=0x971dc0, return_value=0x7fff24e60f20, n_param_values=2, param_values=0x7fff24e60fe0, invocation_hint=0x7fff24e60990)
12	0x7f521b6b28dc	signal_emit_unlocked_R(node=0x768180, detail=0, instance=0x9a12b0, emission_return=0x7fff24e611e0, instance_and_params=0x7fff24e60990)
13	0x7f521b6b3f71	IA__g_signal_emit_valist(instance=0x9a12b0, signal_id=<value optimized out>, detail=0, var_args=0x7fff24e61240)
14	0x7f521b6b45f3	IA__g_signal_emit(instance=0xb442b8, signal_id=619054832, detail=11900256)
15	0x7f521c8de4de	??()
16	0x7f521c7c23d3	gtk_propagate_event()
17	0x7f521c7c341b	gtk_main_do_event()
18	0x7f521c424fac	??()
19	0x7f521b0027ab	IA__g_main_context_dispatch(context=0x78e6c0)
20	0x7f521b005f7d	g_main_context_iterate(context=0x78e6c0, block=1, dispatch=1, self=<value optimized out>)
21	0x7f521b0064ad	IA__g_main_loop_run(loop=0xb00460)
22	0x7f521c7c3837	gtk_main()

Common C Mistakes

Jason Fedin

Common C Mistakes

- **misplacing a semicolon**

```
if ( j == 100 );  
    j = 0;
```

- the value of j will always be set to 0 due to the misplaced semicolon after the closing parenthesis
- semicolon is syntactically valid (it represents the null statement), and, therefore, no error is produced by the compiler
- same type of mistake is frequently made in while and for loops

- **confusing the operator = with the operator ==**

- usually made inside an if, while, or do statement
- perfectly valid and has the effect of assigning 2 to a and then executing the printf() call
- printf() function will always be called because the value of the expression contained in the if statement will always be nonzero

```
if ( a = 2 )  
    printf ("Your turn.\n");
```

Common C Mistakes

- omitting prototype declarations

```
result = squareRoot (2);
```

- If squareRoot is defined later in the program, or in another file, and is not explicitly declared otherwise
- compiler assumes that the function returns an int
- always safest to include a prototype declaration for all functions that you call (either explicitly yourself or implicitly by including the correct header file in your program)

- failing to include the header file that includes the definition for a C-programming library function being used in the program

```
double answer = sqrt(value1);
```

- if this program does not #include the <math.h> file, this will generate an error that sqrt() is undefined

Common C Mistakes

- confusing a character constant and a character string

```
text = 'a';
```

- a single character is assigned to text

```
text = "a";
```

- a pointer to the character string "a" is assigned to text
- in the first case, text is normally declared to be a char variable
- in the second case, it should be declared to be of type "pointer to char"

Common C Mistakes

- using the wrong bounds for an array

```
int a[100], i, sum = 0;

...
for ( i = 1; i <= 100; ++i )
    sum += a[i];
```

- valid subscripts of an array range from 0 through the number of elements minus one
- the preceding loop is incorrect because the last valid subscript of a is 99 and not 100
- also probably intended to start with the first element of the array; therefore, i should have been initially set to 0
- forgetting to reserve an extra location in an array for the terminating null character of a string
- when declaring character arrays they need to be large enough to contain the terminating null character
- the character string "hello" would require six locations in a character array if you wanted to store a null at the end

Common C Mistakes

- confusing the operator `->` with the operator `.` when referencing structure members.
- the operator `.` is used for structure variables
- the operator `->` is used for structure pointer variables
- omitting the ampersand before nonpointer variables in a `scanf()` call

```
int number;  
...  
scanf ("%i", number);
```

- all arguments appearing after the format string in a `scanf()` call must be pointers

Common C Mistakes

- **using a pointer variable before it's initialized**

```
char *char_pointer;  
*char_pointer = 'X';
```

- you can only apply the indirection operator to a pointer variable after you have set the variable pointing somewhere
- char_pointer is never set pointing to anything, so the assignment is not meaningful
- **omitting the break statement at the end of a case in a switch statement**
- if a break is not included at the end of a case, then execution continues into the next case

Common C Mistakes

- **inserting a semicolon at the end of a preprocessor definition**
- usually happens because it becomes a matter of habit to end all statements with semicolons

```
#define END_OF_DATA 999;
```

- leads to a syntax error if used in an expression such as

```
if ( value == END_OF_DATA )  
    ...
```

- the compiler will see this statement after preprocessing

```
if ( value == 999; )  
    ...
```

Common C Mistakes

- **omitting a closing parenthesis or closing quotation marks on any statement**

```
total_earning = (cash + (investments * inv_interest) + (savings * sav_interest);  
printf("Your total money to date is %.2f, total_earning);
```

- the use of embedded parentheses to set apart each portion of the equation makes for a more readable line of code
- however, there is always the possibility of missing a closing parenthesis (or in some occasions, adding one too many)
- the second line is missing a closing quotation mark for the string being sent to the printf() function
- both of these will generate a compiler error
- sometimes the error will be identified as coming on a different line
- depending on whether the compiler uses a parenthesis or quotation mark on a subsequent line to complete the expression which moves the missing character to a place later in the program

Understanding Compiler Errors and Warnings

Jason Fedin

Overview

- it is sometimes very hard to understand what the compiler is complaining about
- need to understand compiler errors in order to fix them
- it is sometimes difficult to identify the true reason behind a compiler error

- the compiler makes decisions about how to translate the code that the programmer has not written in the code
- is convenient because the programs can be written more succinctly (only expert programmers take advantage of this feature)

- you should use an option for the compiler to notify all cases where there are implicit decisions
- this option is -Wall

Overview

- the compiler shows two types of problems
- errors
 - a condition that prevents the creation of a final program
 - no executable is obtained until all the errors have been corrected
 - The first errors shown are the most reliable because the translation is finished but there are some errors that may derive from previous ones
 - Fix the first errors are first, it is recommended to compile again and see if other later errors also disappeared.
- warnings
 - messages that the compiler shows about “special” situations in which an anomaly has been detected
 - non-fatal errors
 - the final executable program may be obtained with any number of warning
- compile always with the -Wall option and do not consider the program correct until all warnings have been eliminated

most common compiler messages

- **'variable' undeclared (first use in this function)**

- this is one of the most common and easier to detect
- the symbol shown at the beginning of the message is used but has not been declared

- **warning: implicit declaration of function '...'**

- this warning appears when the compiler finds a function used in the code but no previous information has been given about it
- need to declare a function prototype

- **warning: control reaches end of non-void function**

- this warning appears when a function has been defined as returning a result but no return statement has been included to return this result
- either the function is incorrectly defined or the statement is missing

most common compiler messages

- **warning: unused variable `...'**

- this warning is printed by the compiler when a variable is declared but not used in the code
- message disappears if the declaration is removed

- **undefined reference to `...'**

- appears when there is a function invoked in the code that has not been defined anywhere
- compiler is telling us that there is a reference to a function with no definition
- check which function is missing and make sure its definition is compiled

- **error: conflicting types for `...'**

- two definitions of a function prototype have been found
- one is the prototype (the result type, name, parenthesis including the parameters, and a semicolon)
- the other is the definition with the function body
- the information in both places is not identical, and a conflict has been detected
- the compiler shows you in which line the conflict appears and the previous definition that caused the contradiction

runtime errors

- the execution of C programs may terminate abruptly (crash) when a run-time error is detected
- C programs only print the succinct message Segmentation fault
- usually results in a core file depending on the signal that has been thrown
- can analyze the core file and the call stack

Pointers

Jason Fedin

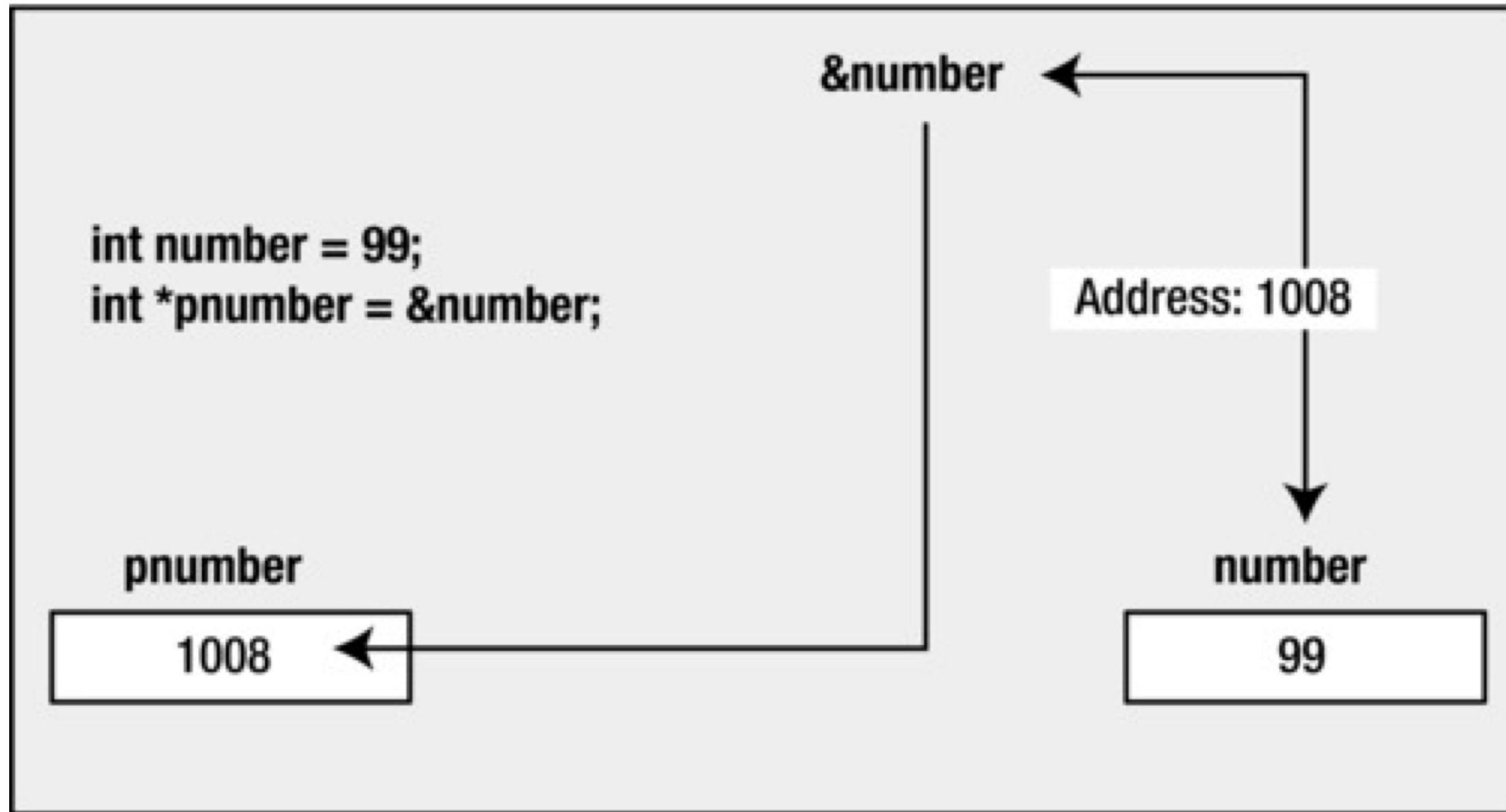
Indirection

- pointers are very similar to the concept of indirection that you employ in your everyday life
 - suppose you need to buy a new ink cartridge for your printer
 - all purchases are handled by the purchasing department
 - you call Joe in purchasing and ask him to order the new cartridge for you
 - Joe then calls the local supply store to order the cartridge
 - you are not ordering the cartridge directly from the supply store yourself (indirection)
-
- in programming languages, indirection is the ability to reference something using a name, reference, or container, instead of the value itself
-
- the most common form of indirection is the act of manipulating a value through its memory address
-
- a pointer provides an indirect means of accessing the value of a particular data item
 - a variable whose value is a memory address
 - its value is the address of another location in memory that can contain a value

Overview

- just as there are reasons why it makes sense to go through the purchasing department to order new cartridges (you don't have to know which particular store the cartridges are being ordered from)
- there are good reasons why it makes sense to use pointers in C
- using pointers in your program is one of the most powerful tools available in the C language
- pointers are also one of the most confusing concepts of the C language
- it is important you get this concept figured out in the beginning and maintain a clear idea of what is happening as you dig deeper
- the compiler must know the type of data stored in the variable to which it points
- need to know how much memory is occupied or how to handle the contents of the memory to which it points
- every pointer will be associated with a specific variable type
- it can be used only to point to variables of that type
- pointers of type "pointer to int" can point only to variables of type int
- pointers of type "pointer to float" can point only to variables of type float

Overview (cont'd)



(taken from Beginning C, Horton)

- the value of `&number` is the address where `number` is located
- this value is used to initialize `pnumber` in the second statement

Why use pointers?

- accessing data by means of only variables is very limiting
- with pointers, you can access any location (you can treat any position of memory as a variable for example) and perform arithmetic with pointers
- pointers in C make it easier to use arrays and strings
- pointers allow you to refer to the same space in memory from multiple locations
- means that you can update memory in one location and the change can be seen from another location in your program
- can also save space by being able to share components in your data structures
- pointers allow functions to modify data passed to them as variables
- pass by reference - passing arguments to function in way they can be changed by function
- can also be used to optimize a program to run faster or use less memory than it would otherwise

Why use pointers?

- pointers allow us to get multiple values from the function
- a function can return only one value but by passing arguments as pointers we can get more than one values from the pointer
- with pointers dynamic memory can be created according to the program use
- we can save memory from static (compile time) declarations
- pointers allow us to design and develop complex data structures like a stack, queue, or linked list
- pointers provide direct memory access.

Defining Pointers

Jason Fedin

Declaring pointers

- pointers are not declared like normal variables

```
pointer ptr;    // not the way to declare a pointer/
```

- it is not enough to say that a variable is a pointer
- you also have to specify the kind of variable to which the pointer points
- different variable types take up different amounts of storage
- some pointer operations require knowledge of that storage size

- you declare a pointer to a variable of type int with:

```
int *pnumber;
```

- the type of the variable with the name pnumber is int*
- can store the address of any variable of type int

```
int * pi;        // pi is a pointer to an integer variable
char * pc;       // pc is a pointer to a character variable
float * pf, * pg; // pf, pg are pointers to float variables
```

Declaring pointers (cont'd)

- the space between the * and the pointer name is optional
- programmers use the space in a declaration and omit it when dereferencing a variable
- the value of a pointer is an address, and it is represented internally as an unsigned integer on most systems
- however, you shouldn't think of a pointer as an integer type
- things you can do with integers that you can not do with pointers, and vice versa
- you can multiply one integer by another, but you can not multiply one pointer by another
- a pointer really is a new type, not an integer type
- %p represents the format specifier for pointers
- the previous declarations creates the variable but does not initialize it
- dangerous when not initialized
- you should always initialize a pointer when you declare it

NULL Pointers

- you can initialize a pointer so that it does not point to anything:

```
int *pnumber = NULL;
```

- NULL is a constant that is defined in the standard library
- is the equivalent of zero for a pointer
- NULL is a value that is guaranteed not to point to any location in memory
- means that it implicitly prevents the accidental overwriting of memory by using a pointer that does not point to anything specific
- add an #include directive for stddef.h to your source file

Address of operator

- if you want to initialize your variable with the address of a variable you have already declared
- use the address of operator, &

```
int number = 99;  
int *pnumber = &number;
```

- the initial value of pnumber is the address of the variable number
- the declaration of number must precede the declaration of the pointer that stores its address
- compiler must have already allocated space and thus an address for number to use it to initialize pnumber

Be careful

- there is nothing special about the declaration of a pointer
- can declare regular variables and pointers in the same statement

double value, *pVal, fnum;

- only the second variable, pVal, is a pointer

int *p, q;

- the above declares a pointer, p of type int*, and a variable, q, that is of type int
- a common mistake to think that both p and q are pointers
- also, it is a good idea to use names beginning with p as pointer names

Accessing Pointers

Jason Fedin

Accessing pointer values

- you use the indirection operator, *, to access the value of the variable pointed to by a pointer
- also referred to as the dereference operator because you use it to “dereference” a pointer

```
int number = 15;  
int *pointer = &number;  
int result = 0;
```

- the pointer variable contains the address of the variable number
- you can use this in an expression to calculate a new value for result

```
result = *pointer + 5;
```

- the expression *pointer will evaluate to the value stored at the address contained in the pointer
- the value stored in number, 15, so result will be set to 15 + 5, which is 20
- the indirection operator, *, is also the symbol for multiplication, and it is used to specify pointer types
- depending on where the asterisk appears, the compiler will understand whether it should interpret it as an indirection operator, as a multiplication sign, or as part of a type specification
- context determines what it means in any instance

Example

```
int main (void)
{
    int  count = 10, x;
    int  *int_pointer;

    int_pointer = &count;
    x = *int_pointer;

    printf ("count = %i, x = %i\n", count, x);

    return 0;
}
```


Displaying a pointers value

- to output the address of a variable, you use the output format specifier %p
- outputs a pointer value as a memory address in hexadecimal form

```
int number = 0;           // A variable of type int initialized to 0
int *pnumber = NULL;      // A pointer that can point to type int

number = 10;
pnumber = &number;
printf("pnumber's value: %p\n", pnumber);           // Output the value (an address)
```

- pointers occupy 8 bytes and the addresses have 16 hexadecimal digits
- if a machine has a 64-bit operating system and my compiler supports 64-bit addresses
- some compilers only support 32-bit addressing, in which case addresses will be 32-bit addresses

Displaying an address (cont'd)

```
printf("number's address: %p\n", &number);           // Output the address  
printf("pnumber's address: %p\n", (void*)&pnumber);    // Output the address
```

- remember, a pointer itself has an address, just like any other variable
- you use %p as the conversion specifier to display an address
- you use the & (address of) operator to reference the address that the pnumber variable occupies
- the cast to void* is to prevent a possible warning from the compiler
- the %p specification expects the value to be some kind of pointer type, but the type of &pnumber is “pointer to pointer to int”

Displaying the number of bytes a pointer is using

- you use the sizeof operator to obtain the number of bytes a pointer occupies
- on my machine this shows that a pointer occupies 8 bytes
- a memory address on my machine is 64 bits

- you may get a compiler warning when using sizeof this way
- size_t is an implementation-defined integer type
- to prevent the warning, you could cast the argument to type int like this:

```
printf("pnumber's size: %d bytes\n", (int)sizeof(pnumber)); // Output the size
```

Example

```
int main(void)
{
    int number = 0;           // A variable of type int initialized to 0
    int *pnumber = NULL;      // A pointer that can point to type int

    number = 10;

    printf("number's address: %p\n", &number);           // Output the address
    printf("number's value: %d\n\n", number);            // Output the value

    pnumber = &number;      // Store the address of number in pnumber

    printf("pnumber's address: %p\n", (void*)&pnumber);   // Output the address
    printf("pnumber's size: %zd bytes\n", sizeof(pnumber)); // Output the size
    printf("pnumber's value: %p\n", pnumber);            // Output the value (an address)
    printf("value pointed to: %d\n", *pnumber);          // Value at the address
    return 0;
}
```

Using Pointers

Jason Fedin

Overview

- C offers several basic operations you can perform on pointers
- you can assign an address to a pointer
- assigned value can be an array name, a variable preceded by address operator (&), or another second pointer.
- you can dereference a pointer
- the * operator gives the value stored in the pointed-to location
- you can take a pointer address
- the & operator tells you where the pointer itself is stored
- you can perform pointer arithmetic
- use the + operator to add an integer to a pointer or a pointer to an integer (integer is multiplied by the number of bytes in the pointed-to type and added to the original address)
- Increment a pointer by one (useful in arrays when moving to the next element)
- use the - operator to subtract an integer from a pointer (integer is multiplied by the number of bytes in the pointed-to type and subtracted from the original address)
- decrementing a pointer by one (useful in arrays when going back to the previous element)

Overview

- you can find the difference between two pointers
- you do this for two pointers to elements that are in the same array to find out how far apart the elements are
- you can use the relational operators to compare the values of two pointers
- pointers must be the same type
- remember, there are two forms of subtraction
- you can subtract one pointer from another to get an integer
- you can subtract an integer from a pointer and get a pointer
- be careful when incrementing or decrementing pointers and causing an array “out of bounds” error
- computer does not keep track of whether a pointer still points to an array element

pointers used in expressions

- the value referenced by a pointer can be used in an arithmetic expressions
- if a variable is defined to be of type “pointer to integer” then it is evaluated using the rules of integer arithmetic

```
int number = 0;           // A variable of type int initialized to 0
int *pnumber = NULL;      // A pointer that can point to type int
number = 10;
pnumber = &number;        // Store the address of number in pnumber
*pnumber += 25;
```

- increments the value of the number variable by 25
- * indicates you are accessing the contents to which the variable called pnumber is pointing to
- if a pointer points to a variable x
- that pointer has been defined to be a pointer to the same data type as is x
- use of *pointer in an expression is identical to the use of x in the same expression

pointers in expressions (cont'd)

- a variable defined as a “pointer to int” can store the address of any variable of type int

```
int value = 999;  
pnumber = &value;  
*pnumber += 25;
```

- the statement will operate with the new variable, value
- the new contents of value will be 1024
- a pointer can contain the address of any variable of the appropriate type
- you can use one pointer variable to change the values of many different variables
- as long as they are of a type compatible with the pointer type

Example

```
int main(void)
{
    long num1 = 0L;
    long num2 = 0L;
    long *pnum = NULL;

    pnum = &num1;           // Get address of num1
    *pnum = 2L;              // Set num1 to 2
    ++num2;                  // Increment num2
    num2 += *pnum;           // Add num1 to num2

    pnum = &num2;           // Get address of num2
    ++*pnum;                 // Increment num2 indirectly

    printf("num1 = %ld  num2 = %ld  *pnum = %ld  *pnum + num2 = %ld\n",
           num1, num2, *pnum, *pnum + num2);

    return 0;
}
```

when receiving Input

- when we have used scanf() to input values, we have used the & operator to obtain the address of a variable
- on the variable that is to store the input (second argument)
- when you have a pointer that already contains an address, you can use the pointer name as an argument for scanf()

```
int value = 0;
int *pvalue = &value;           // Set pointer to refer to value

printf ("Input an integer: ");
scanf(" %d", pvalue);           // Read into value via the pointer

printf("You entered %d.\n", value); // Output the value entered
```

Testing for NULL

- there is one rule you should burn into your memory
- do not dereference an uninitialized pointer

```
int * pt; // an uninitialized pointer
*pt = 5;  // a terrible error
```

- second line means store the value 5 in the location to which pt points
- pt has a random value, there is no knowing where the 5 will be placed
- It might go somewhere harmless, it might overwrite data or code, or it might cause the program to crash
- creating a pointer only allocates memory to store the pointer itself
- it does not allocate memory to store data
- before you use a pointer, it should be assigned a memory location that has already been allocated
- assign the address of an existing variable to the pointer
- or you can use the malloc() function to allocate memory first

Testing for NULL (cont'd)

- we already know that when declaring a pointer that does not point to anything, we should initialize it to NULL

```
int *pvalue = NULL;
```

- NULL is a special symbol in C that represents the pointer equivalent to 0 with ordinary numbers
- the below also sets a pointer to null using 0

```
int *pvalue = 0;
```

- because NULL is the equivalent of zero, if you want to test whether pvalue is NULL, you can do this:
- or you can do it explicitly by using == NULL

```
if(!pvalue) .....
```

- you want to check for NULL before you dereference a pointer
- often when pointers are passed to functions

Keep practicing

- stay with me
 - pointers can be confusing :)
 - you can work with addresses
 - you can work with values
 - you can work with pointers
 - you can work with variables
-
- sometimes it is hard to work out what exactly is going on
-
- the best thing to understand this concept of a pointer is to keep writing short programs that use pointers
 - getting values using pointers
 - changing values using pointers
 - printing addresses, etc.
-
- this is the only way to really get confident about using pointers, practice!!!!

Pointers and Const

Jason Fedin

Overview

- when we use the const modifier on a variable or an array it tells the compiler that the contents of the variable/array will not be changed by the program
- with pointers, we have to consider two things when using the const modifier
- whether the pointer will be changed
- whether the value that the pointer points to will be changed
- you can use the const keyword when you declare a pointer to indicate that the value pointed to must not be changed

```
long value = 9999L;  
const long *pvalue = &value;           // defines a pointer to a constant
```

- you have declared the value pointed to by pvalue to be const
- the compiler will check for any statements that attempt to modify the value pointed to by pvalue and flag such statements as an error
- the following statement will now result in an error message from the compiler
- ```
*pvalue = 8888L; // Error - attempt to change const location
```



# pointers to constants

---

- you can still modify value (you have only applied const to the pointer)

```
value = 7777L;
```

- the value pointed to has changed, but you did not use the pointer to make the change
- the pointer itself is not constant, so you can still change what it points to:

```
long number = 8888L;
pvalue = &number; // OK - changing the address in pvalue
```

- will change the address stored in pvalue to point to number
- still cannot use the pointer to change the value that is stored
- you can change the address stored in the pointer as much as you like
- using the pointer to change the value pointed to is not allowed, even after you have changed the address stored in the pointer.

# constant pointers

---

- you might also want to ensure that the address stored in a pointer cannot be changed
- you can do this by using the const keyword in the declaration of the pointer

```
int count = 43;
int *const pcount = &count; // Defines a constant pointer
```

- the above ensures that a pointer always points to the same thing
- indicates that the address stored must not be changed
- compiler will check that you do not inadvertently attempt to change what the pointer points to elsewhere in your code

```
int item = 34;
pcount = &item; // Error - attempt to change a constant pointer
```

- it is all about where you place the const keyword, either before the type or after the type
- `const int * .....` // value can not be changed
- `int *const .....` // pointer address cannot change

# constant pointers (cont'd)

---

- you can still change the value that pcount points to using pcount

```
*pcount = 345; // OK - changes the value of count
```

- references the value stored in count through the pointer and changes its value to 345
- you can create a constant pointer that points to a value that is also constant:

```
int item = 25;
const int *const pitem = &item;
```

- the pitem is a constant pointer to a constant so everything is fixed
- cannot change the address stored in pitem
- cannot use pitem to modify what it points to
- you can still change the value of item directly
- if you wanted to make everything not change, you could specify item as const as well

# void pointers

## Jason Fedin

# Overview

---

- the type name void means absence of any type
- a pointer of type void\* can contain the address of a data item of any type
- void\* is often used as a parameter type or return value type with functions that deal with data in a type-independent way
- any kind of pointer can be passed around as a value of type void\*
  - the void pointer does not know what type of object it is pointing to, so, it cannot be dereferenced directly
  - the void pointer must first be explicitly cast to another pointer type before it is dereferenced
- the address of a variable of type int can be stored in a pointer variable of type void\*
- when you want to access the integer value at the address stored in the void\* pointer, you must first cast the pointer to type int\*



# Example

---

```
int i = 10;
float f = 2.34;
char ch = 'k';
```

```
void *vptr;
```

```
vptr = &i;
printf("Value of i = %d\n", *(int *)vptr);
```

```
vptr = &f;
printf("Value of f = %.2f\n", *(float *)vptr);
```

```
vptr = &ch;
printf("Value of ch = %c\n", *(char *)vptr);
```

# Pointers and Arrays

---

By Jason Fedin

# Overview

---

- an array is a collection of objects of the same type that you can refer to using a single name
- a pointer is a variable that has as its value a memory address that can reference another variable or constant of a given type
  - you can use a pointer to hold the address of different variables at different times (must be same type)
- arrays and pointers seem quite different, but, they are very closely related and can sometimes be used interchangeably
- one of the most common uses of pointers in C is as pointers to arrays
- the main reasons for using pointers to arrays are ones of notational convenience and of program efficiency
- pointers to arrays generally result in code that uses less memory and executes faster



# Arrays and Pointers

---

- if you have an array of 100 integers

```
int values[100];
```

- you can define a pointer called valuesPtr, which can be used to access the integers contained in this array

```
int *valuesPtr;
```

- when you define a pointer that is used to point to the elements of an array, you do not designate the pointer as type “pointer to array”
  - you designate the pointer as pointing to the type of element that is contained in the array
- to set valuesPtr to point to the first element in the values array, you write

```
valuesPtr = values;
```

- the address operator is not used
  - the C compiler treats the appearance of an array name without a subscript as a pointer to the array
  - specifying values without a subscript has the effect of producing a pointer to the first element of values

# Arrays and Pointers

---

- an equivalent way of producing a pointer to the start of values is to apply the address operator to the first element of the array

```
valuesPtr = &values[0];
```

- So, you can use the above example or the one on the previous slide

```
valuesPtr = values;
```

- either one is fine and a matter of programmer preference

# Summary

---

- the two expressions `ar[i]` and `*(ar+i)` are equivalent in meaning
  - both work if `ar` is the name of an array, and both work if `ar` is a pointer variable
  - using an expression such as `ar++` only works if `ar` is a pointer variable

# Pointers and Arrays (Example)

## Jason Fedin

# Example

---

This program demonstrates the effect of adding an integer value to a pointer.

```
#include <stdio.h>
#include <string.h>

int main(void)
{
 char multiple[] = "a string";
 char *p = multiple;

 for(int i = 0 ; i < strlen(multiple, sizeof(multiple)) ; ++i)
 printf("multiple[%d] = %c *(p+%d) = %c &multiple[%d] = %p p+%d = %p\n",
 i, multiple[i], i, *(p+i), i, &multiple[i], i, p+i);
 return 0;
}
```

---



# Another Example

---

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
 long multiple[] = {15L, 25L, 35L, 45L};
```

```
 long *p = multiple;
```

```
 for(int i = 0 ; i < sizeof(multiple)/sizeof(multiple[0]) ; ++i)
```

```
 printf("address p+%d (&multiple[%d]): %llu *(p+%d) value: %d\n",
 i, i, (unsigned long long)(p+i), i, *(p+i));
```

```
 printf("\n Type long occupies: %d bytes\n", (int)sizeof(long));
```

```
 return 0;
```

```
}
```

# Pointer Arithmetic

## Jason Fedin

# pointer arithmetic

---

- the real power of using pointers to arrays comes into play when you want to sequence through the elements of an array

`*valuesPtr` // can be used to access the first integer of the values array, that is, `values[0]`

- to reference `values[3]` through the `valuesPtr` variable, you can add 3 to `valuesPtr` and then apply the indirection operator

`*(valuesPtr + 3)`

- the expression, `*(valuesPtr + i)` can be used to access the value contained in `values[i]`
- to set `values[10]` to 27, you could do the following

`values[10] = 27;`

- or, using `valuesPtr`, you could

`*(valuesPtr + 10) = 27;`



# pointer arithmetic (cont'd)

---

- to set valuesPtr to point to the second element of the values array, you can apply the address operator to values[1] and assign the result to valuesPtr

```
valuesPtr = &values[1];
```

- If valuesPtr points to values[0], you can set it to point to values[1] by simply adding 1 to the value of valuesPtr

```
valuesPtr += 1;
```

- this is a perfectly valid expression in C and can be used for pointers to any data type

# pointer arithmetic (cont'd)

---

- the increment and decrement operators ++ and -- are particularly useful when dealing with pointers
- using the increment operator on a pointer has the same effect as adding one to the pointer
- using the decrement operator has the same effect as subtracting one from the pointer

`++valuesPtr;`

- sets valuesPtr pointing to the next integer in the values array (values[1])

`--textPtr;`

- sets valuesPtr pointing to the previous integer in the values array, assuming that valuesPtr was not pointing to the beginning of the values array

# Example

---

```
int arraySum (int array[], const int n)
{
 int sum = 0, *ptr;
 int * const arrayEnd = array + n;

 for (ptr = array; ptr < arrayEnd; ++ptr)
 sum += *ptr;

 return sum;
}

void main (void)
{
 int arraySum (int array[], const int n);
 int values[10] = { 3, 7, -9, 3, 6, -1, 7, 9, 1, -5 };

 printf ("The sum is %i\n", arraySum (values, 10));
}
```

# Example (cont'd)

---

- to pass an array to a function, you simply specify the name of the array
- to produce a pointer to an array, you need only specify the name of the array
- this implies that in the call to the arraySum() function, what was passed to the function was actually a pointer to the array values
- explains why you are able to change the elements of an array from within a function
- so, you might wonder why the formal parameter inside the function is not declared to be a pointer

```
int arraySum (int *array, const int n)
```

- the above is perfectly valid
- pointers and arrays are intimately related in C
- this is why you can declare array to be of type “array of ints” inside the arraySum function or to be of type “pointer to int.”
- if you are going to be using index numbers to reference the elements of an array that is passed to a function, declare the corresponding formal parameter to be an array
- more correctly reflects the use of the array by the function
- if you are using the argument as a pointer to the array, declare it to be of type pointer.

# Example with pointer notation

---

```
int arraySum (int *array, const int n)
{
 int sum = 0;
 int * const arrayEnd = array + n;

 for (; array < arrayEnd; ++array)
 sum += *array;

 return sum;
}

void main (void)
{
 int arraySum (int *array, const int n);
 int values[10] = { 3, 7, -9, 3, 6, -1, 7, 9, 1, -5 };

 printf ("The sum is %i\n", arraySum (values, 10));
}
```



# Summary

---

```
int urn[3];
int * ptr1, * ptr2;
```

| Valid                         | Invalid                          |
|-------------------------------|----------------------------------|
| <code>ptr1++;</code>          | <code>urn++;</code>              |
| <code>ptr2 = ptr1 + 2;</code> | <code>ptr2 = ptr2 + ptr1;</code> |
| <code>ptr2 = urn + 1;</code>  | <code>ptr2 = urn * ptr1;</code>  |

Taken from C Primer Plus, Prata

# Summary (cont'd)

---

- functions that process arrays actually use pointers as arguments
- you have a choice between array notation and pointer notation for writing array-processing functions
- using array notation makes it more obvious that the function is working with arrays
- array notation has a more familiar look to programmers versed in FORTRAN, Pascal, Modula-2, or BASIC
- other programmers might be more accustomed to working with pointers and might find the pointer notation more natural
- closer to machine language and, with some compilers, leads to more efficient code

# Pointers and Strings

---

By Jason Fedin



# Overview

---

- we now know how arrays relate to pointers and the concept of pointer arithmetic
- these concepts can be very useful when applied to character arrays (strings)
- one of the most common applications of using a pointer to an array is as a pointer to a character string
  - the reasons are one of notational convenience and efficiency
  - using a variable of type pointer to char to reference a string gives you a lot of flexibility
- lets look at an example that uses an array to copy a string

# Example (array parameter vs char \* parameter)

---

```
void copyString (char to[], char from[])
{
 int i;

 for (i = 0; from[i] != '\0'; ++i)
 to[i] = from[i];

 to[i] = '\0';
}
```

```
void copyString (char *to, char *from)
{
 for (; *from != '\0'; ++from, ++to)
 *to = *from;

 *to = '\0';
}
```

# char arrays as pointers

---

- if you have an array of characters called text, you could similarly define a pointer to be used to point to elements in text

```
char *textPtr;
```

- if textPtr is set pointing to the beginning of an array of chars called text

```
++textPtr;
```

- the above sets textPtr pointing to the next character in text, which is text[1]

```
--textPtr;
```

- the above sets textPtr pointing to the previous character in text, assuming that textPtr was not pointing to the beginning of text prior to the execution of this statement

# Example optimized

---

```
void copyString (char *to, char *from)
{
 while (*from) // the null character is equal to the value 0, so will jump out then
 *to++ = *from++;

 *to = '\0';
}
```

```
int main (void)
{
 char string1[] = "A string to be copied.";
 char string2[50];

 copyString (string2, string1);
 printf ("%s\n", string2);
}
```

# Pass by reference

---

By Jason Fedin



# pass by value

---

- there are a few different ways you can pass data to a function
  - pass by value
  - pass by reference
- pass by value is when a function copies the actual value of an argument into the formal parameter of the function
  - changes made to the parameter inside the function have no effect on the argument
- C programming uses call by value to pass arguments
  - means the code within a function cannot alter the arguments used to call the function
  - there are no changes in the values, though they had been changed inside the function

# Example, pass by value

---

```
/* function definition to swap the values */
void swap(int x, int y) {
 int temp;

 temp = x; /* save the value of x */
 x = y; /* put y into x */
 y = temp; /* put temp into y */

 return;
}
```

# Example (cont'd)

---

```
int main () {

 /* local variable definition */
 int a = 100;
 int b = 200;

 printf("Before swap, value of a : %d\n", a);
 printf("Before swap, value of b : %d\n", b);

 /* calling a function to swap the values */
 swap(a, b);

 printf("After swap, value of a : %d\n", a);
 printf("After swap, value of b : %d\n", b);

 return 0;
}
```



# Passing data using copies of pointers

---

- pointers and functions get along quite well together
  - you can pass a pointer as an argument to a function and you can also have a function return a pointer as its result
- pass by reference copies the address of an argument into the formal parameter
  - the address is used to access the actual argument used in the call
  - means the changes made to the parameter affect the passed argument
- to pass a value by reference, argument pointers are passed to the functions just like any other value
  - you need to declare the function parameters as pointer types
  - changes inside the function are reflected outside the function as well
  - unlike call by value where the changes do not reflect outside the function

# Example using pointers to pass data

---

```
/* function definition to swap the values */
void swap(int *x, int *y) {

 int temp;
 temp = *x; /* save the value at address x */
 *x = *y; /* put y into x */
 y = temp; / put temp into y */

 return;
}
```

# Example using pointers to pass data (cont'd)

---

```
int main () {

 /* local variable definition */
 int a = 100;
 int b = 200;

 printf("Before swap, value of a : %d\n", a);
 printf("Before swap, value of b : %d\n", b);

 swap(&a, &b);

 printf("After swap, value of a : %d\n", a);
 printf("After swap, value of b : %d\n", b);

 return 0;
}
```

# Summary of syntax

---

- you can communicate two kinds of information about a variable to a function

function1(x):

- you transmit the value of x and the function must be declared with the same type as x

```
int function1(int num)
```

function2(&x):

- you transmit the address of x and requires the function definition to include a pointer to the correct type

```
int function2(int * ptr)
```

# const pointer parameters

---

- you can qualify a function parameter using the const keyword
  - indicates that the function will treat the argument that is passed for this parameter as a constant
  - only useful when the parameter is a pointer
- you apply the const keyword to a parameter that is a pointer to specify that a function will not change the value to which the argument points

```
bool SendMessage(const char* pmessage)
{
 // Code to send the message
 return true;
}
```

- the type of the parameter, pmessage, is a pointer to a const char.
  - it is the char value that's const, not its address.
  - you could specify the pointer itself as const too, but this makes little sense because the address is passed by value
    - you cannot change the original pointer in the calling function
- the compiler knows that an argument that is a pointer to constant data will be safe
- If you pass a pointer to constant data as the argument for a parameter then the parameter must be a use the above



# returning pointers from a function

---

- returning a pointer from a function is a particularly powerful capability
  - it provides a way for you to return not just a single value, but a whole set of values
- you would have to declare a function returning a pointer

```
int * myFunction() {
 .
 .
 .
}
```

- be careful though, there are specific hazards related to returning a pointer
  - use local variables to avoid interfering with the variable that the argument points to

# Dynamic memory allocation

---

By Jason Fedin

# Overview

---

- whenever you define a variable in C, the compiler automatically allocates the correct amount of storage for you based on the data type
- it is frequently desirable to be able to dynamically allocate storage while a program is running
- if you have a program that is designed to read in a set of data from a file into an array in memory, you have three choices
  - define the array to contain the maximum number of possible elements at compile time
  - use a variable-length array to dimension the size of the array at runtime
  - allocate the array dynamically using one of C's memory allocation routines



# Dynamic memory allocation

---

- with the first approach, you have to define your array to contain the maximum number of elements that would be read into the array

```
Int dataArray [1000];
```

- the data file cannot contain more than 1000 elements, if it does, your program will not work
  - If it is larger than 1000 you must go back to the program, change the size to be larger and recompile it
  - no matter what value you select, you always have the chance of running into the same problem again in the future
- using the dynamic memory allocation functions, you can get storage as you need it
  - this approach enables you to allocate memory as the program is executing

# Dynamic memory allocation

---

- dynamic memory allocation depends on the concept of a pointer and provides a strong incentive to use pointers in your code
- dynamic memory allocation allows memory for storing data to be allocated dynamically when your program executes
  - allocating memory dynamically is possible only because you have pointers available
- the majority of production programs will use dynamic memory allocation
- allocating data dynamically allows you to create pointers at runtime that are just large enough to hold the amount of data you require for the task

# Heap vs. Stack

---

- dynamic memory allocation reserves space in a memory area called the heap
- the stack is another place where memory is allocated
  - function arguments and local variables in a function are stored here
  - when the execution of a function ends, the space allocated to store arguments and local variables is freed
- the memory in the heap is different in that it is controlled by you
  - when you allocate memory on the heap, it is up to you to keep track of when the memory you have allocated is no longer required
  - you must free the space you have allocated to allow it to be reused

# malloc, calloc, and realloc

---

By Jason Fedin

# malloc

---

- the simplest standard library function that allocates memory at runtime is called malloc()
  - need to include the stdlib.h header file
  - you specify the number of bytes of memory that you want allocated as the argument
  - returns the address of the first byte of memory that it allocated
  - because you get an address returned, a pointer is the only place to put it

```
int *pNumber = (int*)malloc(100);
```

- in the above, you have requested 100 bytes of memory and assigned the address of this memory block to pNumber
  - pNumber will point to the first int location at the beginning of the 100 bytes that were allocated.
  - can hold 25 int values on my computer, because they require 4 bytes each
  - assumes that type int requires 4 bytes
- it would be better to remove the assumption that ints are 4 bytes

```
int *pNumber = (int*)malloc(25*sizeof(int));
```

- the argument to malloc() above is clearly indicating that sufficient bytes for accommodating 25 values of type int should be made available
- also notice the cast (int\*), which converts the address returned by the function to the type pointer to int
  - malloc returns a pointer of type pointer to void, so you have to cast



# malloc (cont'd)

---

- you can request any number of bytes
- if the memory that you requested can not be allocated for any reason
  - malloc() returns a pointer with the value NULL
  - It is always a good idea to check any dynamic memory request immediately using an if statement to make sure the memory is actually there before you try to use it

```
int *pNumber = (int*)malloc(25*sizeof(int));
if(!pNumber)
{
 // code to deal with memory allocation failure . . .
}
```

- you can at least display a message and terminate the program
  - much better than allowing the program to continue and crash when it uses a NULL address to store something

# releasing memory

---

- when you allocate memory dynamically, you should always release the memory when it is no longer required
- memory that you allocate on the heap will be automatically released when your program ends
  - better to explicitly release the memory when you are done with it, even if it's just before you exit from the program
- a memory leak occurs when you allocate some memory dynamically and you do not retain the reference to it, so you are unable to release the memory
  - often occurs within a loop
  - because you do not release the memory when it is no longer required, your program consumes more and more of the available memory on each loop iteration and eventually may occupy it all
- to free memory that you have allocated dynamically, you must still have access to the address that references the block of memory

# releasing memory (cont'd)

---

- to release the memory for a block of dynamically allocated memory whose address you have stored in a pointer

```
free(pNumber);
pNumber = NULL;
```

- the free() function has a formal parameter of type void\*
  - you can pass a pointer of any type as the argument
- as long as pNumber contains the address that was returned when the memory was allocated, the entire block of memory will be freed for further use
- you should always set the pointer to NULL after the memory that it points to has been freed



# calloc

---

- the `calloc()` function offers a couple of advantages over `malloc()`
    - it allocates memory as a number of elements of a given size
    - it initializes the memory that is allocated so that all bytes are zero
  - `calloc()` function requires two argument values
    - number of data items for which space is required
    - size of each data item
  - is declared in the `stdlib.h` header
- ```
int *pNumber = (int*) calloc(75, sizeof(int));
```
- the return value will be `NULL` if it was not possible to allocate the memory requested
 - very similar to using `malloc()`, but the big plus is that you know the memory area will be initialized to 0

realloc

- the `realloc()` function enables you to reuse or extend memory that you previously allocated using `malloc()` or `calloc()`
- expects two argument values
 - a pointer containing an address that was previously returned by a call to `malloc()`, `calloc()`
 - the size in bytes of the new memory that you want allocated
- allocates the amount of memory you specify by the second argument
 - transfers the contents of the previously allocated memory referenced by the pointer that you supply as the first argument to the newly allocated memory
 - returns a `void*` pointer to the new memory or `NULL` if the operation fails for some reason
- the most important feature of this operation is that `realloc()` preserves the contents of the original memory area

Example

```
int main () {
    char *str;

    /* Initial memory allocation */
    str = (char *) malloc(15);
    strcpy(str, "jason");
    printf("String = %s, Address = %u\n", str, str);

    /* Reallocating memory */
    str = (char *) realloc(str, 25);
    strcat(str, ".com");
    printf("String = %s, Address = %u\n", str, str);

    free(str);

    return(0);
}
```

guidelines

- avoid allocating lots of small amounts of memory
 - allocating memory on the heap carries some overhead with it
 - allocating many small blocks of memory will carry much more overhead than allocating fewer larger blocks
- only hang on to the memory as long as you need it
 - as soon as you are finished with a block of memory on the heap, release the memory
- always ensure that you provide for releasing memory that you have allocated
 - decide where in your code you will release the memory when you write the code that allocates it
- make sure you do not inadvertently overwrite the address of memory you have allocated on the heap before you have released it
 - will cause a memory leak
 - be especially careful when allocating memory within a loop

Challenge

By Jason Fedin

Requirements

- in this challenge, you are going to learn how to create, initialize, assign, and access a pointer
- write a program that creates an integer variable with a hard-coded value. Assign that variable's address to a pointer variable
- display as output the address of the pointer, the value of the pointer, and the value of what the pointer is pointing to.

Challenge

By Jason Fedin

Requirements

- In this challenge, you are going to write a program that tests your understanding of pass by reference
- write a function that squares a number by itself
 - the function should define as a parameter an int pointer

Challenge

By Jason Fedin

Requirements

- In this challenge, you are going to write a program that tests your understanding of pointer arithmetic and the const modifier
- write a function that calculates the length of a string
 - the function should take as a parameter a const char pointer
 - the function can only determine the length of the string using pointer arithmetic
 - incrementation operator (++pointer) to get to the end of the string
 - you are required to use a while loop using the value of the pointer to exit
 - the function should subtract two pointers (one pointing to the end of the string and one pointing to the beginning of the string)
 - the function should return an int that is the length of the string passed into the function

Challenge

By Jason Fedin

Requirements

- In this challenge, you are going to write a program that tests your understanding of dynamic memory allocation
- write a program that allows a user to input a text string. The program will print the text that was inputted. The program will utilize dynamic memory allocation.
- the user can enter the limit of the string they are entering
 - you can use this limit when invoking malloc
- the program should create a char pointer only, no character arrays
- be sure to release the memory that was allocated

Creating and Using Structures

By Jason Fedin

Overview

- structures in C provide another tool for grouping elements together
 - a powerful concept that you will use in many C programs that you develop
- suppose you want to store a date inside a program
 - we could create variables for month, day, and year to store the date

```
int month = 9, day = 25, year = 2015;
```

- suppose your program also needs to store the date of purchase of a particular item
 - you must keep track of three separate variables for each date that you use in the program
 - these variables are logically related and should be grouped together
- it would be much better if you could somehow group these sets of three variables together
 - this is precisely what the structure in C allows you to do

Creating a structure

- a structure declaration describes how a structure is put together
 - what elements are inside the structure
- the struct keyword enables you to define a collection of variables of various types called a structure that you can treat as a single unit

```
struct date
{
    int month;
    int day;
    int year;
};
```

- the above statement defines what a date structure looks like to the C compiler
 - there is no memory allocation for this declaration
- the variable names within the date structure, month, day, and year, are called members or fields
 - members of the structure appear between the braces that follow the struct tag name date

Using a structure

- the definition of date defines a new type in the language
 - variables can now be declared to be of type struct date

```
struct date today;
```

- you can now declare more variables of type struct date

```
struct date purchaseDate;
```

- the above statement declares a variable to be of type struct date
 - memory is now allocated for the variables above
 - memory is allocated for three integer values for each variable
- be certain you understand the difference between defining a structure and declaring variables of the particular structure type

Accessing members in a struct

- now that you know how to define a structure and declare structure variables, you need to be able to refer to the members of a structure
- a structure variable name is not a pointer
 - you need a special syntax to access the members
- you refer to a member of a structure by writing the variable name followed by a period, followed by the member variable name
 - the period between the structure variable name and the member name is called the member selection operator
 - there are no spaces permitted between the variable name, the period, and the member name
- to set the value of the day in the variable today to 25, you write

```
today.day = 25;  
today.year = 2015;
```

- to test the value of month to see if it is equal to 12

```
if ( today.month == 12 )  
    nextMonth = 1;
```

Example

```
struct date
{
    int month;
    int day;
    int year;
};
```

```
struct date today;
```

```
today.month = 9;
today.day = 25;
today.year = 2015;
```

```
printf ("Today's date is %i/%i/%.2i.\n", today.month, today.day, today.year % 100);
```

Structures in expressions

- when it comes to the evaluation of expressions, structure members follow the same rules as ordinary variables do
 - division of an integer structure member by another integer is performed as an integer division

```
century = today.year / 100 + 1;
```

Defining the structure and variable at the same time

- you do have some flexibility in defining a structure
 - it is valid to declare a variable to be of a particular structure type at the same time that the structure is defined
 - include the variable name (or names) before the terminating semicolon of the structure definition
 - you can also assign initial values to the variables in the normal fashion

```
struct date
{
    int month;
    int day;
    int year;
} today;
```

- in the above, an instance of the structure, called today, is declared at the same time that the structure is defined
 - today is a variable of type date

Un-named Structures

- you also do not have to give a structure a tag name
 - If all of the variables of a particular structure type are defined when the structure is defined, the structure name can be omitted

```
struct
{
    // Structure declaration and...
    int day;
    int year;
    int month;
} today; // ...structure variable declaration combined
```

- a disadvantage of the above is that you can no longer define further instances of the structure in another statement
 - all the variables of this structure type that you want in your program must be defined in the one statement

Initializing Structures

- initializing structures is similar to initializing arrays
 - the elements are listed inside a pair of braces, with each element separated by a comma
 - the initial values listed inside the curly braces must be constant expressions

```
struct date  today = { 7, 2, 2015 };
```

- just like an array initialization, fewer values might be listed than are contained in the structure

```
struct date  date1 = { 12, 10 };
```

- sets date1.month to 12 and date1.day to 10 but gives no initial value to date.year

Initializing structures

- you can also specify the member names in the initialization list
 - enables you to initialize the members in any order, or to only initialize specified members

`.member = value`

```
struct date date1 = { .month = 12, .day = 10 };
```

- set just the year member of the date structure variable today to 2015

```
struct date today = { .year = 2015 };
```

Assignment with compound literals

- you can assign one or more values to a structure in a single statement using what is known as compound literals

```
today = (struct date) { 9, 25, 2015 };
```

- this statement can appear anywhere in the program
 - it is not a declaration statement
 - the type cast operator is used to tell the compiler the type of the expression
 - the list of values follows the cast and are to be assigned to the members of the structure, in order
 - listed in the same way as if you were initializing a structure variable
- you can also specify values using the .member notation

```
today = (struct date) { .month = 9, .day = 25, .year = 2015 };
```

- the advantage of using this approach is that the arguments can appear in any order

Structures and Arrays

By Jason Fedin

Arrays of structures

- you have seen how useful a structure is in enabling you to logically group related elements together
 - for example, it is only necessary to keep track of one variable, instead of three, for each date that is used by the program
 - to handle 10 different dates in a program, you only have to keep track of 10 different variables, instead of 30
- a better method for handling the 10 different dates involves the combination of two powerful features of the C programming language
 - structures and arrays.
 - it is perfectly valid to define an array of structures
 - the concept of an array of structures is a very powerful and important one in C
- declaring an array of structures is like declaring any other kind of array

```
struct date myDates[10];
```

- defines an array called myDates, which consists of 10 elements
 - each element inside the array is defined to be of type struct date

Array of structures

- to identify members of an array of structures, you apply the same rule used for individual structures
 - follow the structure name with the dot operator and then with the member name
- referencing a particular structure element inside the array is quite natural
 - to set the second date inside the myDates array to August 8, 1986

```
myDates[1].month = 8;  
myDates[1].day   = 8;  
myDates[1].year  = 1986;
```

Initializing an array of structures

- initialization of arrays containing structures is similar to initialization of multidimensional arrays

```
struct date myDates[5] = { {12, 10, 1975}, {12, 30, 1980}, {11, 15, 2005} };
```

- sets the first three dates in the array myDate to 12/10/1975, 12/30/1980, and 11/15/2005
- the inner pairs of braces are optional

```
struct date myDates[5] = { 12, 10, 1975, 12, 30, 1980, 11, 15, 2005 };
```

- initializes just the third element of the array to the specified value

```
struct date myDates[5] = { [2] = {12, 10, 1975} };
```

- sets just the month and day of the second element of the myDates array to 12 and 30

```
struct date myDates[5] = { [1].month = 12, [1].day = 30 };
```

Structures containing arrays

- it is also possible to define structures that contain arrays as members
 - most common use is to set up an array of characters inside a structure
- suppose you want to define a structure called month that contains as its members the number of days in the month as well as a three-character abbreviation for the month name

```
struct month
{
    int    numberOfDays;
    char   name[3];
};
```

- this sets up a month structure that contains an integer member called numberOfDays and a character member called name
 - member name is actually an array of three characters

Structures containing arrays

- you can now define a variable to be of type struct month and set the proper fields inside aMonth for January

```
struct month aMonth;  
aMonth.numberOfDay = 31;  
aMonth.name[0] = 'J';  
aMonth.name[1] = 'a';  
aMonth.name[2] = 'n';
```

- you can also initialize this variable to the same values

```
struct month aMonth = { 31, { 'J', 'a', 'n' } };
```

- you can set up 12-month structures inside an array to represent each month of the year

```
struct month months[12];
```

Nested Structures

By Jason Fedin

Nested Structures

- C allows you to define a structure that itself contains other structures as one or more of its members
- you have seen how it is possible to logically group the month, day, and year into a structure called date
 - how about grouping the hours, minutes, and seconds into a structure called time

```
struct time
{
    int hours;
    int minutes;
    int seconds;
};
```

- in some applications, you might have the need to group both a date and a time together
 - you might need to set up a list of events that are to occur at a particular date and time

Nested Structures

- you want to have a convenient way to associate both the date and the time together
 - define a new structure, called, for example, `dateAndTime`, which contains as its members two elements
 - date and time

```
struct dateAndTime
{
    struct date    sdate;
    struct time    stime;
};
```

- the first member of this structure is of type `struct date` and is called `sdate`
- the second member of the `dateAndTime` structure is of type `struct time` and is called `stime`
- variables can now be defined to be of type `struct dateAndTime`

```
struct dateAndTime event;
```

Accessing members in a nested structure

- to reference the date structure of the variable event, the syntax is the same as referencing any member

`event.sdate`

- to reference a particular member inside one of these structures, a period followed by the member name is tacked on the end
 - the below statement sets the month of the date structure contained within event to October, and adds one to the seconds contained within the time structure

```
event.sdate.month = 10;  
++event.stime.seconds;
```

Accessing members in a nested structure

- the event variable can be initialized just like normal
 - sets the date in the variable event to February 1, 2015, and sets the time to 3:30:00.

```
struct dateAndTime event = { { 2, 1, 2015 }, { 3, 30, 0 } };
```

- you can use members' names in the initialization

```
struct dateAndTime event =  
    { { .month = 2, .day = 1, .year = 2015 },  
      { .hour = 3, .minutes = 30, .seconds = 0 }  
    };
```

An array of nested structures

- it is also possible to set up an array of `dateAndTime` structures

```
struct dateAndTime events[100];
```

- the array `events` is declared to contain 100 elements of type `struct dateAndTime`
 - the fourth `dateAndTime` contained within the array is referenced in the usual way as `events[3]`
- to set the first time in the array to noon

```
events[0].stime.hour   = 12;  
events[0].stime.minutes = 0;  
events[0].stime.seconds = 0;
```

Declaring a structure within a structure

- you can define the Date structure within the time structure definition

```
struct Time
```

```
{  
    struct Date  
    {  
        int day;  
        int month;  
        int year;  
    } dob;  

```

```
    int hour;  
    int minutes;  
    int seconds;  
};
```

- the declaration is enclosed within the scope of the Time structure definition
 - it does not exist outside it
 - it becomes impossible to declare a Date variable external to the Time structure

Structures and Pointers

By Jason Fedin

Structures and Pointers

- C allows for pointers to structures
- pointers to structures are easier to manipulate than structures themselves
- in some older implementations, a structure cannot be passed as an argument to a function, but a pointer to a structure can.
- even if you can pass a structure as an argument, passing a pointer is more efficient
- many data representations use structures containing pointers to other structures

Declaring a struct as a pointer

- you can define a variable to be a pointer to a struct

```
struct date *datePtr;
```

- the variable datePtr can be assigned just like other pointers
 - you can set it to point to todaysDate with the assignment statement

```
datePtr = &todaysDate;
```

- you can then indirectly access any of the members of the date structure pointed to by datePtr

```
(*datePtr).day = 21;
```

- the above has the effect of setting the day of the date structure pointed to by datePtr to 21
 - parentheses are required because the structure member operator . has higher precedence than the indirection operator *

Using structs as pointers

- to test the value of month stored in the date structure pointed to by datePtr

```
if ( (*datePtr).month == 12 )
```

```
...
```

- pointers to structures are so often used in C that a special operator exists
 - the structure pointer operator ->, which is the dash followed by the greater than sign, permits

```
(*x).y
```

to be more clearly expressed as

```
x->y
```

- the previous if statement can be conveniently written as

```
if ( datePtr->month == 12 )
```

```
...
```

Example

```
struct date
```

```
{
```

```
    int month;
```

```
    int day;
```

```
    int year;
```

```
};
```

```
struct date  today, *datePtr;
```

```
datePtr = &today;
```

```
datePtr->month = 9;
```

```
datePtr->day = 25;
```

```
datePtr->year = 2015;
```

```
printf ("Today's date is %i/%i/%.2i.\n", datePtr->month, datePtr->day, datePtr->year % 100);
```

Structures containing pointers

- a pointer also can be a member of a structure

```
struct intPtrs
{
    int *p1;
    int *p2;
};
```

- a structure called intPtrs is defined to contain two integer pointers
 - the first one called p1
 - the second one p2
- you can define a variable of type struct intPtrs

```
struct intPtrs pointers;
```

- the variable pointers can now be used just like other structs
 - pointers itself is not a pointer, but a structure variable that has two pointers as its members

Example

```
struct intPtrs
{
    int *p1;
    int *p2;
};
```

```
struct intPtrs pointers;
int i1 = 100, i2;
```

```
pointers.p1 = &i1;
pointers.p2 = &i2;
*pointers.p2 = -97;
```

```
printf ("i1 = %i, *pointers.p1 = %i\n", i1, *pointers.p1);
printf ("i2 = %i, *pointers.p2 = %i\n", i2, *pointers.p2);
```

Character arrays or character pointers??

```
struct names {  
    char first[20];  
    char last[20];  
};
```

OR

```
struct pnames {  
    char * first;  
    char * last;  
};
```

- you can do both, however, you need to understand what is happening here

Character arrays or character pointers??

```
struct names veep = {"Talia", "Summers"};  
struct pnames treas = {"Brad", "Fallingjaw"};  
printf("%s and %s\n", veep.first, treas.first);
```

- the struct names variable veep
 - strings are stored inside the structure
 - structure has allocated a total of 40 bytes to hold the two names
- the struct pnames variable treas
 - strings are stored wherever the compiler stores string constants
 - the structure holds the two addresses, which takes a total of 16 bytes on our system
 - the struct pnames structure allocates no space to store strings
 - it can be used only with strings that have had space allocated for them elsewhere
 - such as string constants or strings in arrays
- the pointers in a pnames structure should be used only to manage strings that were created and allocated elsewhere in the program

Character arrays or character pointers??

- one instance in which it does make sense to use a pointer in a structure to handle a string is if you are dynamically allocating that memory
 - use a pointer to store the address
 - has the advantage that you can ask malloc() to allocate just the amount of space that is needed for a string

Example

```
struct namect {  
    char * fname; // using pointers instead of arrays  
    char * lname;  
    int letters;  
};
```

- understand that the two strings are not stored in the structure
 - stored in the chunk of memory managed by malloc()
 - the addresses of the two strings are stored in the structure
 - addresses are what string-handling functions typically work with

Example

```
void getinfo (struct namect * pst)
{
    char temp[SLEN];
    printf("Please enter your first name.\n");
    s_gets(temp, SLEN);

    // allocate memory to hold name
    pst->fname = (char *) malloc(strlen(temp) + 1);

    // copy name to allocated memory
    strcpy(pst->fname, temp);
    printf("Please enter your last name.\n");
    s_gets(temp, SLEN);
    pst->lname = (char *) malloc(strlen(temp) + 1);
    strcpy(pst->lname, temp);
}
```


Structures and Functions

By Jason Fedin

Structures as arguments to functions

- after declaring a structure named Family, how do we pass this structure as an argument to a function?

```
struct Family {  
    char name[20];  
    int age;  
    char father[20];  
    char mother[20];  
};
```

```
bool siblings(struct Family member1, struct Family member2) {  
    if(strcmp(member1.mother, member2.mother) == 0)  
        return true;  
    else  
        return false;  
}
```

- this function has two parameters, each of which is a structure

Pointers to Structures as function arguments

- you should use a pointer to a structure as an argument
 - it can take quite a bit of time to copy large structures as arguments, as well as requiring whatever amount of memory to store the copy of the structure.
 - pointers to structures avoid the memory consumption and the copying time (only a copy of the pointer argument is made)

```
bool siblings(struct Family *pmember1, struct Family *pmember2)
{
    if(strcmp(pmember1->mother, pmember2->mother) == 0)
        return true;
    else
        return false;
}
```

- you can also use the const modifier to not allow any modification of the members of the struct (what the struct is pointing to)

```
bool siblings(Family const *pmember1, Family const *pmember2)
{
    if(strcmp(pmember1->mother, pmember2->mother) == 0)
        return true;
    else
        return false;
}
```

Pointers to Structures as function arguments

- you can also use the const modifier to not allow any modification of the pointers address
 - any attempt to change those structures will cause an error message during compilation

```
bool siblings(Family *const pmember1, Family *const pmember2)
{
    if(strcmp(pmember1->mother, pmember2->mother) == 0)
        return true;
    else
        return false;
}
```

- the indirection operator in each parameter definition is now in front of the const keyword
 - not in front of the parameter name
 - you cannot modify the addresses stored in the pointers
 - its the pointers that are protected here, not the structures to which they point

Returning a structure from a function

- the function prototype has to indicate this return value in the normal way

```
struct Date my_fun(void);
```

- this is a prototype for a function taking no arguments that returns a structure of type Date
- it is often more convenient to return a pointer to a structure
 - when returning a pointer to a structure, it should be created on the heap

Example

```
struct funds {  
    char  bank[FUNDLEN];  
    double bankfund;  
    char  save[FUNDLEN];  
    double savefund;  
};
```

```
double sum(struct funds moolah)  
{  
    return(moolah.bankfund + moolah.savefund);  
}
```

```
int main(void)  
{  
    struct funds stan = {  
        "Garlic-Melon Bank",  
        4032.27,  
        "Lucky's Savings and Loan",  
        8543.94  
    };  
  
    printf("Stan has a total of $%.2f.\n",  
sum(stan));  
  
    return 0;  
}
```

Reminder

- I mentioned earlier that you should always use pointers when passing structures to a function
 - it works on older as well as newer C implementations and that it is quick (you just pass a single address)
- however, you have less protection for your data
 - some operations in the called function could inadvertently affect data in the original structure
 - use const qualifier solves that problem
- advantages of passing structures as arguments
 - the function works with copies of the original data, which is safer than working with the original data
 - the programming style tends to be clearer
- main disadvantages to passing structures as arguments
 - older implementations might not handle the code
 - wastes time and space
 - especially wasteful to pass large structures to a function that uses only one or two members of the structure
- programmers use structure pointers as function arguments for reasons of efficiency and use const when necessary
- passing structures by value is most often done for structures that are small

Challenge

By Jason Fedin

Requirements

- write a program that declares a structure and prints out its content
 - create an employee structure with 3 members
 - name (character array)
 - hireDate (int)
 - salary (float)
- declare and initialize an instance of an employee type
- read in a second employee from the console and store it in a structure of type employee
- print out the contents of each employee

Challenge

By Jason Fedin

Requirements

- write a C program that creates a structure pointer and passes it to a function
 - create a structure named item with the following members
 - itemName – pointer
 - quantity – int
 - price – float
 - amount – float (stores quantity * price)
- create a function named readItem that takes a structure pointer of type item as a parameter
 - this function should read in (from the user) a product name, price, and quantity
 - the contents read in should be stored in the passed in structure to the function
- create a function named print item that takes as a parameter a structure pointer of type item
 - function prints the contents of the parameter
- the main function should declare an item and a pointer to the item
 - you will need to allocate memory for the itemName pointer
 - the item pointer should be passed into both the read and print item functions

Overview

By Jason Fedin

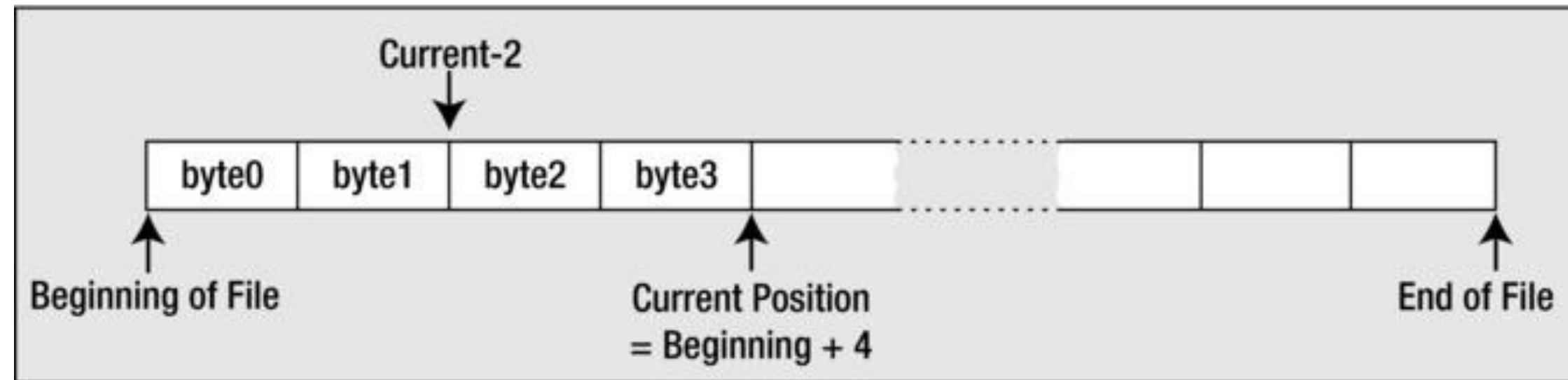
Overview

- up until this point, all data that our program accesses is via memory
 - scope and variety of applications you can create is limited
- all serious business applications require more data than would fit into main memory
 - also depend on the ability to process data that is persistent and stored on an external device such as a disk drive
- C provides many functions in the header file `stdio.h` for writing to and reading from external devices
 - the external device you would use for storing and retrieving data is typically a disk drive
 - however, the library will work with virtually any external storage device
- with all the examples up to now, any data that the user enters is lost once the program ends
 - if the user wants to run the program with the same data, he or she must enter it again each time
 - very inconvenient and limits programming
 - referred to as volatile memory

Files

- programs need to store data on permanent storage
 - non-volatile
 - continues to be maintained after your computer is turned off
- a file can store non-volatile data and is usually stored on a disk or a solid-state device
 - a named section of storage
 - `stdio.h` is a file containing useful information
- C views a file as a continuous sequence of bytes
 - each byte can be read individually
 - corresponds to the file structure in the Unix environment

Files



Taken from Beginning C,
Horton

- a file has a beginning and an end and a current position (defined as so many bytes from the beginning)
- the current position is where any file action (read/write) will take place
 - you can move the current position to any point in the file (even the end)

Text and binary files

- there are two ways of writing data to a stream that represents a file
 - text
 - binary
- text data is written as a sequence of characters organized as lines (each line ends with a newline)
- binary data is written as a series of bytes exactly as they appear in memory
 - image data, music encoding – not readable
- you can write any data you like to a file
 - once a file has been written, it just consists of a series of bytes
- you have to understand the format of the file in order to read it
 - a sequence of 12 bytes in a binary file could be 12 characters, 12 8-bit signed integers, 12 8-bit unsigned integers, etc.
 - in binary mode, each and every byte of the file is accessible

Streams

- C programs automatically open three files on your behalf
 - standard input - the normal input device for your system, usually your keyboard
 - standard output - usually your display screen
 - standard error - usually your display screen
- standard input is the file that is read by `getchar()` and `scanf()`
- standard output is used by `putchar()`, `puts()`, and `printf()`
 - redirection causes other files to be recognized as the standard input or standard output.
- the purpose of the standard error output file is to provide a logically distinct place to send error messages
- a stream is an abstract representation of any external source or destination for data
 - the keyboard, the command line on your display, and files on a disk are all examples of things you can work with as streams
 - the C library provides functions for reading and writing to or from data streams
 - you use the same input/output functions for reading and writing any external device that is mapped to a stream

Accessing Files

By Jason Fedin

Accessing Files

- files on disk have a name and the rules for naming files are determined by your operating system
 - You may have to adjust the names depending on what OS your program is running
- a program references a file through a file pointer (or stream pointer, since it works on more than a file)
 - you associate a file pointer with a file programmatically when the program is run
 - pointers can be reused to point to different files on different occasions
- a file pointer points to a struct of type FILE that represents a stream
 - contains information about the file
 - whether you want to read or write or update the file
 - the address of the buffer in memory to be used for data
 - a pointer to the current position in the file for the next operation
 - the above is all set via input/output file operations
- if you want to use several files simultaneously in a program, you need a separate file pointer for each file
 - there is a limit to the number of files you can have open at one time
 - defined as FOPEN_MAX in stdio.h

Opening a File

- you associate a specific external file name with an internal file pointer variable through a process referred to as opening a file
 - via the `fopen()` function
 - returns the file pointer for a specific external file
- the `fopen()` function is defined in `stdio.h`

`FILE *fopen(const char * restrict name, const char * restrict mode);`

- The first argument to the function is a pointer to a string that is the name of the external file you want to process
 - you can specify the name explicitly or use a char pointer that contains the address of the character string that defines the file name
 - you can obtain the file name through the command line, as input from the user, or defined as a constant in your program
- the second argument to the `fopen()` function is a character string that represents the file mode
 - specifies what you want to do with the file
 - a file mode specification is a character string between double quotes
- assuming the call to `fopen()` is successful, the function returns a pointer of type `FILE*` that you can use to reference the file in further input/output operations using other functions in the library
- if the file cannot be opened for some reason, `fopen()` returns `NULL`

File Modes (only apply to text files)

Mode	Description
"w"	Open a text file for write operations. If the file exists, its current contents are discarded.
"a"	Open a text file for append operations. All writes are to the end of the file.
"r"	Open a text file for read operations.
"w+"	Open a text file for update (reading and writing), first truncating the file to zero length if it exists or creating the file if it does not exist
"a+"	Open a text file for update (reading and writing) appending to the end of the existing file, or creating the file if it does not yet exist
"r+"	Open a text file for update (for both reading and writing)

Write Mode

- If you want to write to an existing text file with the name myfile.txt

```
FILE *pfile = NULL;
char *filename = "myfile.txt";
pfile = fopen(filename, "w"); // Open myfile.txt to write it
If(pfile != NULL)
    printf("Failed to open %s.\n", filename);
```

- opens the file and associates the file with the name myfile.txt with your file pointer pfile
 - the mode as "w" means you can only write to the file
 - you cannot read it
- If a file with the name myfile.txt does not exist, the call to fopen() will create a new file with this name
- if you only provide the file name without any path specification, the file is assumed to be in the current directory
 - you can also specify a string that is the full path and name for the file
- on opening a file for writing, the file length is truncated to zero and the position will be at the beginning of any existing data for the first operation
 - any data that was previously written to the file will be lost and overwritten by any write operations

Append Mode

- If you want to add to an existing text file rather than overwrite it
 - specify mode "a"
 - the append mode of operation.
- this positions the file at the end of any previously written data
 - If the file does not exist, a new file will be created

```
pFile = fopen("myfile.txt", "a");    // Open myfile.txt to add to it
```

- do not forget that you should test the return value for null each time
- when you open a file in append mode
 - all write operations will be at the end of the data in the file on each write operation
 - all write operations append data to the file and you cannot update the existing contents in this mode

Read Mode

- if you want to read a file
 - open it with mode argument as "r"
 - you can not write to this file

```
pFile = fopen("myfile.txt", "r");
```

- this positions the file to the beginning of the data
- if you are going to read the file
 - it must already exist
- If you try to open a file for reading that does not exist, `fopen()` will return a file pointer of `NULL`
- you always want to check the value returned from `fopen`

Renaming a file

- renaming a file is very easy
 - use the rename() function

```
int rename(const char *oldname, const char *newname);
```

- the integer that is returned will be 0 if the name change was successful and nonzero otherwise
- the file must not be open when you call rename(), otherwise the operation will fail

```
if(rename( "C:\\temp\\myfile.txt", "C:\\temp\\myfile_copy.txt"))  
    printf("Failed to rename file.");  
else  
    printf("File renamed successfully.");
```

- this will change the name of myfile.txt in the temp directory on drive C to myfile_copy.txt
 - if the file path is incorrect or the file does not exist, the renaming operation will fail
-

Closing a file

- when you have finished with a file, you need to tell the operating system so that it can free up the file
 - you can do this by calling the `fclose()` function
- `fclose()` accepts a file pointer as an argument
 - returns EOF (int) if an error occurs
 - EOF is a special character called the end-of-file character
 - defined in `stdio.h` as a negative integer that is usually equivalent to the value `-1`
 - 0 if successful

```
fclose(pfile);           // Close the file associated with pfile
pfile = NULL;
```

- the result of calling `fclose()` is that the connection between the pointer, `pfile`, and the physical file is broken
 - `pfile` can no longer be used to access the file
- if the file was being written, the current contents of the output buffer are written to the file to ensure that data is not lost
- It is good programming practice to close a file as soon as you have finished with it
 - protects against output data loss
- you must also close a file before attempting to rename it or remove it.

Deleting a file

- you can delete a file by invoking the `remove()` function
 - declared in `stdio.h`

```
remove("myfile.txt");
```

- will delete the file that has the name `myfile.txt` from the current directory
- the file cannot be open when you try to delete it
- you should always double check with operations that delete files
 - you could wreck your system if you do not

Reading from a file

By Jason Fedin

Reading characters from a text file

- the `fgetc()` function reads a character from a text file that has been opened for reading
- takes a file pointer as its only argument and returns the character read as type `int`

```
int mchar = fgetc(pfile);           // Reads a character into mchar with pfile a File pointer
```

- the `mchar` is type `int` because `EOF` will be returned if the end of the file has been reached
- the function `getc()`, which is equivalent to `fgetc()`, is also available
 - requires an argument of type `FILE*` and returns the character read as type `int`
 - virtually identical to `fgetc()`
 - only difference between them is that `getc()` may be implemented as a macro, whereas `fgetc()` is a function
- you can read the contents of a file again when necessary
 - the `rewind()` function positions the file that is specified by the file pointer argument at the beginning

```
rewind(pfile);
```

C FOR BEGINNERS

Reading from a file

Example

```
#include <stdio.h>
```

```
int main () {
```

```
    FILE *fp;
```

```
    int c;
```

```
    fp = fopen("file.txt","r");
```

```
    if(fp == NULL) {
```

```
        perror("Error in opening file");
```

```
        return(-1);
```

```
    }
```

```
    // read a single char
```

```
    while((c = fgetc(fp)) != EOF)
```

```
        printf("%c", c);
```

```
    fclose(fp);
```

```
    fp = NULL;
```

```
    return(0);
```

```
} // return from main
```

Reading a string from a text file

- you can use the `fgets()` function to read from any file or stream

`char *fgets(char *str, int nchars, FILE *stream)`

- the function reads a string into the memory area pointed to by `str`, from the file specified by `stream`
 - characters are read until either a `'\n'` is read or `nchars-1` characters have been read from the stream, whichever occurs first
- if a newline character is read, it's retained in the string
 - a `'\0'` character will be appended to the end of the string
- if there is no error, `fgets()` returns the pointer, `str`
- if there is an error, `NULL` is returned
- reading EOF causes `NULL` to be returned

Example

```
#include <stdio.h>

int main () {
    FILE *fp;
    char str[60];

    /* opening file for reading */
    fp = fopen("file.txt" , "r");

    if(fp == NULL) {
        perror("Error opening file");
        return(-1);
    }

    if( fgets (str, 60, fp)!=NULL ) {
        /* writing content to stdout */
        printf("%s", str);
    }

    fclose(fp);
    fp = NULL;
    return(0);

} // end main
```


Reading formatted input from a file

- you can get formatted input from a file by using the standard fscanf() function

```
int fscanf(FILE *stream, const char *format, ...)
```

- the first argument to this function is the pointer to a FILE object that identifies the stream
- the second argument to this function is the format
 - a C string that contains one or more of the following items
 - whitespace character
 - non-whitespace character
 - format specifiers
 - usage is similar to scanf, but, from a file
- function returns the number of input items successfully matched and assigned

Overview

```
#include <stdio.h>
#include <stdlib.h>

int main () {
    char str1[10], str2[10], str3[10];
    int year;
    FILE * fp;

    fp = fopen ("file.txt", "w+");
    if (fp != NULL)
        fputs("Hello how are you", fp);
```

```
rewind(fp);

fscanf(fp, "%s %s %s %d", str1, str2, str3, &year);

printf("Read String1 |%s|\n", str1 );
printf("Read String2 |%s|\n", str2 );
printf("Read String3 |%s|\n", str3 );
printf("Read Integer |%d|\n", year );

fclose(fp);

return(0);
} // end of main
```

Writing to a file

Jason Fedin

Writing characters to a text file

- the simplest write operation is provided by the function `fputc()`
 - writes a single character to a text file

```
int fputc(int ch, FILE *pfile);
```

- the function writes the character specified by the first argument to the file identified by the second argument (file pointer)
 - returns the character that was written if successful
 - Return EOF if failure
- In practice, characters are not usually written to a physical file one by one
 - extremely inefficient
- the `putc()` function is equivalent to `fputc()`
 - requires the same arguments and the return type is the same
 - difference between them is that `putc()` may be implemented in the standard library as a macro, whereas `fputc()` is a function

Example

```
#include <stdio.h>
```

```
int main () {
```

```
    FILE *fp;
```

```
    int ch;
```

```
    fp = fopen("file.txt", "w+");
```

```
    for( ch = 33 ; ch <= 100; ch++ ) {
```

```
        fputc(ch, fp);
```

```
    }
```

```
    fclose(fp);
```

```
    return(0);
```

```
}
```


Writing a string to a text file

- you can use the `fputs()` function to write to any file or stream

```
int fputs(const char * str, FILE * pfile);
```

- the first argument is a pointer to the character string that is to be written to the file
- the second argument is the file pointer
- this function will write characters from a string until it reaches a `'\0'` character
 - does not write the null terminator character to the file
 - can complicate reading back variable-length strings from a file that have been written by `fputs()`
 - expecting to write a line of text that has a newline character at the end

Example

```
#include <stdio.h>
```

```
int main () {
```

```
    FILE *filePointer;
```

```
    filePointer = fopen("file.txt", "w+");
```

```
    fputs("This is Jason Fedin Course.", filePointer);
```

```
    fputs("I am happy to be here", filePointer);
```

```
    fclose(filePointer);
```

```
    return(0);
```

```
}
```

Writing formatted output to a file

- the standard function for formatted output to a stream is `fprintf()`

`int fprintf(FILE *stream, const char *format, ...)`

- the first argument to this function is the pointer to a FILE object that identifies the stream
- the second argument to this function is the format
 - a C string that contains one or more of the following items
 - whitespace character
 - non-whitespace character
 - format specifiers
 - usage is similar to `printf`, but, to a file
- if successful, the total number of characters written is returned otherwise, a negative number is returned

Example

```
#include <stdio.h>
#include <stdlib.h>

int main () {
    FILE * fp;

    fp = fopen ("file.txt", "w+");
    fprintf(fp, "%s %s %s %s %d", "Hello", "my", "number", "is", 555);

    fclose(fp);
    return(0);
}
```

File Positioning

By Jason Fedin

File Positioning

- for many applications, you need to be able to access data in a file other than sequential order
- there are various functions that you can use to access data in random sequence
- there are two aspects to file positioning
 - finding out where you are in a file
 - moving to a given point in a file
- you can access a file at a random position regardless of whether you opened the file

Finding out where you are

- you have two functions to tell you where you are in a file
 - `ftell()`
 - `fgetpos()`

```
long ftell(FILE *pfile);
```

- this function accepts a file pointer as an argument and returns a long integer value that specifies the current position in the file

```
long fpos = ftell(pfile);
```

- the `fpos` variable now holds the current position in the file and you can use this to return to this position at any subsequent time
 - value is the offset in bytes from the beginning of the file

ftell() example

```
FILE *fp;
int len;

fp = fopen("file.txt", "r");
if( fp == NULL ) {
    perror ("Error opening file");
    return(-1);
}
fseek(fp, 0, SEEK_END);

len = ftell(fp);
fclose(fp);

printf("Total size of file.txt = %d bytes\n", len);
```


fgetpos()

- the second function providing information on the current file position is a little more complicated

```
int fgetpos(FILE * pfile, fpos_t * position);
```

- the first parameter is a file pointer
- the second parameter is a pointer to a type that is defined in stdio.h
 - fpos_t - a type that is able to record every position within a file
- the fgetpos() function is designed to be used with the positioning function fsetpos()
- the fgetpos() function stores the current position and file state information for the file in position and returns 0 if the operation is successful
 - returns a nonzero integer value for failure

```
fpos_t here;  
fgetpos(pfile, &here);
```

- the above records the current file position in the variable here
- you must declare a variable of type fpos_t
 - cannot declare a pointer of type fpos_t* because there will not be any memory allocated to store the position data

fgetpos() example

```
FILE *fp;
```

```
fpos_t position;
```

```
fp = fopen("file.txt", "w+");
```

```
fgetpos(fp, &position);
```

```
fputs("Hello, World!", fp);
```

```
fclose(fp);
```

Setting a position in a file

- as a complement to `ftell()`, you have the `fseek()` function

```
int fseek(FILE *pfile, long offset, int origin);
```

- the first parameter is a pointer to the file you are repositioning
- the second and third parameters define where you want to go in the file
 - second parameter is an offset from a reference point specified by the third parameter
 - reference point can be one of three values that are specified by the predefined names
 - `SEEK_SET` - defines the beginning of the file
 - `SEEK_CUR` - defines the current position in the file
 - `SEEK_END` - defines the end of the file
- for a text mode file, the second argument must be a value returned by `ftell()`
- the third argument for text mode files must be `SEEK_SET`.
 - for text files, all operations with `fseek()` are performed with reference to the beginning of the file
 - for binary files, the offset argument is simply a relative byte count
 - can therefore supply positive or negative values for the offset when the reference point is specified as `SEEK_CUR`

fseek() example

```
#include <stdio.h>
```

```
int main () {
```

```
    FILE *fp;
```

```
    fp = fopen("file.txt","w+");
```

```
    fputs("This is Jason", fp);
```

```
    fseek( fp, 7, SEEK_SET );
```

```
    fputs(" Hello how are you", fp);
```

```
    fclose(fp);
```

```
    return(0);
```

```
}
```

fsetpos()

- you have the fsetpos() function to go with fgetpos()

```
int fsetpos(FILE *pfile, const fpos_t *position);
```

- the first parameter is a pointer to the open file
- the second is a pointer of the fpos_t type
 - the position that is stored at the address was obtained by calling fgetpos()

```
fsetpos(pfile, &here);
```

- the variable here was previously set by a call to fgetpos()
- the fsetpos() returns a nonzero value on error or 0 when it succeeds
- this function is designed to work with a value that is returned by fgetpos()
 - you can only use it to get to a place in a file that you have been before
 - fseek() allows you to go to any position just by specifying the appropriate offset

fgetpos() example

```
#include <stdio.h>
```

```
int main () {
```

```
    FILE *fp;
```

```
    fpos_t position;
```

```
    fp = fopen("file.txt", "w+");
```

```
    fgetpos(fp, &position);
```

```
    fputs("Hello, World!", fp);
```

```
    fsetpos(fp, &position);
```

```
    fputs("This is going to override previous content", fp);
```

```
    fclose(fp);
```

```
    return(0);
```

```
}
```

Challenge

By Jason Fedin

Requirements

- write a program to find the total number of lines in a text file
- create a file that contains some lines of text
- open your test file
- use the fgetc function to parse characters in a file until you get to the EOF
 - If EOF increment counter
- display as output the total number of lines in the file
- close the file

Challenge

By Jason Fedin

Requirements

- write a program that converts all characters of a file to uppercase and write the results out to a temporary file. Then rename the temporary file to the original filename and remove the temporary filename
- use the `fgetc` and `fputc` functions
- use the `rename` and `remove` functions
- use the `islower` function
 - can convert a character to upper case by subtracting 32 from it
- display the contents of the original file to standard output
 - in uppercase

Challenge

By Jason Fedin

Requirements

- write a program that will print the contents of a file in reverse order
- use the fseek function to seek to the end of the file
- use the ftell function to get the position of the file pointer
- display as output the file in reverse order

Standard Header Files

By Jason Fedin

stddef.h

<stddef.h> - contains some standard definitions

Define	Meaning
NULL	A null pointer constant
offsetof (<i>structure</i> , <i>member</i>)	The offset in bytes of the member <i>member</i> from the start of the structure <i>structure</i> ; the type of the result is <code>size_t</code>
ptrdiff_t	The type of integer produced by subtracting two pointers
size_t	The type of integer produced by the <code>sizeof</code> operator
wchar_t	The type of the integer required to hold a wide character (see Appendix A, “C Language Summary”)

Taken from Programming in C,
Kochan

limits.h

<limits.h> - contains various implementation-defined limits for character and integer data types

Define	Meaning
CHAR_BIT	Number of bits in a char (8)
CHAR_MAX	Maximum value for object of type char (127 if sign extension is done on chars, 255 otherwise)
CHAR_MIN	Minimum value for object of type char (-127 if sign extension is done on chars, 0 otherwise)
SCHAR_MAX	Maximum value for object of type signed char (127)
SCHAR_MIN	Minimum value for object of type signed char (-127)
UCHAR_MAX	Maximum value for object of type unsigned char (255)
SHRT_MAX	Maximum value for object of type short int (32767)
SHRT_MIN	Minimum value for object of type short int (-32767)
USHRT_MAX	Maximum value for object of type unsigned short int (65535)
INT_MAX	Maximum value for object of type int (32767)
INT_MIN	Minimum value for object of type int (-32767)
UINT_MAX	Maximum value for object of type unsigned int (65535)
LONG_MAX	Maximum value for object of type long int (2147483647)
LONG_MIN	Minimum value for object of type long int (-2147483647)
ULONG_MAX	Maximum value for object of type unsigned long int (4294967295)
LLONG_MAX	Maximum value for object of type long long int (9223372036854775807)
LLONG_MIN	Minimum value for object of type long long int (-9223372036854775807)
ULLONG_MAX	Maximum value for object of type unsigned long long int (18446744073709551615)

Taken from Programming in C,
Kochan

stdbool.h

<stdbool.h> - file contains definitions for working with Boolean variables (type `_Bool`)

Define	Meaning
<code>bool</code>	Substitute name for the basic <code>_Bool</code> data type
<code>true</code>	Defined as 1
<code>false</code>	Defined as 0

Taken from Programming in C,
Kochan

Various functions

By Jason Fedin

Various functions

I wanted to remind you of some of the various functions from the C standard library that we have already addressed

String functions

- to use any of these functions, you need to include the header file `<string.h>`
- `char *strcat (s1, s2)`
 - concatenates the character string `s2` to the end of `s1`, placing a null character at the end of the final string. The function returns `s1`
- `char *strchr (s, c)`
 - searches the string `s` for the first occurrence of the character `c`. If it is found, a pointer to the character is returned; otherwise, a null pointer is returned
- `int strcmp (s1, s2)`
 - compares strings `s1` and `s2` and returns a value less than zero if `s1` is less than `s2`, equal to zero if `s1` is equal to `s2`, and greater than zero if `s1` is greater than `s2`
- `char *strcpy (s1, s2)`
 - copies the string `s2` to `s1`, returning `s1`
- `size_t strlen (s)`
 - returns the number of characters in `s`, excluding the null character

String functions

- `char *strncat (s1, s2, n)`
 - copies s2 to the end of s1 until either the null character is reached or n characters have been copied, whichever occurs first. Returns s1
- `int strncmp (s1, s2, n)`
 - performs the same function as `strcmp`, except that at most n characters from the strings are compared
- `char *strncpy (s1, s2, n)`
 - copies s2 to s1 until either the null character is reached or n characters have been copied, whichever occurs first. Returns s1
- `char *strrchr (s, c)`
 - searches the string s for the last occurrence of the character c. If found, a pointer to the character in s is returned; otherwise, the null pointer is returned
- `char *strstr (s1, s2)`
 - searches the string s1 for the first occurrence of the string s2. If found, a pointer to the start of where s2 is located inside s1 is returned; otherwise, if s2 is not located inside s1, the null pointer is returned
- `char *strtok (s1, s2)`
 - breaks the string s1 into tokens based on delimiter characters in s2

character functions

- to use these character functions, you must include the file `<ctype.h>`
- `isalnum`, `isalpha`, `isblank`, `iscntrl`, `isdigit`, `isgraph`, `islower`, `isspace`, `ispunct`, `isupper`, `isxdigit`
- `int tolower(c)`
 - returns the lowercase equivalent of `c`. If `c` is not an uppercase letter, `c` itself is returned
- `int toupper(c)`
 - returns the uppercase equivalent of `c`. If `c` is not a lowercase letter, `c` itself is returned.

I/O functions

- to use the most common I/O functions from the C library you should include the header file `<stdio.h>`
- included in this file are declarations for the I/O functions and definitions for the names EOF, NULL, stdin, stdout, stderr (all constant values), and FILE
- int fclose (filePtr)
 - closes the file identified by filePtr and returns zero if the close is successful, or returns EOF if an error occurs
- int feof (filePtr)
 - returns nonzero if the identified file has reached the end of the file and returns zero otherwise
- int fflush (filePtr)
 - flushes (writes) any data from internal buffers to the indicated file, returning zero on success and the value EOF if an error occurs

I/O functions

- `int fgetc (filePtr)`
 - returns the next character from the file identified by `filePtr`, or the value `EOF` if an end-of-file condition occurs. (Remember that this function returns an `int`)
- `int fgetpos (filePtr, fpos)`
 - gets the current file position for the file associated with `filePtr`, storing it into the `fpos_t` (defined in `<stdio.h>`) variable pointed to by `fpos`. `fgetpos` returns zero on success, and returns nonzero on failure
- `char *fgets (buffer, i, filePtr)`
 - reads characters from the indicated file, until either `i - 1` characters are read or a newline character is read, whichever occurs first
- `FILE *fopen (fileName, accessMode)`
 - opens the specified file with the indicated access mode
- `int fprintf (filePtr, format, arg1, arg2, ..., argn)`
 - writes the specified arguments to the file identified by `filePtr`, according to the format specified by the character string `format`

I/O functions

- `int fputc (c, filePtr)`
 - writes the value of `c` to the file identified by `filePtr`, returning `c` if the write is successful, and the value `EOF` otherwise
- `int fputs (buffer, filePtr)`
 - writes the characters in the array pointed to by `buffer` to the indicated file until the terminating null character in `buffer` is reached
- `int fscanf (filePtr, format, arg1, arg2, ..., argn)`
 - reads data items from the file identified by `filePtr`, according to the format specified by the character string `format`
- `int fseek (filePtr, offset, mode)`
 - positions the indicated file to a point that is `offset` (a long int) bytes from the beginning of the file, from the current position in the file, or from the end of the file, depending upon the value of `mode` (an integer)
- `int fsetpos (filePtr, fpos)`
 - sets the current file position for the file associated with `filePtr` to the value pointed to by `fpos`, which is of type `fpos_t` (defined in `<stdio.h>`). Returns zero on success, and nonzero on failure

I/O functions

- `long ftell (filePtr)`
 - returns the relative offset in bytes of the current position in the file identified by `filePtr`, or `-1L` on error
- `int printf (format, arg1, arg2, ..., argn)`
 - writes the specified arguments to `stdout`, according to the format specified by the character string. Returns the number of characters written.
- `int remove (fileName)`
 - removes the specified file. A nonzero value is returned on failure
- `int rename (fileName1, fileName2)`
 - renames the file `fileName1` to `fileName2`, returning a nonzero result on failure.
- `int scanf (format, arg1, arg2, ..., argn)`
 - reads items from `stdin` according to the format specified by the string format

Conversion functions

- to use these functions that convert character strings to numbers you must include the header file `<stdlib.h>`
- double `atof (s)`
 - converts the string pointed to by `s` into a floating-point number, returning the result
- int `atoi (s)`
 - converts the string pointed to by `s` into an int, returning the result
- int `atol (s)`
 - converts the string pointed to by `s` into a long int, returning the result
- int `atoll (s)`
 - converts the string pointed to by `s` into a long long int, returning the result

Dynamic Memory functions

- to use these functions that allocate and free memory dynamically you must include the `stdlib.h` header file
- `void *calloc (n, size)`
 - allocates contiguous space for `n` items of data, where each item is `size` bytes in length. The allocated space is initially set to all zeroes. On success, a pointer to the allocated space is returned; on failure, the null pointer is returned
- `void free (pointer)`
 - returns a block of memory pointed to by `pointer` that was previously allocated by a `calloc()`, `malloc()`, or `realloc()` call
- `void *malloc (size)`
 - allocates contiguous space of `size` bytes, returning a pointer to the beginning of the allocated block if successful, and the null pointer otherwise
- `void *realloc (pointer, size)`
 - changes the size of a previously allocated block to `size` bytes, returning a pointer to the new block (which might have moved), or a null pointer if an error occurs

Math Functions

By Jason Fedin

Math functions

- to use common math functions you must include the math.h header file and link to the math library
- double acosh (x)
 - returns the hyperbolic arccosine of x, $x \geq 1$
- double asin (x)
 - returns the arcsine of x as an angle expressed in radians in the range $[-\pi/2, \pi/2]$. x is in the range $[-1, 1]$
- double atan (x)
 - returns the arctangent of x as an angle expressed in radians in the range $[-\pi/2, \pi/2]$
- double ceil (x)
 - returns the smallest integer value greater than or equal to x. Note that the value is returned as a double

Math functions

- `double cos (r)`
 - returns the cosine of r
- `double floor (x)`
 - returns the largest integer value less than or equal to x . Note that the value is returned as a double
- `double log (x)`
 - returns the natural logarithm of x , $x \geq 0$
- `double nan (s)`
 - returns a NaN, if possible, according to the content specified by the string pointed to by s
- `double pow (x, y)`
 - returns xy . If x is less than zero, y must be an integer. If x is equal to zero, y must be greater than zero
- `double remainder (x, y)`
 - returns the remainder of x divided by y

Math functions

- double round (x)
 - returns the value of x rounded to the nearest integer in floating-point format. Halfway values are always rounded away from zero (so 0.5 always rounds to 1.0)
- double sin (r)
 - returns the sine of r
- double sqrt (x)
 - returns the square root of x, $x \geq 0$
- double tan (r)
 - returns the tangent of r
- And so many more
 - complex arithmetic

Utility Functions

By Jason Fedin

Utility functions

- to use these routines, include the header file `<stdlib.h>`
- `int abs (n)`
 - returns the absolute value of its int argument n
- `void exit (n)`
 - terminates program execution, closing any open files and returning the exit status specified by its int argument n
 - `EXIT_SUCCESS` and `EXIT_FAILURE`, defined in `<stdlib.h>`
 - other related routines in the library that you might want to refer to are `abort` and `atexit`
- `char *getenv (s)`
 - returns a pointer to the value of the environment variable pointed to by s, or a null pointer if the variable doesn't exist
 - used to get environment variables

Utility functions

- `void qsort (arr, n, size, comp_fn)`
 - sorts the data array pointed to by the void pointer `arr`
 - there are `n` elements in the array, each `size` bytes in length. `n` and `size` are of type `size_t`
 - the fourth argument is of type “pointer to function that returns `int` and that takes two void pointers as arguments.”
 - `qsort` calls this function whenever it needs to compare two elements in the array, passing it pointers to the elements to compare
- `int rand (void)`
 - returns a random number in the range `[0, RAND_MAX]`, where `RAND_MAX` is defined in `<stdlib.h>` and has a minimum value of 32767
- `void srand (seed)`
 - seeds the random number generator to the unsigned `int` value `seed`
- `int system (s)`
 - gives the command contained in the character array pointed to by `s` to the system for execution, returning a system-defined value
 - `system ("mkdir /usr/tmp/data");`

Assert Library

- the assert library, supported by the assert.h header file, is a small one designed to help with debugging programs
- it consists of a macro named assert()
 - it takes as its argument an integer expression
 - If the expression evaluates as false (nonzero), the assert() macro writes an error message to the standard error stream (stderr) and calls the abort() function, which terminates the program

```
z = x * x - y * y; /* should be + */
```

```
assert(z >= 0); // asserts that z is greater than or equal to 0
```


Other useful header files

- time.h
 - defines macros and functions supporting operations with dates and times
- errno.h
 - defines macros for the reporting of errors
- locale.h
 - defines functions and macros to assist with formatting data such as monetary units for different countries
- signal.h
 - defines facilities for dealing with conditions that arise during program execution, including error conditions
- stdarg.h
 - defines facilities that enable a variable number of arguments to be passed to a function

Further topics of study

By Jason Fedin

Further topics of study

- more on data types
 - defining your own types (typedef)
- more on the preprocessor
 - string concatenation
- more on void*'s
- static libraries and shared objects
- macros
- unions

Further topics of study

- function pointers
- advanced pointers
- variable arguments to functions (variadic functions)
- dynamic linking (dlopen)
- signals, forking and inter-process communication
- threading and concurrency
- sockets

Further topics of study

- setjmp and longjmp for restoring state
- more on memory management and fragmentation
- more on making your program portable
- Interfacing with kernel modules (drivers and ioctls)
- more on compiler and linker flags

Further topics of study

- advanced use of gdb
- profiling and tracing tools (gprof, dtrace, strace)
- memory debugging tools such as valgrind

Course Summary

By Jason Fedin

Topics (cont'd)

- Code Blocks
- Basic Operators – logical, arithmetic and assignment
- Conditional Statements – making decisions (if, switch)
- Repeating code – looping (for, while, and do-while)
- Arrays – defining and initializing, multi-dimensional
- Functions – declaration and use, arguments and parameters, call by value vs. call by reference
- Debugging – call stack, common mistakes, understanding compiler messages
- Structs – initializing, nested structures

Topics (cont'd)

- Character Strings – basics, arrays of chars, character operations
- pointers – definition and use, using with functions and arrays, malloc, pointer arithmetic
- the Preprocessor - #define, #include
- Input and Output – command line arguments, scanf, printf,
- File Input/Output – reading and writing to a file, fgetc, fgets, fputc, fgetc, fseek, etc.
- Standard C Library – string functions, math functions, utility functions, standard header files

Course Outcomes

- you are now able to write beginner C programs
- you are now able to write efficient, high quality C code
 - modular
 - low coupling
 - naming conventions, indentation
- you are now able to find and fix your errors
 - you understand compiler messages
 - you know how to use a debugger
- you now understand fundamental aspects of the C Programming language

CONGRATULATIONS!!!!

do not hesitate to ask questions

provide feedback via the ratings

offer constructive criticism
always looking to improve the course

will make course updates to improve the class